Go for JavaScript Developers

From React and Next.js to Go, and building real applications with Grit

Muke JohnBaptist

Written against Grit v3.403.0 and Go 1.25

Contents

Why Go, when you already ship JavaScript 2
What actually pushed you here 2
The shape of Go, in one page 3
What you give up, stated plainly 3
The one example that explains everything 4
What this book does 5
How to read it 6
Installing Go and your first program 8
Installing 8
Your first program 8
Capital letters are the access modifier 9
Modules: go.mod is your package.json 10
The commands you will actually use 10
Compiling a real binary 11
Variables, types, and the compiler that argues with you 14
Declaring things 14
The types you will actually use 15
Zero values: there is no undefined 16
No truthiness 16
Strings are bytes, and that matters 17
Control flow, briefly 18
Functions, multiple returns, and the end of try/catch 20
Functions 20
Two return values, and why 21
`error` is just an interface 21
Wrapping: adding context without losing the original 22
The argument about verbosity 23
Named returns and `defer` 24
Structs and methods, against objects and classes 28
Declaring and creating 28
Methods: functions with a receiver 29
Value receivers and pointer receivers 29
Composition instead of inheritance 30
Struct tags: metadata for encoders 31
Constructors are just functions 31

Comparing structs 32
Slices: the array that is not an array 36
Arrays have a fixed length, in the type 36
Slices are a view onto an array 36
append, and the thing it does not promise 37
Iterating 38
Passing slices to functions 38
Sorting 39
Maps, and why iteration order is random on purpose 42
Declaring and using 42
Missing keys return the zero value 42
Iteration order is deliberately random 43
Maps are reference types 44
Structs as values, and the thing you cannot do 44
Sets 44
Maps are not safe for concurrent use 45
Pointers, without the fear 48
Everything in Go is passed by value 48
Two operators 48
nil, and the panic you will meet 49
When to use a pointer 50
When not to use a pointer 50
The loop variable, and a trap that Go fixed 51
Pointers to struct fields 51
Packages, modules and imports, against npm and ESM 56
A package is a directory 56
Importing 57
The import path is the module path plus the directory 57
internal/ is a real access rule 58
Naming, and the stutter rule 58
Modules, versions and go.mod 59
Major versions are in the path 60
init, and why to avoid it 60
Interfaces, and why Go's point the other way 64
The mechanical part 64
The direction: interfaces belong to the consumer 65
Keep them small 65
The empty interface, and `any` 66
The nil interface trap 67

Interfaces you will meet constantly 68
Why this matters for testing 68
Designing with errors: what a package should return 72
Three kinds of error, and when to use each 72
Wrap, but not at every level 73
Error strings are a contract, lowercase and unpunctuated 74
Do not return both a value and an error 74
Handle an error once 75
Mapping errors at the boundary 75
panic is not an error 76
defer, panic and recover 80
defer: cleanup that cannot be forgotten 80
The three rules of defer 80
defer can change the return value 82
defer and errors from Close 82
panic: stop, this should not have happened 83
recover: only for a boundary 83
Goroutines, against the event loop 88
The one-keyword version 88
What is actually different 88
Waiting properly: sync.WaitGroup 89
Collecting results 90
Races, and the detector that finds them 91
Bounding how many run at once 92
When not to use a goroutine 93
Channels, and the select statement 96
A channel is a typed pipe 96
Unbuffered channels are a handshake 97
Buffered channels have room 97
Closing, and ranging 97
select: waiting on several channels 98
Timeouts, the most common select 99
Signalling with a done channel 99
The nil channel, and why it is useful 100
Worker pools 100
sync: mutexes, and when a channel is the wrong tool 106
Mutex: one goroutine at a time 106
Embedding the mutex 107
RWMutex: many readers, one writer 107

sync.Once: exactly once, no matter who asks 108
atomic: for single values 108
sync.Map, and why you probably do not want it 109
When to use which 109
Context: cancellation, deadlines, and request scope 114
The problem it solves 114
Creating one 114
Using one 115
Cancellation is cooperative 116
The standard library already honours it 116
Timeouts compose 116
Values, and why to be sparing 117
The conventions 117
Generics, briefly and honestly 122
The syntax 122
Constraints are interfaces 123
The constraints you will actually use 123
Generic types 124
What the standard library gives you 125
When to write your own, and when not to 125
Interfaces are still usually the answer 126
Testing: the runner is already installed 130
The rules 130
Error messages are the whole skill 131
Errorf against Fatalf 131
Table-driven tests 131
Comparing things that are not numbers 132
Helpers and t.Helper 133
Setup and cleanup 133
Parallel tests 134
Testing errors 134
Coverage 135
Benchmarks 135
JSON, struct tags, and the capital letter that bites 140
Encoding and decoding 140
The capital letter rule, again 140
Struct tags in full 141
omitempty, and what it actually means 142

Decoding unknown shapes 142
Decoding is case-insensitive, and lenient 142
Streaming with Encoder and Decoder 143
Times, and the format you will meet 143
Custom marshalling 144
Files, streams and the io interfaces 148
Reading a whole file 148
Streaming line by line 148
os.Open against os.Create 149
Buffer your writes 149
io.Copy, and the pattern it enables 150
Writing testable code against interfaces 151
Paths, portably 152
Temporary files and directories 153
net/http: a server with no framework 158
Five lines 158
The two core types 159
Routing with ServeMux 159
Always configure a Server 160
Middleware is a function that wraps a handler 160
Passing values down: context, not globals 161
Graceful shutdown 162
Testing handlers without a server 163
Talking to a database with database/sql 168
Opening a database 168
The pool is the important part 168
Querying one row 169
Querying many rows 169
Writing, and what you get back 170
Placeholders are not optional 171
NULL, and why your struct fields lie 171
Transactions 172
The N+1 query problem 173
Should you use an ORM? 174
Project: a URL shortener 178
What it does 178
The schema and the store 178
Generating codes 180

Validating the URL 181
The server 182
Running it 186
The tests 186
Configuration, environment and secrets 190
Environment variables, the baseline 190
Parse once, into a struct 190
Fail at startup, not on the first request 192
Secrets 193
Flags, for tools rather than services 194
Building, cross compiling and shipping one binary 198
The build 198
Smaller binaries 198
Embedding the version 199
Cross compiling 199
Containers, in two stages 200
Embedding files into the binary 201
Production flags worth knowing 201
Automating releases 201
The twenty six problems, and what a framework is for 206
Two lists, and they belong to different people 206
The developer's twelve 206
The client's fourteen 207
Why these survive a decade of excellent libraries 208
The question a framework has to answer 208
What this means for the rest of the book 209
grit new: what lands, and the shape of a project 212
Installing 212
Creating a project 212
What arrives 213
The internal packages 213
Running it 214
grit.json, and the seeding that matters 215
Modules you can switch off 215
It is still just Go 216
The architecture: a modular monolith, layer by layer 218
The short answer 218
The request path 218
The rule the layering exists for 219

Where the rules live, concretely 220
The event bus 220
What it deliberately is not 221
Organised by layer, not by feature 222
Can you refactor to something else? 222
The resource, and why it is the unit of generation 224
The claim 224
One command 224
Why one description is the point 225
A field type is a decision made once 225
It is still your code afterwards 226
What it does not generate 227
Field types: what each one decides 230
The argument 230
The full list 230
Text in three sizes 231
Choices, in three shapes 231
Relationships 231
Identity and files 232
Validated strings 232
Numbers with meaning 232
Time, in three precisions 233
Choosing well, in one table 233
Models, migrations and seeding at scale 236
The model 236
Migrations, and the honest problem 237
The refusal that protects you 237
Seeding, and why it is not optional 238
A million rows, with the heap flat 238
Backups, and the half that counts 239
Services: where the rules live, and why not the handler 242
The rule, and the reason 242
What a generated service contains 243
The two whitelists are the chapter 243
Ownership travels in the context 244
Errors, by design 244
Adding your own method 245
Where a rule does not belong in a service 246
Build your own key-value store: a write-ahead log and crash recovery 250

The problem, stated precisely 250
The record format 250
Encoding a record 251
The store 252
Replay, and tolerating a torn tail 253
Writing 254
The one system call that matters 255
Compaction 256

0

Why Go, when you already ship JavaScript

You already build and ship software. You know React, you know Next.js, and you can take an idea to production without asking anybody how. So the first question this book owes you is not "what is Go", it is "why would you spend a month learning a second language when the one you have works".

Here is the honest answer, and then the honest list of what you give up.

What actually pushed you here

Most JavaScript developers do not come to Go out of curiosity. They come because something in their stack got painful in a specific way. Usually one of these four.

The deployment got heavy. Your Next.js app needs Node on the server, a `node_modules` directory with a few hundred megabytes in it, and a build step that has to run somewhere. You want to copy one file onto a server and run it. A compiled Go program is one file. It has no runtime to install, no package directory, and no build step on the target machine. You build it on your laptop and the output is the thing that runs.

The work stopped being request-and-respond. An API that answers a question in forty milliseconds is a perfect job for Node. A process that holds ten thousand open connections, or runs a job queue, or chews through a large file while staying responsive, is where the single-threaded event loop stops being an elegance and starts being a constraint you work around.

You wanted the errors earlier. In JavaScript, a renamed field is found by a user. TypeScript helps enormously and you should keep using it, but it is a layer on top of a language that does not have types, and it disappears at runtime. In Go the types are the language. A program that compiles has already had a whole class of mistake removed from it.

Somebody said the word "backend" and meant it. Payment handling, a scheduler, an audit trail, something that must still be correct at 3am when nobody is watching. Go was built by people who operate systems like that for a living, and it shows in what the language refuses to let you do.

> **IF YOU KNOW JAVASCRIPT**
> None of this makes Go better than JavaScript. It makes it different in ways that matter for one kind of work. You are not replacing React. In the second half of this book you will have a Go backend and a React frontend in the same project, talking to each other with types generated from one definition. That is the shape most people actually want.

The shape of Go, in one page

Before any syntax, here is the character of the language, because it explains every decision you are about to meet.

Go is **small on purpose**. The entire specification is about one hundred pages. There are twenty five keywords. JavaScript has roughly double that and a standard library that has grown for thirty years. You can hold all of Go in your head, which is a design goal rather than an accident, and it is why Go teams onboard people in days rather than weeks.

Go is **explicit on purpose**. There is no implicit type coercion, no `this` that changes meaning depending on how a function was called, no prototype chain to walk. When you read a line of Go, what happens is what is written. This is sometimes tedious and it is never surprising, and after a few weeks you will notice you have stopped debugging a category of problem entirely.

Go is **opinionated about formatting**, which sounds trivial and is not. There is one correct way to format Go code, a tool called `gofmt` produces it, and every Go project on earth uses it. No Prettier config, no debate about semicolons, no `.editorconfig`. The argument is over because the language ended it.

Go **compiles fast**. Not "fast for a compiled language", fast in absolute terms: a medium project builds in a couple of seconds. This matters more than it sounds, because it means the compile step never becomes a reason to avoid checking your work.

What you give up, stated plainly

A book that only lists advantages is marketing. Here is the other side, and you should weigh it before chapter two.

You will write more lines. Go has no `map`, `filter` or `reduce` built into the language the way JavaScript does. Where you write one chained expression, you will often write a four-line loop. Go's position is that the loop is easier to read six months later. You may disagree, and you will still write the loop.

Error handling is verbose. You are about to meet `if err != nil` and you will type it hundreds of times. There is no `try` / `catch` wrapping half a function. Every single oper-

ation that can fail says so, at the point it can fail. This is the single biggest adjustment coming from JavaScript, and chapter four is entirely about making peace with it.

The ecosystem is smaller. npm has over two million packages. Go has far fewer. For web application work this matters less than you would think, because Go's standard library covers an enormous amount of what you would reach for a package to do in Node. But if you need an obscure SDK, check before you commit.

No null safety. Go has `nil`, and a nil pointer dereference will crash your program at runtime exactly like reading a property of `undefined` crashes yours. Go's type system is simpler than TypeScript's, and in this one specific area TypeScript is genuinely ahead.

> **WATCH OUT**
> If someone tells you Go is simply better than JavaScript, stop listening to them. The two languages were built for different jobs by people with different problems. Go's concurrency is better than the event loop for sustained work, and the event loop is a beautiful fit for a UI. Use both.

The one example that explains everything

Here is the same program in both languages: read three URLs, and print how long each took. It is worth studying before you understand any Go syntax, because the difference in shape is the whole argument.

JAVASCRIPT

```
async function timeAll(urls) {
  const results = await Promise.all(
    urls.map(async (url) => {
      const start = Date.now();
      await fetch(url);
      return { url, ms: Date.now() - start };
    })
  );
  for (const r of results) {
    console.log(r.url, r.ms);
  }
}
```

GO

```
func timeAll(urls []string) {
    var wg sync.WaitGroup
    results := make([]result, len(urls))

    for i, url := range urls {
        wg.Add(1)
```

```
        go func() {
            defer wg.Done()
            start := time.Now()
            http.Get(url)
            results[i] = result{url, time.Since(start)}
        }()
    }

    wg.Wait()
    for _, r := range results {
        fmt.Println(r.url, r.ms)
    }
}
```

The JavaScript is shorter and, if you know JavaScript, clearer. Do not let anybody tell you otherwise.

But look at what the Go version is doing. The word `go` in front of that function call starts a **goroutine**: an independent line of execution that the Go runtime schedules onto a real operating system thread. Three URLs here, but the same code with fifty thousand URLs would start fifty thousand goroutines, and Go would handle it, because a goroutine costs about two kilobytes of memory rather than the megabyte an OS thread costs.

The JavaScript version is also concurrent, and for three HTTP requests it is just as fast, because all three are waiting on the network and the event loop is excellent at waiting. The difference appears when the work is not waiting. If each of those tasks had to parse a large document or resize an image, the JavaScript version would do them one at a time on one core, and the Go version would use every core you have.

> **THE MENTAL MODEL SHIFT**
> In JavaScript, concurrency is something you **await**, and parallelism is something you get by starting more processes. In Go, concurrency is something you **start**, with one keyword, and parallelism comes free because the runtime spreads goroutines across cores. Chapter eleven is where this clicks properly. For now, notice only that `go doSomething()` is the entire syntax.

What this book does

This book has two halves and they have different jobs.

Part one through three teach you Go, assuming you know JavaScript and nothing else. Every concept is introduced by naming the JavaScript thing it resembles, then showing exactly where the resemblance breaks. You will build small things constantly:

a CLI tool, a JSON API with no framework at all, a URL shortener backed by a real database. By the end you can read and write Go without this book open.

Part four and five teach you Grit, which is a framework that generates a Go API, a React admin panel and the typed client between them from one description of your data. The reason it comes second is that a framework you cannot read is a liability. Once you know Go, Grit's generated code is just Go you did not have to type, and you can change any of it.

> **WHAT YOU WILL HAVE BUILT BY THE END**
> A command line tool that formats JSON. An HTTP API written with nothing but the standard library. A URL shortener with a SQLite database. A task tracker, a blog with a working admin panel, and an invoicing application with customers, line items and money handled correctly. All of them small, all of them finished, none of them a toy that stops at the interesting part.

How to read it

Type the code. Do not copy and paste it. This is advice everybody gives and almost nobody follows, so here is the specific reason it matters for Go: the compiler's error messages are a large part of how you learn the language, and you only see them if you make mistakes. Typing produces mistakes. Pasting does not.

Run everything. Every code block that is a complete program is marked, and you should run it, break it deliberately, and run it again. Go builds in seconds, so the cost of an experiment is almost nothing.

Do the "Your turn" boxes. They are short, and they are the difference between recognising Go and writing it.

> **YOUR TURN**
> Before the next chapter, answer this for your own situation in two sentences: what is the specific thing in your current stack that made you pick this book up? Write it down. In chapter twenty three there is a list of twenty six problems that every project runs into, and you will want to know which of them is yours.

0

Installing Go and your first program

This chapter gets Go on your machine, gets a program running, and introduces the toolchain. If you have ever set up a Node project you will find this shorter than you expect, because most of the steps you are used to do not exist.

Installing

Go to go.dev/dl and download the installer for your platform. There is no version manager to install first, no equivalent of `nvm`, and for most people there never needs to be. Go's compatibility promise means a program written for Go 1.0 in 2012 still compiles today, so "which version" is rarely a question that costs you an afternoon.

Check it worked:

```
go version
```

You should see something like `go version go1.25.0 windows/amd64`. If the command is not found, the installer did not put Go on your `PATH`; restart your terminal first, because that fixes it more often than anything else.

> **IF YOU KNOW JAVASCRIPT**
> There is no `nvm`, no `.nvmrc`, and no per-project runtime. One Go installation builds every project on your machine. When a project needs a specific version, it says so in one line of `go.mod` and the toolchain downloads that compiler itself.

Your first program

Make a directory and put one file in it.

```
// main.go
package main

import "fmt"

func main() {
    fmt.Println("Hello from Go")
}
```

Run it:

```
go run main.go
```

Four things in that tiny file deserve explanation, because each is a rule you will meet constantly.

`package main` declares which package this file belongs to. Every Go file starts with a package declaration. The name `main` is special: it means this package builds into a runnable program rather than a library.

`import "fmt"` pulls in the formatting package from the standard library. Note there is no `npm install`. `fmt` ships with Go, as does an HTTP server, a JSON encoder, a test runner, a crypto suite and a templating engine.

`func main()` is the entry point. A program starts here. There is exactly one `main` function in a program.

`fmt.Println` prints a line. Capital `P` matters enormously, and it is the next section.

Capital letters are the access modifier

This is the first genuinely surprising rule in Go, and it catches everybody.

An identifier starting with a capital letter is exported. One starting with a lowercase letter is private to its package. That is the entire visibility system. There is no `public`, no `private`, no `export` keyword.

```
package money

// Convert is callable from other packages, because it is capitalised.
func Convert(cents int) string { ... }

// round is not. Only code inside package money can call it.
func round(f float64) int { ... }
```

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript you choose what leaves a module with `export`. In Go the name itself decides. `fmt.Println` works because `Println` is capitalised inside package `fmt`; if it were `println` you could not reach it. This applies to functions, types, struct fields, constants and variables alike.

> **WATCH OUT**
> This bites hardest with struct fields and JSON. A lowercase field is invisible to the JSON encoder, because the encoder lives in another package and genuinely cannot see it.

A struct whose fields are all lowercase encodes to `{}`, with no error and no warning. Chapter sixteen covers this properly, but remember the symptom now: empty JSON usually means lowercase fields.

Modules: go.mod is your package.json

A single file is fine for a demo. A real project is a **module**. Initialise one:

```
mkdir hello && cd hello
go mod init github.com/yourname/hello
```

That writes `go.mod`:

```
module github.com/yourname/hello

go 1.25
```

That is the whole file. Compare it to a `package.json` for an equivalent project and notice what is missing: no `scripts`, no `devDependencies`, no `main`, no `type: module`. The module path doubles as the import path, which is why it looks like a URL: if you push this to GitHub, anybody can import it at exactly that address with no registry publish step.

Add a dependency and `go.mod` grows a line:

```
go get github.com/google/uuid
```

```
module github.com/yourname/hello

go 1.25

require github.com/google/uuid v1.6.0
```

IF YOU KNOW JAVASCRIPT

`go.sum` is your lockfile, equivalent to `package-lock.json`. Commit it. There is no `node_modules`: downloaded packages go to a shared module cache outside your project, and the compiler links what it needs directly into the binary. Deleting a project directory does not take 300MB of dependencies with it, because they were never there.

The commands you will actually use

Six commands cover almost everything.

`go run .`	Compile and run the current package, without keeping the binary
`go build`	Compile to an executable in the current directory
`go test ./...`	Run every test in this module
`go mod tidy`	Add what is imported, remove what is not. Run it often
`gofmt -w .`	Format every file. Your editor probably does this on save
`go vet ./...`	Report suspicious code the compiler allows

The `./...` pattern means "this directory and everything under it", and you will type it constantly.

Compiling a real binary

This is the part that sells Go to people who have deployed Node.

```
go build -o hello .
./hello
```

You now have one executable file. It contains your code, every dependency, and the Go runtime. The target machine needs nothing installed. You can also build for a platform you are not on:

```
GOOS=linux GOARCH=amd64 go build -o hello-linux .
```

That produces a Linux binary from a Mac or Windows laptop, with no container, no cross-compilation toolchain and no configuration. Copy it to a Linux server and it runs.

PROJECT 1: A GREETING WITH ARGUMENTS

Write a program that takes a name as a command line argument and greets it, falling back to "world" when none is given. You need `os.Args`, which is a slice of strings where `os.Args[0]` is the program name itself.

```
package main

import (
    "fmt"
    "os"
)

func main() {
    name := "world"
    if len(os.Args) > 1 {
        name = os.Args[1]
```

```
    }
    fmt.Printf("Hello, %s\n", name)
}
```

Build it, run `./hello Ada`, then run it with no argument. Then break it deliberately: change `len(os.Args) > 1` to `len(os.Args) > 0` and run it with no argument to see what a real Go panic looks like. Reading panics early is worth more than avoiding them.

YOUR TURN

Extend the greeting to accept several names and greet each on its own line. You will need a `for` loop over a slice, which looks like `for _, n := range os.Args[1:] { ... }`. The next chapter explains every piece of that line, so guessing now is fine and the compiler will tell you when you are wrong.

0

Variables, types, and the compiler that argues with you

This is the chapter where Go feels most foreign, and it is worth knowing why in advance. In JavaScript, a variable is a box that holds anything and can hold something else tomorrow. In Go, a variable has one type forever, decided when it is created, and the compiler refuses to build your program if you break that promise.

That refusal is the feature. Work with it for a week and you will notice a whole category of bug has stopped happening to you.

Declaring things

Go has two ways to make a variable, and you will use one of them ninety percent of the time.

```
var name string = "Ada"   // the long form, type stated
var age = 36              // type inferred, still a var
// short form: declare and infer, inside functions
name := "Ada"
```

The `:=` operator is the one you want. It declares a new variable and infers its type from the value on the right. It only works inside a function, which is why package level variables use `var`.

> **IF YOU KNOW JAVASCRIPT**
> `:=` is closest to `let` in that it creates a new binding. There is no equivalent of `const` for variables created with `:=`. Go's `const` exists, but it is only for values known at compile time: numbers, strings, booleans. You cannot declare a constant struct or slice. If you want "do not reassign this", Go's answer is to write a small enough function that it is obvious.

An unused variable is a **compile error**, not a warning.

```
func main() {
    count := 10
```

```
    fmt.Println("hello")
}
// ./main.go:4:5: declared and not used: count
```

This feels hostile on day one and reasonable by day ten. Dead variables are usually the residue of a half-finished edit, and Go would rather stop you than let the residue accumulate. The same rule applies to unused imports.

> **WATCH OUT**
> The escape hatch is `_`, the blank identifier, which means "I know there is a value here and I am deliberately ignoring it". You will use it constantly in loops and multiple returns. Do not reach for it to silence a real warning: `_ = count` compiles and tells the next reader nothing.

The types you will actually use

Go has a lot of numeric types. You will use about four of them.

Type	JavaScript	Notes
`string`	`string`	Immutable, UTF-8 bytes. Indexing gives a byte, not a character
`int`	`number`	Whole numbers. 64 bit on any machine you own
`float64`	`number`	Decimals. The default for a fractional literal
`bool`	`boolean`	`true` or `false`. No truthiness, ever
`byte`	-	Alias for `uint8`. A byte of data
`rune`	-	Alias for `int32`. One Unicode code point

The one to notice is that JavaScript's single `number` splits into `int` and `float64`, and **Go will not convert between them for you**.

```
count := 10          // int
price := 2.5         // float64
total := count * price
// invalid operation: count * price (mismatched types int and float64)
```

You must say what you mean:

```
total := float64(count) * price   // 25
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript coerces almost anything: `"3" * 2` is `6`, `[] + {}` is a string, and `0.1 + 0.2` is famously not `0.3`. Go does none of this. There is no implicit conversion between any two types, not even `int` and `int64`. Every conversion is written out, which is more typing and removes an entire family of surprise.

Zero values: there is no undefined

Every type in Go has a **zero value**, and a declared variable always has it. There is no `undefined`, and no uninitialised memory.

`string`	`""`
`int`, `float64`	`0`
`bool`	`false`
pointers, slices, maps, interfaces, functions	`nil`

```
var name string
var count int
var ready bool
fmt.Println(name == "", count == 0, ready == false) // true true true
```

This is genuinely useful. A struct you create is immediately valid, with every field at a sensible default, and you never check whether a field "exists".

> **WATCH OUT**
> It is also the one place zero values hurt: you cannot tell "the caller set this to zero" apart from "the caller did not set it". If that distinction matters, for example a form field where empty and absent mean different things, you need a pointer (`*int`, where `nil` means absent) or a dedicated type. Chapter eight covers pointers properly. This problem is real, and it is the usual reason you see `*string` in a struct.

No truthiness

An `if` in Go takes a `bool`. Not a value that is bool-ish. A `bool`.

JAVASCRIPT

```
if (name) {
  greet(name);
}
if (items.length) {
  render(items);
}
```

GO

```go
if name != "" {
    greet(name)
}
if len(items) > 0 {
    render(items)
}
```

More characters, and no memorising which values are falsy. There is no `==` and `===` distinction either, because there is only one `==` and it does not coerce.

Strings are bytes, and that matters

A Go string is a read-only slice of bytes holding UTF-8. Indexing it gives you a byte.

```go
s := "héllo"
fmt.Println(len(s))     // 6, not 5: é takes two bytes
fmt.Println(s[1])       // 195, a byte value, not "é"
```

To work with characters, range over the string, which decodes runes:

```go
for i, r := range "héllo" {
    fmt.Printf("%d: %c\n", i, r)
}
// 0: h
// 1: é      <- index jumps to 3 next, because é was two bytes
// 3: l
// 4: l
// 5: o
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript strings are UTF-16 and have the same class of problem: `"👍".length` is `2`. Both languages make you think about encoding the moment you leave ASCII. Go is at least explicit that you are holding bytes.

Strings are immutable, so building one in a loop with `+=` allocates a new string every time. For anything beyond a few concatenations use a builder:

```go
var b strings.Builder
for _, word := range words {
    b.WriteString(word)
    b.WriteString(" ")
}
result := b.String()
```

Control flow, briefly

`if` and `for` are almost familiar, with two differences worth stating.

There are no parentheses around the condition, and braces are mandatory even for one line.

```
if count > 10 {
    fmt.Println("plenty")
} else if count > 0 {
    fmt.Println("some")
} else {
    fmt.Println("none")
}
```

`for` is the only loop keyword. It covers every case JavaScript spreads across `for`, `while` and `for...of`.

```
for i := 0; i < 10; i++ { }      // classic
for count < 10 { }               // while
for { }                          // infinite
for i, v := range items { }      // for...of, with the index
```

An `if` can carry a statement before the condition, scoped to the `if`. You will see this constantly in real code:

```
if err := doSomething(); err != nil {
    return err
}
// err does not exist here, which is the point
```

IF YOU KNOW JAVASCRIPT

`switch` in Go needs no `break`: cases do not fall through. You can switch on no value at all, which makes it a tidy replacement for a long `else if` chain:

```
switch {
case score > 90:
    grade = "A"
case score > 80:
    grade = "B"
default:
    grade = "C"
}
```

PROJECT 2: A TEMPERATURE TABLE

Write a program that prints a table converting Celsius to Fahrenheit from minus 40 to 100 in steps of 20. The formula is `f = c*9/5 + 32`.

```
package main

import "fmt"

func main() {
    fmt.Printf("%8s %8s\n", "C", "F")
    for c := -40; c <= 100; c += 20 {
        f := float64(c)*9/5 + 32
        fmt.Printf("%8d %8.1f\n", c, f)
    }
}
```

Two things to notice. `float64(c)` is required: without it you get integer division and wrong answers for anything not divisible by five, which is the bug this chapter exists to prevent. And `%8.1f` means "right align in eight columns, one decimal place", which is how Go does string formatting.

Now break it on purpose: remove the `float64(...)` conversion and read the error. Then change it to `c*9/5` with `c` still an `int` and see that it compiles but produces wrong numbers. The compiler catches the type error. It cannot catch the maths error, and knowing which is which is the skill.

YOUR TURN

Add a third column showing Kelvin (`k = c + 273.15`), and right align the header over it. Then make the step size a variable at the top of `main` and confirm the table still lines up when you change it to 10.

0

Functions, multiple returns, and the end of try/catch

This is the chapter that decides whether you get on with Go. Error handling is where the language is most unlike JavaScript, it is the thing newcomers complain about loudest, and the complaint is half right. Let us do the mechanics first and the argument second.

Functions

```
func add(a int, b int) int {
    return a + b
}

func add(a, b int) int {   // same thing, shared type
    return a + b
}
```

The type comes **after** the name, which reads backwards for about a day and then stops mattering. The reason is that it makes complicated declarations read left to right instead of spiralling outwards, which anyone who has read a C function pointer declaration will appreciate.

Functions are values, so the things you do in JavaScript still work:

```
double := func(n int) int { return n * 2 }

func apply(nums []int, f func(int) int) []int {
    out := make([]int, len(nums))
    for i, n := range nums {
        out[i] = f(n)
    }
    return out
}

apply([]int{1, 2, 3}, double)  // [2 4 6]
```

> **IF YOU KNOW JAVASCRIPT**

That `apply` is `Array.prototype.map`, written by hand. Go had no generics until 1.18 and still has no `map`, `filter` or `reduce` in the standard library for slices. You write the loop. Go's argument is that a three-line loop needs no explanation and a chain of five callbacks sometimes does. You are allowed to find this annoying.

Two return values, and why

A Go function can return several values, and this is not a curiosity: it is the foundation of the error system.

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("divide by zero")
    }
    return a / b, nil
}
```

The caller gets both:

```
result, err := divide(10, 2)
if err != nil {
    fmt.Println("failed:", err)
    return
}
fmt.Println(result)
```

The convention is absolute, across the standard library and every package you will use: **the error is the last return value, and you check it immediately**.

IF YOU KNOW JAVASCRIPT

JavaScript has one return value, so a function that can fail either throws or returns something like `{ data, error }`. Go makes the second shape the language default. If you have written Go-style error handling in TypeScript with a `Result` type, or used Rust, you already have the mental model.

`error` is just an interface

There is nothing magic about errors. `error` is an interface with one method:

```
type error interface {
    Error() string
}
```

Anything with an `Error() string` method is an error. That means you can make your own, carrying whatever fields you want:

```
type ValidationError struct {
    Field   string
    Message string
}

func (e ValidationError) Error() string {
    return e.Field + ": " + e.Message
}

func validate(age int) error {
    if age < 0 {
        return ValidationError{
            Field: "age", Message: "must not be negative"}
    }
    return nil
}
```

Wrapping: adding context without losing the original

A bare error from deep in a call stack is useless on its own. `fmt.Errorf` with the `%w` verb wraps one error inside another, keeping both.

```
func loadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("loading config from %s: %w", path, err)
    }
    ...
}
```

The message becomes a trail that reads from the outside in:

```
loading config from /etc/app.json: open /etc/app.json:
  no such file or directory
```

Use `%w`, not `%v`. `%v` formats the error as text and throws the original away, and then both of the checks below stop working.

Two functions read a wrapped chain. `errors.Is` asks "is this error, anywhere in the chain, that specific one":

```
if errors.Is(err, os.ErrNotExist) {
    // the file was missing, however deeply that was wrapped
}
```

`errors.As` asks "is there an error of this type in the chain", and gives it to you with its fields intact:

```
var ve ValidationError
if errors.As(err, &ve) {
    fmt.Println("bad field:", ve.Field)
}
```

WATCH OUT

`err != nil` is the only check that matters, and comparing an error to `nil` has one notorious trap. A function returning a **concrete** error type rather than the `error` interface can produce an error value that is not `nil` even when it holds nothing:

```
func bad() error {
    var e *ValidationError   // nil pointer
    // Not nil: a non-nil interface holding a nil pointer.
    return e
}
```

The fix is a rule: **always declare the return type as `error`, and return a literal `nil`, never a typed nil variable.** This catches experienced people and it is worth reading twice.

The argument about verbosity

Here is the complaint, stated fairly. This:

JAVASCRIPT

```
async function setup() {
  try {
    const cfg = await loadConfig();
    const db = await connect(cfg.dsn);
    await migrate(db);
    return db;
  } catch (e) {
    throw new Error(`setup failed: ${e.message}`);
  }
}
```

GO

```
func setup() (*DB, error) {
    cfg, err := loadConfig()
    if err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    db, err := connect(cfg.DSN)
```

```
    if err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    if err := migrate(db); err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    return db, nil
}
```

The Go is twice as long. That is not a matter of opinion.

What you get for it is that every failure point is visible at the point it can fail. In the JavaScript, which of the three awaited calls threw? You cannot tell from the code, and the catch block treats all three identically whether or not that is what you wanted. In the Go, each failure is a separate decision, and you could handle the migration failure differently from the connection failure by changing one block.

The honest summary: `try` / `catch` is better when you want to treat a whole region of code as one unit that either works or does not. Explicit returns are better when failures deserve individual handling. Most real backend code is the second kind more often than people expect, and Go bet the language on it.

> **WHAT YOU LOSE, AND SHOULD KNOW ABOUT**
>
> In JavaScript an uncaught exception unwinds to the nearest handler, so a forgotten `catch` still surfaces somewhere. In Go a forgotten error check silently continues with a zero value. Nothing warns you.
>
> This is a real hazard and the tooling answers it: `errcheck`, bundled into `golangci-lint`, fails the build on an unchecked error. Install it early. It is the one linter that is not optional.

Named returns and `defer`

A function can name its return values, which is occasionally useful and frequently overused:

```
func split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return          // a "naked" return, sending back x and y
}
```

Use naked returns only in short functions. In a long one, a bare `return` forces the reader to scroll up to learn what is being returned.

Named returns do earn their place with `defer`, which schedules a call to run when the surrounding function exits, no matter how it exits:

```
func process(path string) (err error) {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()     // runs on every return path below

    // ... many returns, all of which close the file
    return nil
}
```

`defer f.Close()` immediately after a successful open is one of the most common patterns in Go, and it is how the language avoids the cleanup bugs that `try` / `finally` exists to solve.

WATCH OUT

Deferred calls run in last-in, first-out order, and their **arguments** are evaluated immediately, not at exit:

```
for i := 0; i < 3; i++ {
    defer fmt.Println(i)   // prints 2, 1, 0
}
```

And a `defer` inside a loop does not run at the end of each iteration, it runs when the **function** returns. Opening a thousand files in a loop with `defer f.Close()` keeps all thousand open until the function ends. Put the body in its own function if you need per-iteration cleanup.

PROJECT 3: A WORD COUNTER THAT HANDLES FAILURE

Write a program that takes a filename, counts the words in it, and reports errors properly rather than crashing.

```
package main

import (
    "fmt"
    "os"
    "strings"
)

func countWords(path string) (int, error) {
    data, err := os.ReadFile(path)
```

```
    if err != nil {
        return 0, fmt.Errorf("counting words in %s: %w", path, err)
    }
    return len(strings.Fields(string(data))), nil
}

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: wc <file>")
        os.Exit(1)
    }
    n, err := countWords(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
    fmt.Printf("%d words\n", n)
}
```

Run it on a real file, then on one that does not exist, and read the wrapped message. Notice `os.Exit(1)` and writing to `os.Stderr`: a command line tool that reports failure on stdout with exit code zero is a tool that breaks every script that uses it.

`strings.Fields` splits on any run of whitespace, which is the right behaviour here and is worth remembering: it is not the same as `strings.Split(s, " ")`, which gives empty strings between consecutive spaces.

YOUR TURN

Add a check that reports a clearer message when the path is a directory rather than a file. `os.ReadFile` returns an error you can inspect; try `errors.Is(err, syscall.EISDIR)` on Unix, or simply call `os.Stat` first and check `info.IsDir()`, which is portable and is the version you should probably ship.

0

Structs and methods, against objects and classes

A JavaScript object is a bag of keys you can add to at any time. A Go struct is a fixed set of named fields with fixed types, decided when you declare the type and never changing. Everything else in this chapter follows from that one difference.

Declaring and creating

```
type User struct {
    ID    int
    Name  string
    Email string
    Admin bool
}
```

There are four ways to make one, and you will see all four in real code.

```
// 1. Field names. Use this one. It survives a field being added.
u := User{ID: 1, Name: "Ada", Email: "ada@example.com"}

// 2. Positional. Fragile: adding a field breaks every call site.
u := User{1, "Ada", "ada@example.com", false}

// 3. Zero value, then assign.
var u User
u.Name = "Ada"

// 4. new(), giving a pointer to a zero value.
u := new(User)   // *User
```

Notice what is missing from the first example: `Admin` was not set, so it is `false`. There is no `undefined`. Every field always has a value, and a struct you create is immediately complete.

> **IF YOU KNOW JAVASCRIPT**

> The closest JavaScript equivalent is a class with a constructor that sets every property. The difference is that you cannot add a field later. `u.Age = 36` on a `User` with no `Age` field is a compile error, not a new property. This is the whole point: the shape of your data is checked, everywhere, for free.

Methods: functions with a receiver

Go has no classes. You attach behaviour to a type by declaring a function with a **receiver**, the parameter in parentheses before the name.

```
func (u User) DisplayName() string {
    if u.Name == "" {
        return u.Email
    }
    return u.Name
}

fmt.Println(u.DisplayName())
```

`u User` is the receiver. It behaves like a parameter, because that is exactly what it is: `u.DisplayName()` is sugar for passing `u` as the first argument.

> **IF YOU KNOW JAVASCRIPT**
> There is no `this`, and that removes an entire class of JavaScript bug. The receiver is a normal named parameter, bound at the call site, and it cannot be rebound by how the function was called. No `.bind(this)`, no arrow function workaround, no `const self = this`.

Value receivers and pointer receivers

This is the first thing in Go that genuinely trips people up, and it is worth getting right now rather than debugging later.

A **value receiver** gets a copy. Changes to it do not affect the original.

```
func (u User) Rename(name string) {
    u.Name = name       // modifies the copy, then throws it away
}

u := User{Name: "Ada"}
u.Rename("Grace")
fmt.Println(u.Name)     // still "Ada"
```

A **pointer receiver** gets the address, so changes stick.

```
func (u *User) Rename(name string) {
    u.Name = name       // modifies the original
}

u := User{Name: "Ada"}
u.Rename("Grace")
fmt.Println(u.Name)     // "Grace"
```

You do not write `(&u).Rename(...)`. Go inserts the address-of automatically when the method needs a pointer and you have an addressable value.

> **WATCH OUT**
> The rule in practice: **if a method modifies the receiver, it must be a pointer receiver.** A value receiver that assigns to a field compiles happily and silently does nothing, which is one of the few truly quiet bugs in Go.
>
> The second rule: **be consistent within a type.** If any method on `User` takes a pointer, give them all pointers. Mixing the two is legal and makes the rules about which calls are addressable surface at the worst moment.

A practical reason to prefer pointer receivers even when not mutating: a value receiver copies the whole struct on every call. For a struct with twenty fields called in a loop, that is real work for no benefit.

Composition instead of inheritance

Go has no `extends`. It has **embedding**, which looks similar and behaves differently in one important way.

```
type Timestamps struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

type Post struct {
    Timestamps          // embedded: no field name
    Title string
    Body  string
}

p := Post{Title: "Hello"}
p.CreatedAt = time.Now()        // promoted, as if declared on Post
fmt.Println(p.Timestamps.CreatedAt)  // the long way, same field
```

The embedded type's fields and methods are **promoted** to the outer type. If `Timestamps` had a method `Age()`, then `p.Age()` would work.

> **IF YOU KNOW JAVASCRIPT**
> This resembles `class Post extends Timestamps`, and the difference matters: there is no polymorphism here. A `Post` is **not** a `Timestamps` and cannot be passed where one is expected. Embedding is a convenience for forwarding fields and methods, not a type relationship. In Go, the thing that lets one type stand in for another is an interface, which is chapter ten.

Struct tags: metadata for encoders

A field can carry a string of metadata, used by libraries through reflection. You will meet this constantly, because it is how Go does JSON.

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
    Admin bool   `json:"-"`                  // never encoded
    Bio   string `json:"bio,omitempty"`      // dropped when empty
}
```

Without tags, `json.Marshal` uses the Go field names, so you get `{"ID":1, "Name":"Ada"}`. With them you get the lowercase JSON your frontend expects.

> **WATCH OUT**
> Those backticks are a raw string literal, and the format inside is strict: `json:"name"` with no space after the colon. `json: "name"` silently does nothing, the tag is ignored, and the field encodes as `Name`. There is no error. If a field is coming out with the wrong key, check for that space first.

Constructors are just functions

Go has no `constructor` keyword. The convention is a package-level function named `New` or `NewThing` that returns the type.

```
func NewUser(name, email string) (*User, error) {
    if email == "" {
        return nil, errors.New("email is required")
    }
    return &User{
        Name:  name,
        Email: email,
    }, nil
}
```

Returning `*User` rather than `User` is the usual choice, because the caller almost always wants to pass it around without copying, and because it lets you return `nil` on failure.

> **IF YOU KNOW JAVASCRIPT**
> A JavaScript constructor cannot fail cleanly: it either throws or returns a half-built object. A Go constructor is an ordinary function, so it returns an error like everything else, and you cannot get a `*User` back unless it succeeded.

Comparing structs

Structs are comparable with `==` when all their fields are, and the comparison is by value, field by field.

```
a := User{ID: 1, Name: "Ada"}
b := User{ID: 1, Name: "Ada"}
fmt.Println(a == b)     // true
```

That is deep equality for free, and it is a genuine surprise if you are used to `a === b` on objects being an identity check.

> **WATCH OUT**
> It stops working the moment a struct contains a slice, a map or a function, because those are not comparable. The compiler tells you. For those, use `reflect.DeepEqual`, or in tests `google/go-cmp`, which prints a readable diff instead of just `false`.

> **PROJECT 4: A CONTACT BOOK IN MEMORY**
> Build a small type with behaviour, and exercise value and pointer receivers.

```
package main

import (
    "errors"
    "fmt"
    "strings"
)

type Contact struct {
    Name  string
    Email string
    Tags  []string
}

// Value receiver: reads only, so a copy is fine.
func (c Contact) Display() string {
    if len(c.Tags) == 0 {
```

```
        return c.Name
    }
    return fmt.Sprintf("%s [%s]", c.Name, strings.Join(c.Tags, ", "))
}

// Pointer receiver: it modifies, so it must be a pointer.
func (c *Contact) Tag(tag string) {
    for _, t := range c.Tags {
        if t == tag {
            return // already there, do nothing
        }
    }
    c.Tags = append(c.Tags, tag)
}

type Book struct {
    contacts []Contact
}

func (b *Book) Add(c Contact) error {
    if c.Email == "" {
        return errors.New("contact needs an email")
    }
    b.contacts = append(b.contacts, c)
    return nil
}

func (b *Book) Find(email string) (*Contact, bool) {
    for i := range b.contacts {
        if b.contacts[i].Email == email {
            return &b.contacts[i], true   // address of the real one
        }
    }
    return nil, false
}

func main() {
    var book Book
    ada := Contact{Name: "Ada", Email: "ada@example.com"}
    if err := book.Add(ada); err != nil {
        fmt.Println(err)
        return
    }

    if c, ok := book.Find("ada@example.com"); ok {
        c.Tag("maths")
        c.Tag("maths") // ignored, already present
    }

    for _, c := range book.contacts {
        fmt.Println(c.Display())  // Ada [maths]
```

```
    }
}
```

The subtle line is `return &b.contacts[i], true`. Ranging with `for _, c := range b.contacts` would give you a **copy** of each contact, and tagging that copy would change nothing. Indexing with `i` and taking the address reaches the real element. Change it to the copying version and watch the tag vanish: that is the lesson of this chapter in one experiment.

YOUR TURN

Add a `Remove(email string) bool` method to `Book`. You will need to build a new slice or use the `slices.Delete` helper from the standard library. Then add a `Count()` method and decide, with a reason you can say out loud, whether it takes a value or a pointer receiver.

0

Slices: the array that is not an array

Go has arrays and it has slices. You will use slices for essentially everything, and the distinction matters exactly twice: when you declare one, and when a surprising bug appears. This chapter covers both.

Arrays have a fixed length, in the type

```
var a [3]int        // an array of exactly three ints
a[0] = 1
fmt.Println(len(a)) // 3
```

The length is **part of the type**. `[3]int` and `[4]int` are different types, and a function taking `[3]int` will not accept `[4]int`. Arrays are also copied whole on assignment. This is almost never what you want, which is why arrays are rare in Go code.

Slices are a view onto an array

```
nums := []int{1, 2, 3}      // no length in the brackets: a slice
nums = append(nums, 4)
fmt.Println(len(nums))      // 4
```

A slice is a small three-field value: a pointer to an underlying array, a length, and a capacity. That is the whole model, and understanding it explains every slice surprise you will ever hit.

`len(s)`	How many elements you can index right now
`cap(s)`	How many fit before the underlying array must be reallocated

```
s := make([]int, 2, 5)   // len 2, cap 5
fmt.Println(len(s), cap(s))  // 2 5
s = append(s, 10)            // fits in the existing array
fmt.Println(len(s), cap(s))  // 3 5
```

`make([]T, len, cap)` creates one with room reserved. When you know roughly how many elements are coming, say so: it avoids repeated reallocation.

IF YOU KNOW JAVASCRIPT
A JavaScript array handles all of this invisibly. `push` grows it and you never think about capacity. Go exposes the machinery, which costs you some attention and buys you control over allocation, which matters a great deal in a hot loop and not at all in a handler that runs once per request.

append, and the thing it does not promise

`append` returns a slice. **You must use the return value.**

```
nums := []int{1, 2, 3}
append(nums, 4)          // compile error: result not used
nums = append(nums, 4)   // correct
```

The reason is that `append` may or may not allocate a new array. If there is spare capacity it writes in place and returns a slice with a longer length. If there is not, it allocates a bigger array, copies everything over, and returns a slice pointing at the **new** array. The caller's original slice still points at the old one.

This is where the famous slice bug lives.

```
a := []int{1, 2, 3, 4, 5}
b := a[1:3]              // [2 3], sharing a's array
b[0] = 99
fmt.Println(a)           // [1 99 3 4 5]  <- a changed
```

Slicing does not copy. `b` is a window onto the same memory. Writing through one is visible through the other.

WATCH OUT
It gets worse with `append`, because whether you corrupt the original depends on capacity, which depends on history:

```
a := []int{1, 2, 3, 4, 5}
b := a[1:3]              // len 2, cap 4 (to the end of a)
b = append(b, 99)        // fits in cap, writes into a's array
fmt.Println(a)           // [1 2 3 99 5]  <- clobbered a[3]
```

The fix, when you want a genuinely independent slice, is a three-index slice that caps capacity, or an explicit copy:

```
b := a[1:3:3]            // cap is now 2, so append must allocate
// or
b := slices.Clone(a[1:3])
```

This bug is rare in everyday handler code and vicious when it appears. The tell is "a value changed and nothing in my code assigned to it".

Iterating

```
for i, v := range nums {
    fmt.Println(i, v)
}

for _, v := range nums {    // index discarded
    fmt.Println(v)
}

for i := range nums {       // value discarded
    nums[i] *= 2
}
```

`range` gives you a **copy** of each element. For ints that is irrelevant. For structs it is the bug from the previous chapter: modifying `v` changes nothing. Use the index when you need to write.

IF YOU KNOW JAVASCRIPT

Go has no `map`, `filter`, `reduce`, `find` or `some` for slices in the language. Since Go 1.21 the `slices` package covers some of it:

```
slices.Contains(nums, 3)        // bool
slices.Index(nums, 3)           // position or -1
slices.Sort(nums)               // in place
slices.Reverse(nums)            // in place
slices.Max(nums)                // and Min
```

For anything else, write the loop. A filter is four lines:

```
var active []User
for _, u := range users {
    if u.Active {
        active = append(active, u)
    }
}
```

Passing slices to functions

A slice is cheap to pass: you are copying three words, not the data. That means a function **can** modify the caller's elements.

```
func zero(nums []int) {
    for i := range nums {
        nums[i] = 0
    }
}

nums := []int{1, 2, 3}
zero(nums)
fmt.Println(nums)   // [0 0 0]
```

But it cannot change the caller's **length**, because the slice header itself was copied:

```
func addOne(nums []int) {
    nums = append(nums, 1)   // changes the local copy's header only
}

nums := []int{1, 2, 3}
addOne(nums)
fmt.Println(len(nums))   // still 3
```

The convention, and the reason `append` works the way it does: a function that grows a slice returns it.

```
func addOne(nums []int) []int {
    return append(nums, 1)
}
nums = addOne(nums)
```

Sorting

```
people := []User{{Name: "Zoe"}, {Name: "Ada"}}

slices.SortFunc(people, func(a, b User) int {
    return strings.Compare(a.Name, b.Name)
})
```

The comparison function returns a negative number, zero, or a positive number, the same contract as JavaScript's `Array.prototype.sort` comparator. For stable sorting use `slices.SortStableFunc`.

> **PROJECT 5: WORD FREQUENCY, TOP TEN**
> Read a file, count how often each word appears, and print the ten most common. This uses slices, sorting and a map together, and it is a real tool you can point at a book.

```
package main

import (
    "fmt"
    "os"
    "slices"
    "strings"
)

type pair struct {
    word  string
    count int
}

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: freq <file>")
        os.Exit(1)
    }

    data, err := os.ReadFile(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }

    counts := map[string]int{}
    for _, w := range strings.Fields(string(data)) {
        w = strings.ToLower(strings.Trim(w, ".,!?;:\"'()"))
        if w != "" {
            counts[w]++
        }
    }

    pairs := make([]pair, 0, len(counts))
    for w, c := range counts {
        pairs = append(pairs, pair{w, c})
    }

    slices.SortFunc(pairs, func(a, b pair) int {
        if b.count != a.count {
            return b.count - a.count       // count descending
        }
        return strings.Compare(a.word, b.word) // then alphabetical
    })

    for _, p := range pairs[:min(10, len(pairs))] {
        fmt.Printf("%6d  %s\n", p.count, p.word)
    }
}
```

Three things worth noticing. `make([]pair, 0, len(counts))` reserves capacity up front because we know exactly how many there will be. The tie-break on word makes the output deterministic, which matters because map iteration order is random and without it two runs would disagree. And `min` is a builtin since Go 1.21, so `pairs[:min(10, len(pairs))]` does not panic on a file with fewer than ten distinct words.

YOUR TURN

Add a flag so the user can ask for a different number of results, for example `freq book.txt 25`. Then add a stop-word list so "the", "a" and "and" are excluded, and decide whether a slice or a map is the better structure for holding those stop words. The next chapter will tell you whether you chose right.

0

Maps, and why iteration order is random on purpose

A Go map is a hash table with typed keys and typed values. It is the closest thing Go has to a JavaScript object used as a dictionary, and it differs in four ways that each cause a specific bug the first time you meet them.

Declaring and using

```
ages := map[string]int{
    "ada":   36,
    "grace": 45,
}

ages["alan"] = 41
fmt.Println(ages["ada"])    // 36
fmt.Println(len(ages))      // 3
delete(ages, "alan")
```

The type is `map[KeyType]ValueType`. Keys must be comparable, so strings, ints, booleans and structs of those are fine; slices, maps and functions are not.

WATCH OUT

A `nil` map is readable but **not writable**, and this is the first map bug everyone hits:

```
var m map[string]int      // nil
fmt.Println(m["a"])       // 0, fine
fmt.Println(len(m))       // 0, fine
m["a"] = 1                // panic: assignment to entry in nil map
```

Always create one before writing: `m := map[string]int{}` or `m := make(map[string]int)`. The same trap hides inside structs, where a map field is nil until you assign one, so a constructor that forgets it produces a struct that panics on first write.

Missing keys return the zero value

Reading a key that is not there is not an error. You get the zero value of the value type.

```
counts := map[string]int{}
fmt.Println(counts["nothing"])   // 0, not undefined
counts["word"]++                 // works: reads 0, writes 1
```

That last line is why counting in Go is so tidy: no “if the key exists, add one, otherwise set it to one”.

To tell “absent” from “present but zero”, use the two-value form:

```
v, ok := ages["nobody"]
if !ok {
    fmt.Println("not in the map")
}
```

`ok` is a bool. This is the **comma ok** idiom, and you will see the same shape for type assertions and channel receives.

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript, `obj.missing` gives `undefined`, and you distinguish absent from set-to-undefined with `in` or `hasOwnProperty`. Go’s `v, ok :=` is the same idea with better ergonomics, because the check and the read are one operation and you cannot forget which you did.

Iteration order is deliberately random

```
for k, v := range ages {
    fmt.Println(k, v)
}
```

Run that twice and the order differs. This is not an implementation accident that might settle down: the runtime randomises the starting point on purpose, so that code which accidentally depends on order fails immediately and loudly rather than in production two years later when the table grows.

To iterate in a stable order, sort the keys:

```
keys := make([]string, 0, len(ages))
for k := range ages {
    keys = append(keys, k)
}
slices.Sort(keys)

for _, k := range keys {
    fmt.Println(k, ages[k])
}
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript objects do have an order: integer-like keys ascending, then string keys in insertion order. Plenty of code quietly relies on it. Go removes the temptation. If your output needs an order, you must say what that order is, which is usually an improvement to the code anyway.

Maps are reference types

Passing a map to a function does not copy it. The function can modify the caller's map.

```
func add(m map[string]int) {
    m["new"] = 1        // visible to the caller
}
```

This is unlike a struct and unlike an array, and like a slice's elements. There is no `const` map and no way to pass one read-only. If a function must not modify a map, the only guards are a copy (`maps.Clone`) or a comment.

Structs as values, and the thing you cannot do

A map of structs has one sharp edge:

```
type Stat struct{ Count int }
m := map[string]Stat{"a": {Count: 1}}

m["a"].Count++      // compile error: cannot assign to struct field
```

The map returns a **copy** of the struct, so incrementing it would modify something about to be discarded, and Go refuses rather than letting you write code that does nothing. Two fixes:

```
// 1. Read, modify, write back.
s := m["a"]
s.Count++
m["a"] = s

// 2. Store pointers instead.
m := map[string]*Stat{"a": {Count: 1}}
m["a"].Count++      // fine
```

The second is usually what you want when the values are large or frequently updated. The first is fine for small structs and avoids a layer of indirection.

Sets

Go has no set type. The idiom is a map with an empty struct value:

```
seen := map[string]struct{}{}
seen["ada"] = struct{}{}

if _, ok := seen["ada"]; ok {
    fmt.Println("already seen")
}
```

`struct{}` is a struct with no fields and it occupies zero bytes, so the map stores only keys. Using `map[string]bool` is also common and slightly more readable at the cost of one byte per entry; both are accepted Go.

> **IF YOU KNOW JAVASCRIPT**
> JavaScript has a real `Set` with `.has()` and `.add()`. Go's version is wordier, and that `struct{}{}` reads as line noise until you have seen it a dozen times. Read it as: the type `struct{}`, then an empty literal of it.

Maps are not safe for concurrent use

Two goroutines writing to the same map at the same time will crash the program with a runtime error, deliberately. Go detects it and stops rather than corrupting the table silently.

The answer is usually a mutex, which is chapter fifteen:

```
type Counter struct {
    mu     sync.Mutex
    counts map[string]int
}

func (c *Counter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.counts[key]++
}
```

> **IF YOU KNOW JAVASCRIPT**
> This has no JavaScript equivalent, because the event loop means two pieces of your code never run at literally the same instant. In Go they do. Any value shared between goroutines needs protection, and a map is the one where forgetting is immediately fatal rather than subtly wrong, which is mercifully the better failure.

> **PROJECT 6: A WORD INDEX WITH LINE NUMBERS**

Extend the frequency counter into something genuinely useful: for each word, record every line it appears on. This is a `map[string][]int`, a map whose values are slices, and it is a shape you will use often.

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "slices"
    "strings"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: index <file>")
        os.Exit(1)
    }

    f, err := os.Open(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
    defer f.Close()

    index := map[string][]int{}

    scanner := bufio.NewScanner(f)
    line := 0
    for scanner.Scan() {
        line++
        for _, w := range strings.Fields(scanner.Text()) {
            w = strings.ToLower(strings.Trim(w, ".,!?;:\"'()"))
            if len(w) < 4 {
                continue    // skip short words
            }
            // Appending to a nil slice works, so no "if absent" branch.
            if !slices.Contains(index[w], line) {
                index[w] = append(index[w], line)
            }
        }
    }
    if err := scanner.Err(); err != nil {
        fmt.Fprintln(os.Stderr, "reading:", err)
        os.Exit(1)
    }

    words := make([]string, 0, len(index))
    for w := range index {
```

```
            words = append(words, w)
        }
        slices.Sort(words)

        for _, w := range words {
            if len(index[w]) > 2 {   // only words on three or more lines
                fmt.Printf("%-16s %v\n", w, index[w])
            }
        }
    }
```

The line that rewards attention is `index[w] = append(index[w], line)`. If `w` is not in the map, `index[w]` is a nil slice, and appending to a nil slice is legal and returns a one-element slice. So the "create the entry if missing" branch that you would write in most languages simply does not exist here. Two zero-value rules, slices and maps, combining to delete a line of code.

Also note `bufio.Scanner`: it reads line by line without loading the file into memory, so this works on a file larger than your RAM, unlike the `os.ReadFile` in the last project. And `scanner.Err()` must be checked, because the loop ends identically on end-of-file and on a read error.

YOUR TURN

Change the value type to a small struct holding both the line numbers and a total count, and watch the compiler stop you from writing `index[w].Count++`. Fix it both ways described above, then decide which you prefer and why. That decision, made consciously once, is the difference between copying Go idioms and understanding them.

0

Pointers, without the fear

Pointers have a reputation they do not deserve, earned in languages where you can do arithmetic on them and walk off the end of memory. Go's pointers cannot do that. There is no pointer arithmetic, and the garbage collector means you never free anything. What is left is a small, useful idea: **a pointer is the address of a value rather than a copy of it**.

You have already been using them. This chapter names what was happening.

Everything in Go is passed by value

This is the single sentence that explains pointers, and it is worth reading twice because it is the opposite of your JavaScript instinct.

When you pass anything to a function, Go copies it. An int, a struct, a slice header, all copied. So a function cannot change what you passed:

```
func rename(u User) {
    u.Name = "changed"      // changes the copy
}

u := User{Name: "Ada"}
rename(u)
fmt.Println(u.Name)         // "Ada"
```

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript, objects are passed by **reference to the same object**, so a function that sets `user.name` changes the caller's object. Primitives are copied. Go removes the distinction: everything is copied, every time, and when you want the other behaviour you ask for it explicitly with a pointer. Less to remember, more to type.

Two operators

```
u := User{Name: "Ada"}

p := &u        // & takes the address: p is a *User
```

```
fmt.Println(*p)     // * dereferences: the User itself
fmt.Println(p.Name) // but you rarely need *, see below
```

`&x` means “the address of x”. `*T` in a type means “pointer to T”. `*p` as an expression means “the value at p”.

Now the function that works:

```
func rename(u *User, name string) {
    u.Name = name
}

rename(&u, "Grace")
fmt.Println(u.Name)     // "Grace"
```

WATCH OUT

You almost never write `(*u).Name`. Go dereferences automatically for field access and method calls, so `u.Name` works whether `u` is a `User` or a `*User`. This is a deliberate convenience and it means pointer-ness mostly disappears from the code that uses the value. The place it reappears is `nil`.

nil, and the panic you will meet

A pointer’s zero value is `nil`. Using a field on a nil pointer crashes:

```
var u *User          // nil
fmt.Println(u.Name)  // panic: runtime error: invalid memory address
                     // or nil pointer dereference
```

This is Go’s `Cannot read properties of undefined`, with the same cause and a louder failure: the program stops rather than continuing with a bad value.

The guard is an ordinary `if`:

```
func display(u *User) string {
    if u == nil {
        return "(none)"
    }
    return u.Name
}
```

IF YOU KNOW JAVASCRIPT

TypeScript’s `strictNullChecks` would make this a compile error. Go has no equivalent: a `*User` might be nil and the compiler will not stop you. This is the one area where

> TypeScript's type system is genuinely ahead, and it is worth saying rather than glossing over. Your defences in Go are: return values rather than pointers where you can, check at the boundary, and let the panic happen in development rather than coding defensively everywhere.

When to use a pointer

Three reasons, and nothing else.

One: you need to modify the caller's value. This is the common case, and it is why methods that change state take pointer receivers.

Two: the struct is large and copied often. A struct with twenty fields copied in a tight loop is real work. For a three-field struct it does not matter, and chasing this before measuring is how code gets uglier for no gain.

Three: nil is a meaningful value. A `*string` field distinguishes "the user did not supply this" from "the user supplied an empty string". You met this in chapter three as the limit of zero values. This is the fix.

```
type UpdateUser struct {
    Name  *string `json:"name"`   // nil means "leave it alone"
    Email *string `json:"email"`
}

func apply(u *User, patch UpdateUser) {
    if patch.Name != nil {
        u.Name = *patch.Name       // explicitly set, even to ""
    }
    if patch.Email != nil {
        u.Email = *patch.Email
    }
}
```

That is how a PATCH endpoint tells "clear this field" apart from "do not touch this field", and you will write it for real in part four.

When not to use a pointer

Slices, maps and channels already hold an internal pointer. Passing `*[]int` or `*map[string]int` is almost always wrong and marks code as written by someone still finding their feet.

```
func addItem(items []string) []string {   // correct
    return append(items, "new")
}
```

```
func addItem(items *[]string) {           // almost always wrong
    *items = append(*items, "new")
}
```

The slice version returns the new slice, which is the convention `append` itself follows. The pointer version works and reads badly.

Small structs you only read are also better by value. A `Point{X, Y int}` is sixteen bytes; copying it is cheaper than following a pointer.

The loop variable, and a trap that Go fixed

For years this was the most common Go bug in existence:

```
// Go 1.21 and earlier
for _, u := range users {
    go func() {
        fmt.Println(u.Name)   // every goroutine saw the SAME u
    }()
}
```

The loop variable was reused across iterations, so all the goroutines shared one variable and usually printed the last element several times.

Go 1.22 changed this. Loop variables are now per-iteration, and the code above does what it looks like it does.

> **WATCH OUT**
> You will still find the old workaround everywhere in blog posts and older codebases:
>
> ```
> for _, u := range users {
>     u := u                // "shadow" it, making a per-iteration copy
>     go func() { ... }()
> }
> ```
>
> It is harmless on modern Go and no longer needed. But check your `go.mod`: the new behaviour only applies if the `go` directive says 1.22 or higher. A module that still says `go 1.21` gets the old semantics **even on a new toolchain**, which is a subtle and genuinely nasty way to inherit this bug.

Pointers to struct fields

You can take the address of a field, and of a slice element, which is how the contact book in chapter five worked:

```
for i := range book.contacts {
    if book.contacts[i].Email == email {
        return &book.contacts[i]    // address of the real element
    }
}
```

What you cannot take the address of is a map entry, because the map may move its contents when it grows:

```
m := map[string]User{"a": {Name: "Ada"}}
p := &m["a"]        // compile error: cannot take address of m["a"]
```

This is the same restriction that stopped `m["a"].Name++` in chapter seven, and it has the same two fixes: read-modify-write, or store `*User` in the map.

PROJECT 7: AN INVENTORY WITH ADJUSTABLE STOCK

Build something where the pointer decisions actually matter, and where getting one wrong is visible.

```
package main

import (
    "errors"
    "fmt"
)

type Item struct {
    SKU   string
    Name  string
    Stock int
}

type Inventory struct {
    items map[string]*Item    // pointers: we mutate these in place
}

func NewInventory() *Inventory {
    return &Inventory{items: map[string]*Item{}}
}

func (inv *Inventory) Add(sku, name string, stock int) {
    inv.items[sku] = &Item{SKU: sku, Name: name, Stock: stock}
}

// Get returns nil when absent, so callers must check. The bool makes
// that
// impossible to forget by accident.
func (inv *Inventory) Get(sku string) (*Item, bool) {
```

```
    it, ok := inv.items[sku]
    return it, ok
}

var ErrInsufficientStock = errors.New("insufficient stock")

func (inv *Inventory) Take(sku string, n int) error {
    it, ok := inv.items[sku]
    if !ok {
        return fmt.Errorf("taking %s: not found", sku)
    }
    if it.Stock < n {
        return fmt.Errorf("taking %d of %s: %w",
            n, sku, ErrInsufficientStock)
    }
    it.Stock -= n       // through the pointer, so the map sees it
    return nil
}

func main() {
    inv := NewInventory()
    inv.Add("A1", "Blue widget", 10)

    if err := inv.Take("A1", 3); err != nil {
        fmt.Println(err)
    }
    if err := inv.Take("A1", 99); err != nil {
        fmt.Println("expected:", err)
        fmt.Println("is stock error:",
            errors.Is(err, ErrInsufficientStock))
    }

    if it, ok := inv.Get("A1"); ok {
        fmt.Printf("%s: %d left\n", it.Name, it.Stock)   // 7 left
    }
}
```

Now do the experiment that teaches the chapter. Change the map to `map[string]Item`, holding values rather than pointers, and try to compile. The line `it.Stock -= n` will still compile, because `it` is now a copy, and the stock will never change. Run it and watch `10 left` print after you took three. Then make it work by reading, modifying and writing back.

That bug, with values where you meant pointers, is the single most common mistake newcomers make with Go collections, and having caused it once on purpose you will recognise it instantly.

YOUR TURN

Add a `Restock(sku string, n int) error` method, then add a `Low(threshold int) []*Item` that returns every item below a stock level. Decide whether `Low` should return `[]*Item` or `[]Item` and be able to say why. One of them lets a caller change your inventory by accident, which might be exactly what you want or exactly what you do not.

0

Packages, modules and imports, against npm and ESM

You have been writing single-file programs. Real projects are many files, and Go's rules for organising them are stricter and simpler than what you are used to. This chapter is the one that makes a Go repository stop looking strange.

A package is a directory

Every `.go` file belongs to a package, declared on its first line. **Every file in a directory must declare the same package name**, and a directory is exactly one package. That is the whole rule.

```
myapp/
├── go.mod              module github.com/you/myapp
├── main.go             package main
├── store/
│   ├── store.go        package store
│   └── user.go         package store
└── money/
    └── money.go        package money
```

Files in `store/` can see each other's identifiers with no import at all, even the lowercase ones. There is no equivalent of importing between files in the same folder, because they are already the same package.

> **IF YOU KNOW JAVASCRIPT**
> This is the biggest structural difference from JavaScript. In a Next.js project every file is its own module, and you import between them constantly. In Go, `store/store.go` and `store/user.go` share a namespace: a function declared in one is callable from the other with no ceremony. You import across **directories**, never within one.

Importing

```
package main

import (
    "fmt"                // standard library
    "net/http"

    "github.com/google/uuid"          // third party

    "github.com/you/myapp/store"      // your own
)

func main() {
    s := store.New()
    fmt.Println(uuid.NewString())
}
```

You refer to things by the **package name**, which is the last element of the path by convention but is really whatever the files declare. `store.New`, not `New`.

The grouping above is conventional: standard library first, then third party, then your own module, separated by blank lines. `gofmt` sorts within groups and `goimports` adds and removes imports as you type.

> **WATCH OUT**
> An unused import is a compile error, exactly like an unused variable. This is why `goimports` in your editor is not optional: without it you will spend real time adding and removing import lines by hand.
>
> The escape hatch, for a package imported only for its side effects, is the blank identifier:
>
> ```
> import _ "github.com/lib/pq"   // registers a database driver
> ```

The import path is the module path plus the directory

There is no `baseUrl`, no `paths` mapping, no `@/components` alias, and no relative imports. `import "./store"` is not valid Go.

If your module is `github.com/you/myapp`, then the directory `store/` is imported as `github.com/you/myapp/store`. Always the full path.

> **IF YOU KNOW JAVASCRIPT**
> Verbose compared to `@/store`, and it buys one thing: an import path is globally unambiguous. You can paste it into a browser and often land on the source. There is no

build-time configuration that has to agree with your editor, your bundler and your test runner, because there is no mapping at all.

internal/ is a real access rule

A directory named `internal` is enforced by the compiler: only code rooted at its parent may import it.

```
myapp/
├── internal/
│   └── db/            importable by myapp and anything under it
└── pkg/
    └── client/        importable by anybody
```

If someone imports `github.com/you/myapp/internal/db` from a different module, the build fails. This is how a library publishes a small surface while keeping the rest private, and it is why nearly every serious Go project puts most of its code under `internal/`.

IF YOU KNOW JAVASCRIPT
The nearest npm equivalent is the `exports` field in `package.json`, which controls what consumers can reach. Go's version is a directory name, needs no configuration and cannot be worked around.

Naming, and the stutter rule

Package names are short, lowercase, single words. No underscores, no camelCase, no plurals.

The identifier is read **with** the package name, which leads to the most quoted style rule in Go:

```
// Bad: the package name is already there.
store.NewStore()
http.HTTPServer
money.MoneyAmount

// Good:
store.New()
http.Server
money.Amount
```

WATCH OUT

Avoid `util`, `helpers`, `common` and `shared`. They are not packages, they are places things go to hide, and they grow into a dependency that everything imports and nobody understands. If a function does not belong anywhere, that usually means the thing it operates on wants a package of its own.

Modules, versions and go.mod

A module is a tree of packages with one `go.mod` at the root. You saw it in chapter two; here is what the rest of it means.

```
module github.com/you/myapp

go 1.25

require (
    github.com/gin-gonic/gin v1.10.0
    gorm.io/gorm v1.25.12
)

require (
    github.com/bytedance/sonic v1.12.0 // indirect
)
```

The second `require` block is indirect dependencies: things your dependencies need. `go mod tidy` manages both blocks, adding what is imported and removing what is not. Run it before every commit.

IF YOU KNOW JAVASCRIPT

`go.sum` is the lockfile and must be committed. Unlike `package-lock.json` it is not a full dependency tree: it is a list of cryptographic hashes. Go uses **minimal version selection**, meaning it picks the lowest version that satisfies everyone, so builds are reproducible from `go.mod` alone and the sum file exists to verify integrity rather than to pin.

There is also no `devDependencies`. A tool you use at development time is either run with `go run` or tracked in a `tools.go` file.

Upgrading is explicit:

```
go get -u ./...          # upgrade everything (minor and patch)
go get github.com/gin-gonic/gin@v1.10.1   # one package, one version
go mod tidy
```

Major versions are in the path

This surprises everyone. From v2 onward, the major version is part of the import path:

```
import "github.com/you/myapp/v2/store"
```

It looks strange and it solves a real problem: two major versions of the same library can coexist in one build, because as far as the compiler is concerned they are different packages. You will see this in Grit itself, which imports as `github.com/MUKE-coder/grit/v3`.

init, and why to avoid it

A package can declare `func init()`, which runs automatically before `main`.

```
func init() {
    // runs once, before main, in no order you control across packages
}
```

Use it rarely. It makes startup order implicit, it cannot return an error, and it runs even when a test only wanted one function from the package. Explicit setup called from `main` is almost always better.

PROJECT 8: SPLIT THE INVENTORY INTO PACKAGES

Take the inventory from chapter seven and give it a real project layout. The point is to feel the import rules rather than to add features.

```
inventory/
├── go.mod                  module github.com/you/inventory
├── main.go                 package main
└── internal/
    └── inventory/
        ├── item.go         package inventory
        └── store.go        package inventory
```

`internal/inventory/item.go`:

```
package inventory

import "errors"

var ErrInsufficientStock = errors.New("insufficient stock")

type Item struct {
    SKU   string
    Name  string
    Stock int
}
```

```
// low is unexported, so only this package can call it. item.go and
// store.go
// are the same package, so store.go can use it with no import.
func (i *Item) low(threshold int) bool {
    return i.Stock < threshold
}
```

`internal/inventory/store.go`:

```
package inventory

import "fmt"

type Store struct {
    items map[string]*Item
}

func New() *Store {
    return &Store{items: map[string]*Item{}}
}

func (s *Store) Add(sku, name string, stock int) {
    s.items[sku] = &Item{SKU: sku, Name: name, Stock: stock}
}

func (s *Store) Take(sku string, n int) error {
    it, ok := s.items[sku]
    if !ok {
        return fmt.Errorf("taking %s: not found", sku)
    }
    if it.Stock < n {
        return fmt.Errorf("taking %d of %s: %w",
            n, sku, ErrInsufficientStock)
    }
    it.Stock -= n
    return nil
}

// Low uses the unexported method from the other file. No import needed.
func (s *Store) Low(threshold int) []*Item {
    var out []*Item
    for _, it := range s.items {
        if it.low(threshold) {
            out = append(out, it)
        }
    }
    return out
}
```

`main.go`:

```go
package main

import (
    "errors"
    "fmt"

    "github.com/you/inventory/internal/inventory"
)

func main() {
    inv := inventory.New()
    inv.Add("A1", "Blue widget", 10)

    if err := inv.Take("A1", 99); err != nil {
        // true
        fmt.Println(errors.Is(err, inventory.ErrInsufficientStock))
    }

    for _, it := range inv.Low(20) {
        fmt.Println("low:", it.Name, it.Stock)
    }
}
```

Now run three experiments. Call `inv.low(5)` from `main.go` and read the error: lowercase means package-private, and the compiler enforces it. Move the whole `internal` directory to `pkg` and confirm it still builds, then put it back. And try `import "./internal/inventory"` to see that relative imports genuinely do not exist.

YOUR TURN

Add a second package, `internal/report`, with a function that takes a `*inventory.Store` and prints a summary. You will hit a design question immediately: `report` needs to read every item, but `Store.items` is unexported. Solve it by adding an exported method to `inventory` rather than by exporting the field, and notice that the package boundary just forced you into a better API than you would have written in one file.

0

Interfaces, and why Go's point the other way

If you take one idea from this book, take this one. Go's interfaces look like TypeScript's and work backwards from them, and once that clicks a great deal of Go code stops looking odd and starts looking inevitable.

The mechanical part

An interface is a set of method signatures.

```
type Shape interface {
    Area() float64
    Perimeter() float64
}
```

Any type with those two methods **is** a `Shape`. You do not say so anywhere.

```
type Rect struct{ W, H float64 }

func (r Rect) Area() float64      { return r.W * r.H }
func (r Rect) Perimeter() float64 { return 2 * (r.W + r.H) }

// Rect is now a Shape. Nothing was declared. Nothing imports Shape.
var s Shape = Rect{W: 3, H: 4}
```

This is **structural typing**, and it is called implicit interface satisfaction.

> **IF YOU KNOW JAVASCRIPT**
> TypeScript interfaces are structural too, so this much is familiar: an object with the right shape satisfies the interface. The difference is where the interface **lives** and who declares it, which is the next section and is the part that actually matters.

The direction: interfaces belong to the consumer

In Java or C#, the implementer declares what it implements: `class Rect implements Shape`. The interface is defined near the thing that provides it, and the provider must import it.

In Go, the convention is the reverse. **The interface is declared by the package that consumes it, and lists only what that consumer needs.**

Consider a function that writes a report somewhere.

```
// In package report. It does not import os, or a logging library, or a
// storage SDK. It says what it needs and nothing more.
type Writer interface {
    Write(p []byte) (int, error)
}

func Generate(w Writer, data []Row) error {
    for _, r := range data {
        _, err := fmt.Fprintf(w, "%s,%d\n", r.Name, r.Count)
        if err != nil {
            return err
        }
    }
    return nil
}
```

Now `Generate` accepts a file, a network connection, an HTTP response, an in-memory buffer, a gzip stream, or anything else with a `Write` method. None of those types know `report` exists. The dependency points **from** the consumer **to** nothing at all.

> **IF YOU KNOW JAVASCRIPT**
> In TypeScript you would often import a type from the module that provides it, or define a shared `types.ts` that both sides import. Go's version removes the shared file: the consumer owns a small interface, the provider owns a concrete type, and neither imports the other. That is the decoupling, and it is why Go codebases have surprisingly few imports between their own packages.

Keep them small

That `Writer` above is the real one. `io.Writer` in the standard library is exactly those four words, and it is arguably the most successful interface ever designed:

```
type Writer interface {
    Write(p []byte) (int, error)
}
```

One method. Hundreds of types satisfy it. Everything composes.

The proverb is **the bigger the interface, the weaker the abstraction**. A one-method interface is satisfied by nearly anything and is easy to fake in a test. A ten-method interface is satisfied by one type and is a class declaration wearing a disguise.

WATCH OUT

The beginner mistake, carried in from other languages, is to define an interface for every struct up front:

```
// Do not do this.
type UserService interface {
    Create(...) error
    Update(...) error
    Delete(...) error
    List(...) ([]User, error)
    // ... nine more
}

type userService struct{}   // the only implementation, ever
```

One implementation and one interface that mirrors it exactly is pure ceremony: you have doubled the code and decoupled nothing. The Go advice is **accept interfaces, return structs**: return the concrete type from your constructor, and let whichever caller needs an abstraction declare the two methods it actually uses.

The empty interface, and `any`

An interface with no methods is satisfied by everything.

```
var x any = 42          // any is an alias for interface{}
x = "hello"
x = User{}
```

`any` is Go's `unknown`, not its `any`. You cannot do anything with the value until you establish what it is. The tool is a **type assertion**:

```
s, ok := x.(string)
if !ok {
    // x was not a string
}
```

Or a type switch, which is the readable form when several types are possible:

```
switch v := x.(type) {
case string:
    fmt.Println("string of length", len(v))
```

```
case int:
    fmt.Println("int doubled:", v*2)
case User:
    fmt.Println("user named", v.Name)
default:
    fmt.Println("something else")
}
```

> **WATCH OUT**
> The single-value assertion `s := x.(string)` **panics** if the type is wrong. The two-value form `s, ok := x.(string)` never panics. Use the two-value form unless you genuinely want to crash, which in a server you almost never do.

Reach for `any` rarely. It throws away everything the type system knows, and most uses of it in a new codebase are better served by an interface with one method or, since Go 1.18, by generics.

The nil interface trap

This one catches experienced people and is worth the page.

An interface value has two parts internally: a **type** and a **value**. It is `nil` only when both are nil.

```
type MyError struct{}
func (e *MyError) Error() string { return "boom" }

func doWork() error {
    var e *MyError = nil    // a nil pointer
    return e                // wrapped in an error interface
}

err := doWork()
fmt.Println(err == nil)     // false!
```

The returned interface holds type `*MyError` and value `nil`. The type is not nil, so the interface is not nil, so `err != nil` is true and your error handling runs for an error that is not there.

> **IF YOU KNOW JAVASCRIPT**
> There is no JavaScript equivalent because there is no such two-part value. The practical rule is the one from chapter four, and now you know why it exists: **declare the return type as `error` and return a literal `nil`**, never a typed nil variable.

```go
func doWork() error {
    if somethingWrong {
        return &MyError{}
    }
    return nil          // literal nil, not a typed nil
}
```

Interfaces you will meet constantly

Four from the standard library carry an enormous amount of Go code.

`io.Reader`	`Read(p []byte) (int, error)`. Anything you can read bytes from
`io.Writer`	`Write(p []byte) (int, error)`. Anything you can write bytes to
`error`	`Error() string`. You met it in chapter four
`fmt.Stringer`	`String() string`. How a type prints itself

`Stringer` is a quick win. Implement it and `fmt.Println` uses it:

```go
func (u User) String() string {
    return fmt.Sprintf("%s <%s>", u.Name, u.Email)
}

fmt.Println(u)      // Ada <ada@example.com>
```

IF YOU KNOW JAVASCRIPT

That is `toString()`, with one important difference: nothing is called implicitly during string concatenation, because Go has no implicit conversion. `"x" + u` does not compile. Only the `fmt` family looks for `String()`.

Why this matters for testing

The payoff arrives when you test. A function that takes an interface can be handed a fake with no mocking library and no dependency injection framework.

```go
// In the package that needs it, listing only what it uses.
type Mailer interface {
    Send(to, subject, body string) error
}

func Register(u User, m Mailer) error {
    // ...
```

```
    return m.Send(u.Email, "Welcome", "Hello "+u.Name)
}
```

The test:

```
type fakeMailer struct {
    sent []string
}

func (f *fakeMailer) Send(to, subject, body string) error {
    f.sent = append(f.sent, to)
    return nil
}

func TestRegisterSendsWelcome(t *testing.T) {
    m := &fakeMailer{}
    if err := Register(User{Email: "ada@example.com"}, m); err != nil {
        t.Fatal(err)
    }
    if len(m.sent) != 1 || m.sent[0] != "ada@example.com" {
        t.Errorf("got %v, want one mail to ada@example.com", m.sent)
    }
}
```

Six lines of fake, no library, no configuration. That is the whole argument for consumer-declared interfaces, and it only works because `Mailer` has one method rather than the fifteen a real mail client has.

PROJECT 9: A NOTIFIER WITH THREE BACKENDS

Build something that uses an interface the way Go intends, then prove the decoupling by adding a backend without touching the code that uses it.

```
package main

import (
    "fmt"
    "io"
    "os"
    "strings"
)

// Notifier is declared here, by the consumer, and has one method.
type Notifier interface {
    Notify(message string) error
}

// Three unrelated implementations. None of them mention Notifier.
```

```
type ConsoleNotifier struct{}

func (ConsoleNotifier) Notify(msg string) error {
    _, err := fmt.Println("[console]", msg)
    return err
}

type WriterNotifier struct{ W io.Writer }

func (w WriterNotifier) Notify(msg string) error {
    _, err := fmt.Fprintf(w.W, "[file] %s\n", msg)
    return err
}

type CollectingNotifier struct{ Messages []string }

func (c *CollectingNotifier) Notify(msg string) error {
    c.Messages = append(c.Messages, msg)
    return nil
}

// The consumer. It never learns which backend it has.
func alertAll(ns []Notifier, msg string) error {
    for _, n := range ns {
        if err := n.Notify(msg); err != nil {
            return fmt.Errorf("notifying: %w", err)
        }
    }
    return nil
}

func main() {
    var buf strings.Builder
    collector := &CollectingNotifier{}

    notifiers := []Notifier{
        ConsoleNotifier{},
        WriterNotifier{W: &buf},
        WriterNotifier{W: os.Stdout},
        collector,
    }

    if err := alertAll(notifiers, "disk almost full"); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }

    fmt.Print("buffer contains: ", buf.String())
    fmt.Println("collector saw:", collector.Messages)
}
```

Notice three things. `WriterNotifier` is constructed twice with completely different destinations, a string builder and standard output, because both satisfy `io.Writer`. `CollectingNotifier` uses a pointer receiver because it mutates, so the slice holds `collector` rather than `CollectingNotifier{}`, and putting the value in instead is a compile error worth seeing. And `alertAll` would work unchanged against an email sender, a Slack client or a test fake.

Now add a fourth: a `PrefixNotifier` that wraps another `Notifier` and adds a prefix before delegating. That is the decorator pattern, in Go, in eight lines, with no base class:

```
type PrefixNotifier struct {
    Prefix string
    Inner  Notifier
}

func (p PrefixNotifier) Notify(msg string) error {
    return p.Inner.Notify(p.Prefix + msg)
}
```

YOUR TURN

Give `CollectingNotifier` a `String() string` method so it satisfies `fmt.Stringer`, then print the collector directly with `fmt.Println(collector)` and watch the output change. Then write a test for `alertAll` that proves it stops and returns a wrapped error when one notifier fails, using a fake that always errors. That test is four lines and needs no library, which is the point of the chapter.

0

Designing with errors: what a package should return

Chapter four covered the mechanics: `if err != nil`, wrapping with `%w`, `errors.Is` and `errors.As`. This chapter is about the decision those mechanics leave you with, which is harder and matters more: **when someone calls your function and it fails, what should they be able to do about it?**

Get this right and the caller writes three lines. Get it wrong and they end up matching on error message strings, which is the clearest sign of an API that did not think about failure.

Three kinds of error, and when to use each

Opaque. The caller can log it and give up. Nothing else.

```
return fmt.Errorf("parsing config: %w", err)
```

This is the default and it is correct far more often than people expect. If there is nothing useful a caller could do differently, do not invent a way for them to try.

Sentinel. A specific, exported value the caller compares against.

```
var ErrNotFound = errors.New("not found")

func (s *Store) Get(id int) (*User, error) {
    ...
    return nil, ErrNotFound
}
```

The caller:

```
u, err := store.Get(id)
if errors.Is(err, ErrNotFound) {
    http.Error(w, "no such user", http.StatusNotFound)
    return
}
```

Use a sentinel when the condition is a **fact with no detail**: not found, already exists, timed out. The name is the entire payload.

Typed. A struct implementing `error`, carrying fields.

```
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Reason)
}
```

The caller:

```
var ve *ValidationError
if errors.As(err, &ve) {
    respond(w, 422, map[string]string{ve.Field: ve.Reason})
}
```

Use a typed error when the caller needs **data** from the failure, not just its identity. A validation failure needs the field name. A rate limit needs the retry-after. A conflict needs the version that is current.

Opaque	Caller logs and gives up. The default
Sentinel	Caller branches on identity. `errors.Is`
Typed	Caller needs fields out of it. `errors.As`

> **WATCH OUT**
> A sentinel or an exported error type is **part of your public API**. Renaming `ErrNotFound`, or changing which conditions return it, breaks callers exactly like changing a function signature. Declare them deliberately and few.
>
> The corollary: do not export an error type just because you made one. If nothing outside the package uses `errors.As` on it, keep it unexported and return it as a plain `error`.

Wrap, but not at every level

`%w` preserves the chain, and the temptation is to wrap at every function boundary. Do not. The result is a message like:

```
handling request: getting user: loading user: querying users:
executing query:
scanning row: sql: no rows in result set
```

Every layer added a word and none added information.

The rule worth following: **wrap when you add context the caller does not already have**. The database layer knows the query. The service layer knows the user ID. The handler knows the request. Each of those is worth one wrap. "I am a function and I called another function" is not context.

```
// Good: adds the thing the caller cannot see.
return fmt.Errorf("loading user %d: %w", id, err)

// Noise: the caller already knows they called LoadUser.
return fmt.Errorf("LoadUser: %w", err)
```

> **IF YOU KNOW JAVASCRIPT**
> This is the same discipline as a JavaScript stack trace, with one difference: Go errors have no stack trace by default, so the chain of wraps **is** your trace. That is why each one should earn its place. Some teams add stack traces with a library; the standard library deliberately does not, on the grounds that a well-wrapped error is more readable than sixty frames.

Error strings are a contract, lowercase and unpunctuated

Convention, followed throughout the standard library:

```
errors.New("connection refused")     // good
errors.New("Connection refused.")    // wrong: capital and full stop
```

The reason is composition. Errors get wrapped into sentences, and a capital in the middle reads badly:

```
loading config: Connection refused.: retrying
```

Also: do not end with a newline, and do not include the word "error". The reader knows.

Do not return both a value and an error

Pick one. A function that returns a partially filled struct **and** an error invites the caller to use the broken value.

```
// Bad: what is the caller supposed to do with a half-built User?
func Load(id int) (User, error)

// Good: nil on failure, impossible to misuse.
func Load(id int) (*User, error)
```

The exceptions are real but narrow: `io.Reader` returns a byte count with an error because a partial read is meaningful, and `strconv.Atoi` returns zero because it has nothing better to offer. If you are not implementing something in that shape, return nil.

Handle an error once

The most common error-handling bug in any language is doing two things with one failure:

```
if err != nil {
    log.Printf("saving user: %v", err)   // handled it here
    return err                           // and again, up the stack
}
```

Now it is logged at every level, and the operator reading the logs sees the same failure five times and has to work out whether it happened once or five times.

Choose: **either** handle it, **or** return it. The usual split is that libraries and services return, and the top layer, a handler or `main`, logs once and decides what the user sees.

Mapping errors at the boundary

This is where the three kinds pay off. One place in your program turns errors into HTTP responses:

```
func writeError(w http.ResponseWriter, err error) {
    var ve *ValidationError
    switch {
    case errors.Is(err, ErrNotFound):
        respond(w, 404, "not found")
    case errors.Is(err, ErrConflict):
        respond(w, 409, "already exists")
    case errors.As(err, &ve):
        respond(w, 422, map[string]string{ve.Field: ve.Reason})
    default:
        log.Printf("unhandled: %v", err)   // the only place that logs
        respond(w, 500, "internal error")
    }
}
```

Everything below this function returns errors and logs nothing. Everything above it gets a correct status code. That is the whole architecture of error handling in a Go service, and you will meet it again in part four, because it is exactly what Grit generates.

> **IF YOU KNOW JAVASCRIPT**
> In an Express app this is the error middleware, `app.use((err, req, res, next) => ...)`, and the design problem is identical: the middleware has to work out what kind of failure it received. JavaScript usually does it with `err instanceof ValidationError` or a `code` property. Go's `errors.As` is the same idea with the chain-walking built in, so it still works after the error has been wrapped three times.

panic is not an error

Go has `panic`, and it is not for expected failures. A panic unwinds the stack and, uncaught, kills the program.

Use `panic` for **programmer error that cannot be recovered from**: an impossible state, a nil dependency at startup, a configuration that cannot be parsed before the server starts serving.

```
func MustCompile(pattern string) *Regexp {
    re, err := Compile(pattern)
    if err != nil {
        panic("regexp: Compile(" + pattern + "): " + err.Error())
    }
    return re
}
```

The `Must` prefix is the convention: it means "this panics instead of returning an error, and you should only call it where failure means the program is broken", typically at package initialisation with a literal you wrote yourself.

Never panic for a bad request, a missing row, or a failed network call. Those are expected, and expected failures are values.

> **PROJECT 10: A USER STORE WITH A DESIGNED ERROR SURFACE**
> Build a small store that uses all three error kinds deliberately, plus the boundary mapping. This is a miniature of the structure every Go service ends up with.
>
> ```
> package main
>
> import (
>     "errors"
> ```

```
    "fmt"
    "strings"
)

// Sentinels: facts with no detail. Part of the public API.
var (
    ErrNotFound = errors.New("user not found")
    ErrConflict = errors.New("email already registered")
)

// Typed: the caller needs the field name out of it.
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Reason)
}

type User struct {
    ID    int
    Name  string
    Email string
}

type Store struct {
    users  map[int]*User
    nextID int
}

func NewStore() *Store {
    return &Store{users: map[int]*User{}, nextID: 1}
}

func (s *Store) Create(name, email string) (*User, error) {
    if strings.TrimSpace(name) == "" {
        return nil, &ValidationError{
            Field: "name", Reason: "must not be empty"}
    }
    if !strings.Contains(email, "@") {
        return nil, &ValidationError{
            Field: "email", Reason: "must be an email"}
    }
    for _, u := range s.users {
        if u.Email == email {
            // Wrapped, so the caller gets the identity and the detail.
            return nil, fmt.Errorf("creating user %s: %w",
                email, ErrConflict)
        }
    }
```

```
    u := &User{ID: s.nextID, Name: name, Email: email}
    s.users[u.ID] = u
    s.nextID++
    return u, nil
}

func (s *Store) Get(id int) (*User, error) {
    u, ok := s.users[id]
    if !ok {
        return nil, fmt.Errorf("getting user %d: %w", id, ErrNotFound)
    }
    return u, nil
}

// The boundary. The only place that decides what a failure means, and
// the
// only place that logs.
func describe(err error) string {
    var ve *ValidationError
    switch {
    case err == nil:
        return "200 ok"
    case errors.Is(err, ErrNotFound):
        return "404 not found"
    case errors.Is(err, ErrConflict):
        return "409 conflict"
    case errors.As(err, &ve):
        return fmt.Sprintf("422 invalid %s (%s)", ve.Field, ve.Reason)
    default:
        return "500 internal: " + err.Error()
    }
}

func main() {
    s := NewStore()

    _, err := s.Create("Ada", "ada@example.com")
    fmt.Println(describe(err))                  // 200 ok

    _, err = s.Create("", "x@example.com")
    fmt.Println(describe(err))                  // 422 invalid name

    _, err = s.Create("Grace", "not-an-email")
    fmt.Println(describe(err))                  // 422 invalid email

    _, err = s.Create("Someone", "ada@example.com")
    fmt.Println(describe(err))                  // 409 conflict

    _, err = s.Get(99)
    fmt.Println(describe(err))                  // 404 not found
}
```

Two things to prove to yourself. First, `errors.Is(err, ErrConflict)` returns true even though the error was wrapped with extra text, which is the entire reason to use `%w` rather than `%v`. Change that one verb to `%v` and watch the conflict case fall through to 500.

Second, note that `Create` never logs. It returns. `describe` is the only function that decides what a failure means to the outside world, so moving this store behind an HTTP API means changing one function.

YOUR TURN

Add an `Update(id int, name string) error` that can fail in three ways: the user is missing, the name is invalid, or the name is unchanged. Decide for each whether it is opaque, sentinel or typed, and be able to defend the choice. “Unchanged” is the interesting one: is that an error at all, or should `Update` quietly succeed? There is a defensible answer either way, and the act of choosing is the skill this chapter is teaching.

0

defer, panic and recover

Three keywords that look like `finally`, `throw` and `catch` and are not. The resemblance is close enough to mislead, which is why they get a chapter.

defer: cleanup that cannot be forgotten

`defer` schedules a function call to run when the **surrounding function** returns, by any path.

```
func readConfig(path string) error {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()     // runs on every return below, and on a panic

    // twenty lines with six return statements
    return nil
}
```

Write the `defer` immediately after the thing that needs cleaning up. That adjacency is the point: open and close are one line apart, so a reviewer can see the pair without scrolling.

> **IF YOU KNOW JAVASCRIPT**
> `try { ... } finally { f.close() }` does the same job by wrapping, which indents the body and separates the close from the open. `defer` keeps them together and costs no indentation. It also composes: ten resources means ten `defer` lines, not ten nested blocks.

The three rules of defer

One: arguments are evaluated immediately, the call happens later.

```
func main() {
    msg := "first"
    defer fmt.Println(msg)   // prints "first", not "second"
```

```
    msg = "second"
}
```

`msg` was read when the `defer` statement ran. To defer the **evaluation**, wrap it in a closure:

```
defer func() { fmt.Println(msg) }()   // prints "second"
```

Two: deferred calls run last in, first out.

```
for i := 0; i < 3; i++ {
    defer fmt.Println(i)
}
// prints 2, then 1, then 0
```

This is right for cleanup: resources are released in the reverse of the order they were acquired, like unwinding a stack.

Three: defer is function-scoped, not block-scoped.

> **WATCH OUT**
> This is the one that causes real bugs:
>
> ```
> func processAll(paths []string) error {
>     for _, p := range paths {
>         f, err := os.Open(p)
>         if err != nil {
>             return err
>         }
>         defer f.Close()     // does NOT run at the end of this iteration
>         // ... use f
>     }
>     return nil
> }
> ```
>
> All the files stay open until `processAll` returns. With a thousand paths you run out of file descriptors.
>
> The fix is to give each iteration its own function, so there is a return for the `defer` to fire on:
>
> ```
> for _, p := range paths {
>     if err := processOne(p); err != nil {
>         return err
>     }
> }
>
> func processOne(p string) error {
> ```

```
    f, err := os.Open(p)
    if err != nil {
        return err
    }
    defer f.Close()     // runs when processOne returns
    // ... use f
    return nil
}
```

Extracting the loop body into a function is the standard answer, and it usually improves the code for other reasons too.

defer can change the return value

A deferred closure can modify a **named** return value after the `return` statement has chosen it. This is the one legitimate use of named returns.

```
func doWork() (err error) {
    defer func() {
        if err != nil {
            err = fmt.Errorf("doWork: %w", err)
        }
    }()

    return doSomethingThatFails()    // gets wrapped on the way out
}
```

Used sparingly this is tidy. Used often it makes a function's return value hard to trace, so prefer explicit wrapping at each return unless you have many of them.

defer and errors from Close

A deferred `Close` discards its error, which is usually fine for reading and sometimes wrong for writing: a failed `Close` on a file you wrote may mean the data never reached the disk.

```
func writeReport(path string, data []byte) (err error) {
    f, err := os.Create(path)
    if err != nil {
        return err
    }
    defer func() {
        if cerr := f.Close(); cerr != nil && err == nil {
            err = fmt.Errorf("closing %s: %w", path, cerr)
        }
    }()
```

```
    _, err = f.Write(data)
    return err
}
```

That is verbose and correct. For a read-only file, `defer f.Close()` is accepted practice and nobody will object.

panic: stop, this should not have happened

`panic` unwinds the stack, running deferred functions as it goes, and if it reaches the top of a goroutine the program crashes with a stack trace.

```
func main() {
    defer fmt.Println("this still runs")
    panic("something impossible")
}
```

```
this still runs
panic: something impossible

goroutine 1 [running]:
main.main()
    /tmp/main.go:7 +0x65
exit status 2
```

Go panics on its own for a handful of runtime faults: nil pointer dereference, index out of range, division by zero, sending on a closed channel, and writing to a nil map.

> **IF YOU KNOW JAVASCRIPT**
> `throw` in JavaScript is for **expected** failures: a 404, a validation problem, a parse error. `panic` is not. Go's expected failures are returned values, and `panic` is reserved for "this program's assumptions are broken".
>
> If you find yourself panicking in a request handler because a user sent bad input, that is the JavaScript habit talking. Return an error.

recover: only for a boundary

`recover` stops a panic, and it only works **inside a deferred function**.

```
func safe() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
```

```
    }()

    panic("boom")
}

err := safe()
fmt.Println(err)    // recovered: boom
```

There are exactly two good reasons to use it.

One: a server that must not die because one request was bad. A panic in one handler should return 500 for that request, not take down the process serving everybody else.

```
func Recover(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                log.Printf("panic on %s %s: %v\n%s",
                    r.Method, r.URL.Path, rec, debug.Stack())
                http.Error(w, "internal server error",
                    http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

Log the stack. A recovered panic with no stack trace is a bug you will never find. Every Go web framework ships this middleware, Gin included, and Grit wires it in for you.

Two: a library turning an internal panic into an error at its edge. Some parsers panic internally to unwind out of deep recursion, then recover at the public function and return an error. The caller never sees a panic. This is legitimate and rare.

> **WATCH OUT**
> `recover` only catches panics in **its own goroutine**. A panic in a goroutine you started is not caught by a recover in the function that started it:
>
> ```
> func main() {
>     defer func() { recover() }()     // does not help
>     go func() {
>         panic("in a goroutine")      // kills the whole program
>     }()
>     time.Sleep(time.Second)
> }
> ```

Every goroutine that could panic needs its own recover. This is the single most common way a Go server dies in production: a background worker with no recover, panicking on a nil map six hours after deployment.

PROJECT 11: A WORKER THAT SURVIVES BAD INPUT

Build a small job runner where one bad job must not stop the rest, and where cleanup is guaranteed. This is the panic boundary in miniature.

```
package main

import (
    "errors"
    "fmt"
    "strings"
)

type Job struct {
    Name string
    Run  func() error
}

// runOne is the boundary. Whatever the job does, this returns an error
// rather than letting a panic escape.
func runOne(j Job) (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("job %q panicked: %v", j.Name, r)
        }
    }()
    defer fmt.Printf("  [%s] finished\n", j.Name)   // runs even on panic

    if j.Run == nil {
        return errors.New("job has no work")
    }
    return j.Run()
}

func runAll(jobs []Job) (failed int) {
    for _, j := range jobs {
        fmt.Printf("running %s\n", j.Name)
        if err := runOne(j); err != nil {
            fmt.Println("  error:", err)
            failed++
        }
    }
    return failed
}

func main() {
```

```
    jobs := []Job{
        {Name: "ok", Run: func() error { return nil }},
        {Name: "fails", Run: func() error {
            return errors.New("disk full")
        }},
        {Name: "panics", Run: func() error {
            var m map[string]int
            m["boom"] = 1              // panic: assignment to nil map
            return nil
        }},
        {Name: "index", Run: func() error {
            s := []int{1, 2, 3}
            fmt.Println(s[10])         // panic: index out of range
            return nil
        }},
        {Name: "no work"},
        {Name: "still runs", Run: func() error {
            fmt.Println("  the runner survived four failures")
            return nil
        }},
    }

    failed := runAll(jobs)
    fmt.Printf("\n%d of %d jobs failed\n", failed, len(jobs))
    _ = strings.TrimSpace("")
}
```

Three things to notice when you run it.

The `finished` line prints for every job including the ones that panicked, because deferred calls run while the stack unwinds. That is what makes `defer` safe for cleanup in a way that a line at the end of the function is not.

The two deferred functions run in reverse order, so `finished` prints before the recover's error is assembled.

And the program exits normally with a count, rather than dying on job three. Delete the `recover` block and run it again: you get a stack trace and jobs four, five and six never run. That difference is the reason every server has this middleware.

YOUR TURN

Move the `recover` from `runOne` up into `runAll`, wrapping the whole loop, and run it again. You will catch the first panic and then the loop stops, because the panic unwound past the loop. That experiment shows exactly what "a boundary" means: it has to be inside the unit of work you want to isolate, not outside it.

0

Goroutines, against the event loop

You already write concurrent code. Every `await` you have typed was concurrency. What you have not had is **parallelism**: two pieces of your own code running at the same instant on two cores. This chapter is about the difference, because it is the whole reason Go's model exists.

The one-keyword version

Put `go` in front of a function call and it runs independently.

```
func main() {
    go fmt.Println("from a goroutine")
    fmt.Println("from main")
    time.Sleep(100 * time.Millisecond)
}
```

That is the entire syntax. No `async` on the function, no `await` at the call, no promise returned. Any function can be a goroutine; nothing about its declaration changes.

> **WATCH OUT**
> The `time.Sleep` is there because **when `main` returns, the program exits immediately**, killing every goroutine still running. Remove it and you will probably see only "from main".
>
> Sleeping is never the real answer. The real answers are `sync.WaitGroup` and channels, both below. But almost every Go tutorial starts with a sleep, and it is worth knowing that it is a placeholder for synchronisation rather than a technique.

What is actually different

In Node, your JavaScript runs on **one** thread. The event loop interleaves work: while one request waits on the database, another gets a turn. That is excellent for I/O and useless for computation, because a function that spends two hundred milliseconds doing arithmetic blocks everything.

Go runs goroutines on a pool of operating system threads, usually one per CPU core. Two goroutines can execute at literally the same moment.

	Node	Go
Unit	Promise / task	Goroutine
Threads	One for your code	One per core, by default
Waiting on I/O	Excellent	Excellent
CPU work	Blocks everything	Uses every core
Cost each	Small object	About 2 KB of stack
Practical limit	Many thousands	Hundreds of thousands

A goroutine starts with a two kilobyte stack that grows as needed. An operating system thread starts with one or two **megabytes**. That thousand-fold difference is why "start a goroutine per request" is a normal design in Go and "start a thread per request" was never viable in Java or C#.

> **IF YOU KNOW JAVASCRIPT**
> The mental shift: in Node you **structure** code so it never blocks, because blocking is catastrophic. In Go you write straight-line blocking code, and the runtime moves the goroutine off the thread while it waits. There is no colour to functions, no `async` infecting every caller up the stack. A function that reads a file looks like a function that reads a file.

Waiting properly: sync.WaitGroup

A `WaitGroup` counts outstanding work.

```
func main() {
    var wg sync.WaitGroup

    for i := 1; i <= 3; i++ {
        wg.Add(1)                  // one more to wait for
        go func() {
            defer wg.Done()        // one fewer, even if this panics
            fmt.Println("worker", i)
        }()
    }

    wg.Wait()                      // blocks until the count is zero
    fmt.Println("all done")
}
```

Three rules, and the bugs that come from breaking them:

`wg.Add` before the `go`, never inside it. Inside, `Wait` may run before the goroutine starts and see a count of zero.

`defer wg.Done()` as the first line. A panic or an early return otherwise leaves the count above zero and `Wait` blocks forever.

Pass the `WaitGroup` by pointer if it crosses a function boundary. Copying one gives the callee its own counter, and the original never reaches zero.

Since Go 1.25 there is a shorthand that removes the first two:

```
var wg sync.WaitGroup
for i := 1; i <= 3; i++ {
    wg.Go(func() {              // Add and Done handled for you
        fmt.Println("worker", i)
    })
}
wg.Wait()
```

Collecting results

Goroutines return nothing. A `go` call is a statement, not an expression, so there is no value to take.

Write into a pre-sized slice, one index per goroutine. No lock is needed because no two goroutines touch the same element.

```
type result struct {
    url string
    ms  int64
    err error
}

func fetchAll(urls []string) []result {
    results := make([]result, len(urls))      // sized up front
    var wg sync.WaitGroup

    for i, url := range urls {
        wg.Go(func() {
            start := time.Now()
            resp, err := http.Get(url)
            if err != nil {
                results[i] = result{url: url, err: err}
                return
            }
            defer resp.Body.Close()
            // Read it, or the connection leaks.
            io.Copy(io.Discard, resp.Body)
            results[i] = result{
```

```
                url: url, ms: time.Since(start).Milliseconds()}
        })
    }

    wg.Wait()
    return results
}
```

> **IF YOU KNOW JAVASCRIPT**
> This is `Promise.all`, with the ordering preserved the same way: by index rather than by completion. The difference is that an error in one goroutine does not reject the whole batch. Here every result carries its own error, which is usually what you wanted from `Promise.allSettled` anyway.
>
> Appending to a shared slice instead of indexing would be a data race, because `append` reads and writes the slice header. Index assignment to distinct elements is safe.

Races, and the detector that finds them

Two goroutines touching the same variable, at least one of them writing, with no synchronisation, is a **data race**. The result is not "the wrong number", it is undefined.

```
func main() {
    count := 0
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Go(func() { count++ })    // RACE
    }
    wg.Wait()
    fmt.Println(count)               // 973, 981, 1000, who knows
}
```

`count++` is three operations: read, add, write. Two goroutines interleaving them lose updates.

Go ships a race detector, and it is the most valuable tool in the toolchain:

```
go run -race main.go
go test -race ./...
```

```
WARNING: DATA RACE
Write at 0x00c0000140a8 by goroutine 8:
  main.main.func1()
      /tmp/main.go:12 +0x3c
Previous write at 0x00c0000140a8 by goroutine 7:
  ...
```

> **WATCH OUT**
> Run your tests with `-race` in CI. Always. A data race can run correctly ten million times and corrupt data on the ten-million-and-first, on a different machine, under load. The detector finds it the first time the racing access happens, which turns an unreproducible production mystery into a failing test.
>
> It costs roughly ten times the CPU and a few times the memory, so it is a CI and development tool rather than something you ship.

The fix for the counter is a mutex:

```
var (
    mu    sync.Mutex
    count int
)

wg.Go(func() {
    mu.Lock()
    defer mu.Unlock()
    count++
})
```

Or, for exactly this shape, an atomic:

```
var count atomic.Int64
wg.Go(func() { count.Add(1) })
fmt.Println(count.Load())
```

Bounding how many run at once

Unbounded goroutines are a real hazard. Ten thousand URLs means ten thousand simultaneous connections, which will exhaust file descriptors or get you blocked.

A buffered channel used as a semaphore caps the concurrency:

```
func fetchAll(urls []string, limit int) []result {
    results := make([]result, len(urls))
    sem := make(chan struct{}, limit)    // room for `limit` tokens
    var wg sync.WaitGroup

    for i, url := range urls {
        wg.Go(func() {
            sem <- struct{}{}            // take a token, blocking if full
            defer func() { <-sem }()     // give it back

            // ... fetch, as before
            _ = i
```

```
            _ = url
        })
    }

    wg.Wait()
    return results
}
```

Channels are the next chapter. For now, read `sem <- struct{}{}` as "wait until there is room, then claim a slot".

> **IF YOU KNOW JAVASCRIPT**
> The JavaScript equivalent is a library like `p-limit`, because the language gives you nothing for it. Here it is two lines and a channel, which is a fair summary of the trade Go makes across the board: fewer batteries, better primitives.

When not to use a goroutine

Starting one is cheap, which makes it tempting. Three cases where it is wrong:

When the work is faster than the coordination. Spawning a goroutine to add two numbers costs more than adding them.

When you do not wait for it. A goroutine whose result nobody collects, with no `WaitGroup` and no channel, is a leak. If `main` exits it is killed silently; if it blocks forever it holds its stack forever.

When it needs ordering. Goroutines have no execution order. If step two must follow step one, write them in sequence.

> **PROJECT 12: A CONCURRENT LINK CHECKER**
> Check a list of URLs in parallel, with a concurrency limit and a timeout, and report which are broken. This is a genuinely useful tool and it exercises everything in this chapter.

```
package main

import (
    "fmt"
    "io"
    "net/http"
    "os"
    "sort"
    "sync"
    "time"
)
```

```
type check struct {
    url    string
    status int
    ms     int64
    err    error
}

func checkAll(urls []string, limit int) []check {
    results := make([]check, len(urls))
    sem := make(chan struct{}, limit)
    var wg sync.WaitGroup

    client := &http.Client{Timeout: 10 * time.Second}

    for i, url := range urls {
        wg.Go(func() {
            sem <- struct{}{}
            defer func() { <-sem }()

            start := time.Now()
            resp, err := client.Get(url)
            if err != nil {
                results[i] = check{url: url, err: err}
                return
            }
            defer resp.Body.Close()
            io.Copy(io.Discard, resp.Body)

            results[i] = check{
                url:    url,
                status: resp.StatusCode,
                ms:     time.Since(start).Milliseconds(),
            }
        })
    }

    wg.Wait()
    return results
}

func main() {
    urls := os.Args[1:]
    if len(urls) == 0 {
        fmt.Fprintln(os.Stderr, "usage: check <url>...")
        os.Exit(1)
    }

    start := time.Now()
    results := checkAll(urls, 5)
    total := time.Since(start)
```

```
    // Slowest first, so the problems are at the top.
    sort.Slice(results, func(a, b int) bool {
        return results[a].ms > results[b].ms
    })

    broken := 0
    for _, r := range results {
        switch {
        case r.err != nil:
            broken++
            fmt.Printf("  ERR  %-45s %v\n", r.url, r.err)
        case r.status >= 400:
            broken++
            fmt.Printf("  %d  %-45s %dms\n", r.status, r.url, r.ms)
        default:
            fmt.Printf("  %d  %-45s %dms\n", r.status, r.url, r.ms)
        }
    }

    fmt.Printf("\nchecked %d urls in %v, %d broken\n",
        len(results), total.Round(time.Millisecond), broken)
    if broken > 0 {
        os.Exit(1)
    }
}
```

Run it against a handful of sites and notice that the total time is roughly the **slowest** request, not the sum of all of them. That is the payoff.

Three details that matter in real code. The `http.Client` has a timeout: without one, a hung server holds a goroutine forever and `wg.Wait` never returns. The response body is read and closed, because an unread body leaks the connection. And the exit code is non-zero when something is broken, which is what makes this usable in CI.

Now run it with `-race`. It will be clean, because each goroutine writes only to `results[i]`. Then introduce a race on purpose: add a shared `var count int` and `count++` inside the goroutine, and run with `-race` again to see the detector's report.

YOUR TURN

Add a `-n` flag for the concurrency limit and try the same URL list at 1, 5 and 50. At 1 the total is the sum of all requests. At 5 it drops sharply. At 50 it probably stops improving, because you are now limited by the network or by the servers themselves. Finding the point where more concurrency stops helping is the actual skill, and it is measured rather than guessed.

0

Channels, and the select statement

Goroutines run independently. Channels are how they talk. The slogan Go is famous for is **do not communicate by sharing memory; share memory by communicating**, which sounds like a fortune cookie until you have written both versions of the same program.

A channel is a typed pipe

```
ch := make(chan int)    // unbuffered
ch <- 42                // send
v := <-ch               // receive
```

The arrow always points in the direction the value travels. `ch <- v` puts a value in, `<-ch` takes one out.

```
func main() {
    ch := make(chan string)

    go func() {
        ch <- "hello from the goroutine"
    }()

    msg := <-ch          // blocks until something arrives
    fmt.Println(msg)
}
```

No `time.Sleep` this time. The receive blocks until the send happens, which is synchronisation and message passing in one operation.

> **IF YOU KNOW JAVASCRIPT**
> The closest JavaScript analogy is an async generator or a stream, with one crucial difference: a channel receive **blocks the goroutine**. There is no callback, no `await`, and no promise. The code reads top to bottom, and the runtime parks the goroutine while it waits.
>
> If you have used `postMessage` between workers, the mental model is closer: separate lines of execution passing values, except channels are typed and synchronous by default.

Unbuffered channels are a handshake

An unbuffered channel has no room. A send blocks until a receiver is ready, and a receive blocks until a sender is ready. The two goroutines meet.

```
ch := make(chan int)
ch <- 1                // deadlock: nobody is receiving
```

```
fatal error: all goroutines are asleep - deadlock!
```

Go detects when every goroutine is blocked and stops the program rather than hanging forever. This is a feature, and it is how you will find most channel bugs.

Buffered channels have room

```
ch := make(chan int, 3)  // capacity 3
ch <- 1                  // does not block
ch <- 2
ch <- 3
ch <- 4                  // now it blocks: the buffer is full
```

A send blocks only when the buffer is full; a receive blocks only when it is empty.

> **WATCH OUT**
> Buffering is not a performance trick to sprinkle on. The capacity is a design decision: it is how many items may be in flight before the producer must slow down. That back pressure is usually what you want, because an unbounded queue in front of a slow consumer is just a memory leak with extra steps.
>
> Choose zero when you want a handshake, and a specific number when you know how much slack the system should tolerate. “I made it 100 so it would not block” is how a queue silently grows to 100 and then blocks anyway.

Closing, and ranging

The sender closes a channel to say “no more values”.

```
func produce(ch chan<- int) {
    for i := 0; i < 5; i++ {
        ch <- i
    }
    close(ch)            // only the sender closes
}

func main() {
    ch := make(chan int)
```

```
    go produce(ch)

    for v := range ch {  // ends when the channel is closed
        fmt.Println(v)
    }
}
```

Receiving from a closed channel returns the zero value immediately. The two-value form tells you which happened:

```
v, ok := <-ch
if !ok {
    // the channel is closed and drained
}
```

> **WATCH OUT**
> Three rules that cover every close-related bug:
>
> **Only the sender closes.** A receiver closing a channel causes the sender to panic on its next send.
>
> **Closing twice panics.** So does sending on a closed channel.
>
> **You do not have to close.** A channel that is garbage collected is fine. Close only to signal completion to a `range` or a `select`.

Note `chan<- int` in that signature: a **directional** channel type, meaning send-only. `<-chan int` is receive-only. Using them in function parameters makes the data flow obvious and lets the compiler stop a receiver from closing.

select: waiting on several channels

`select` blocks until one of its cases can proceed.

```
select {
case msg := <-ch1:
    fmt.Println("from ch1:", msg)
case msg := <-ch2:
    fmt.Println("from ch2:", msg)
case ch3 <- "value":
    fmt.Println("sent to ch3")
}
```

If several are ready it picks one at random, which prevents one busy channel starving another.

A `default` case makes it non-blocking:

```
select {
case msg := <-ch:
    fmt.Println(msg)
default:
    fmt.Println("nothing waiting")   // does not block
}
```

IF YOU KNOW JAVASCRIPT
`select` is `Promise.race`, with two differences worth noting. It can wait on sends as well as receives. And the losing cases are not "cancelled", they simply were not chosen: the other channels still hold their values for the next receive. `Promise.race` leaves the losers running and their results are discarded, which is a frequent source of leaked work in JavaScript.

Timeouts, the most common select

```
select {
case result := <-work:
    fmt.Println("finished:", result)
case <-time.After(2 * time.Second):
    fmt.Println("timed out")
}
```

`time.After` returns a channel that delivers a value after the duration. This is the idiomatic timeout, and it is three lines.

WATCH OUT
`time.After` leaks until it fires. In a tight loop, creating one per iteration piles up timers for the full duration. For a loop, create a `time.NewTimer` once and reset it, or use a context deadline, which is the next chapter.

Signalling with a done channel

A channel carrying no data, used only to signal, is a common shape:

```
func worker(done <-chan struct{}) {
    for {
        select {
        case <-done:
            fmt.Println("stopping")
            return
        default:
            // do a unit of work
```

```
        }
    }
}

func main() {
    done := make(chan struct{})
    go worker(done)

    time.Sleep(time.Second)
    close(done)          // closing broadcasts to every receiver
    time.Sleep(10 * time.Millisecond)
}
```

Closing is the broadcast mechanism: **every** goroutine blocked on `<-done` wakes up at once, because a closed channel always yields immediately. Sending a value would wake only one.

This pattern is so common that the standard library wrapped it, and that wrapper is `context`, which is chapter sixteen. Understand the done channel first and `context` becomes obvious rather than magical.

The nil channel, and why it is useful

Sending or receiving on a nil channel blocks forever. That sounds like a pure bug and it has one real use: disabling a `select` case.

```
for {
    select {
    case v, ok := <-in:
        if !ok {
            in = nil        // disable this case, keep the loop alive
            continue
        }
        process(v)
    case <-done:
        return
    }
}
```

Once `in` is nil its case can never be chosen, so the loop keeps serving `done` without spinning on a closed channel returning zeros forever.

Worker pools

The canonical channel program: a fixed number of workers pulling from a jobs channel.

```
func worker(id int, jobs <-chan int, results chan<- string,
    wg *sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs {           // ends when jobs is closed
        results <- fmt.Sprintf("worker %d did job %d", id, j)
    }
}

func main() {
    jobs := make(chan int, 100)
    results := make(chan string, 100)
    var wg sync.WaitGroup

    for w := 1; w <= 3; w++ {
        wg.Add(1)
        go worker(w, jobs, results, &wg)
    }

    for j := 1; j <= 9; j++ {
        jobs <- j
    }
    close(jobs)                     // tells the workers no more are coming

    wg.Wait()                       // wait for them to finish
    close(results)                  // now safe to close

    for r := range results {
        fmt.Println(r)
    }
}
```

The ordering is the whole lesson. Close `jobs` so the workers' `range` ends. `Wait` for the workers. **Then** close `results`, because closing it earlier would panic a worker still writing.

IF YOU KNOW JAVASCRIPT
Compare this to the semaphore from chapter thirteen. Both bound concurrency. The semaphore starts a goroutine per item and limits how many run; the worker pool starts a fixed number and feeds them. The pool is better when goroutine startup matters or when each worker holds an expensive resource, like a database connection. The semaphore is simpler and usually enough.

PROJECT 13: A PIPELINE WITH CANCELLATION
Build a three-stage pipeline where each stage is a goroutine connected by channels, and where the whole thing can be stopped early. This is the shape of a great deal of real Go.

```
package main

import (
    "fmt"
    "strings"
    "sync"
    "time"
)

// Stage 1: produce lines. Returns a channel rather than taking one,
// which
// is the convention: the producer owns and closes what it creates.
func generate(done <-chan struct{}, lines []string) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        for _, l := range lines {
            select {
            case out <- l:
            case <-done:
                return          // stop early without leaking
            }
        }
    }()
    return out
}

// Stage 2: normalise.
func normalise(done <-chan struct{}, in <-chan string) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        for l := range in {
            select {
            case out <- strings.ToLower(strings.TrimSpace(l)):
            case <-done:
                return
            }
        }
    }()
    return out
}

// Stage 3: count words, fanned out across several goroutines.
func count(done <-chan struct{}, in <-chan string,
    workers int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup

    for i := 0; i < workers; i++ {
        wg.Go(func() {
```

```
                for l := range in {
                    time.Sleep(10 * time.Millisecond)   // pretend it is slow
                    select {
                    case out <- len(strings.Fields(l)):
                    case <-done:
                        return
                    }
                }
            })
        }

        // One goroutine whose only job is to close out once every worker is
        // finished. Without this, nothing ever closes out and main hangs.
        go func() {
            wg.Wait()
            close(out)
        }()

        return out
    }

    func main() {
        lines := []string{
            "  The quick brown fox  ", "jumps over", "THE LAZY DOG",
            "and then", "runs away quickly", "into the woods",
        }

        done := make(chan struct{})
        defer close(done)            // every stage stops when main returns

        start := time.Now()
        total := 0
        pipeline := count(done, normalise(done, generate(done, lines)), 3)
        for n := range pipeline {
            total += n
        }

        fmt.Printf("%d words in %v
", total,
            time.Since(start).Round(time.Millisecond))
    }
```

Run it and note the time: six lines at ten milliseconds each would be sixty milliseconds serially, and with three workers it is closer to twenty.

Then do the experiment that proves the cancellation works. Change the loop in `main` to stop after the first two results:

```
n := 0
for range count(done, normalise(done, generate(done, lines)), 3) {
```

```
    n++
    if n == 2 {
        break
    }
}
```

The `defer close(done)` fires, every stage's `select` takes the `<-done` case, and all the goroutines exit. Remove the `done` channel entirely and the producer blocks forever trying to send into a pipeline nobody is reading: a goroutine leak, which in a long-running server is a slow memory leak you will eventually have to find.

Run the whole thing with `-race` as well. It is clean, because every value moves through a channel and nothing is shared.

YOUR TURN

Add a fourth stage between normalise and count that drops any line shorter than three words, and confirm the total changes. Then count how many goroutines the program runs at its peak. If your answer is not "one per stage, plus the workers, plus the closer", re-read `count`: that little goroutine whose only job is `wg.Wait()` then `close(out)` is the piece people forget, and forgetting it is a deadlock rather than a wrong answer.

0

sync: mutexes, and when a channel is the wrong tool

The previous chapter sold channels. This one is the correction: **channels are not always the answer**, and reaching for one where a mutex belongs produces code that is slower and harder to read.

The Go team's own guidance, from the wiki: use a channel to pass ownership of data or to distribute work; use a mutex to protect shared state. The rest of this chapter is what that means in practice.

Mutex: one goroutine at a time

```
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func (c *Counter) Value() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.n
}
```

`Lock` blocks until no other goroutine holds the lock. `defer c.mu.Unlock()` on the line after is the pattern: a path that returns early without unlocking deadlocks the whole program.

> **WATCH OUT**
> Four rules, each the source of a real bug:

The zero value works. `var mu sync.Mutex` is ready. Never `mu := sync.Mutex{}` then copy it around.

Never copy a struct containing a mutex. Copying duplicates the lock, so two copies protect nothing. This is why `Inc` has a pointer receiver. `go vet` catches most of these.

Lock the reads too. A read racing with a write is still a data race, and the race detector will say so. Reading an `int` is not atomic in any sense you can rely on.

Keep the critical section small. Never do I/O, call an unknown function, or send on a channel while holding a lock. That is how a lock becomes a queue and then a deadlock.

Embedding the mutex

Common style, and it reads well:

```
type SafeMap struct {
    sync.Mutex                  // embedded: m.Lock() works directly
    items map[string]int
}

func (m *SafeMap) Set(k string, v int) {
    m.Lock()
    defer m.Unlock()
    m.items[k] = v
}
```

The cost is that `Lock` and `Unlock` become part of the type's public API, so callers can lock your struct. Keep the mutex a named field when the type is exported from a package.

RWMutex: many readers, one writer

When reads vastly outnumber writes, `sync.RWMutex` lets readers run concurrently.

```
type Cache struct {
    mu    sync.RWMutex
    items map[string]string
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock()                // read lock: others may also read
    defer c.mu.RUnlock()
    v, ok := c.items[k]
    return v, ok
}

func (c *Cache) Set(k, v string) {
    c.mu.Lock()                 // write lock: exclusive
```

```
    defer c.mu.Unlock()
    c.items[k] = v
}
```

> **WATCH OUT**
> `RWMutex` is not automatically faster. It has more bookkeeping than a plain `Mutex`, so for short critical sections with moderate contention a `Mutex` often wins. Use `RWMutex` when reads are frequent **and** the critical section is long enough to matter, and confirm with a benchmark rather than a feeling.
>
> Also: never upgrade an `RLock` to a `Lock` while holding it. That deadlocks. Release, acquire the write lock, and re-check the condition, because things may have changed in between.

sync.Once: exactly once, no matter who asks

```
var (
    once   sync.Once
    client *http.Client
)

func Client() *http.Client {
    once.Do(func() {
        client = &http.Client{Timeout: 10 * time.Second}
    })
    return client
}
```

`Do` runs the function once, ever. Concurrent callers block until the first one finishes, so nobody sees a half-built value.

> **IF YOU KNOW JAVASCRIPT**
> This is the lazy singleton you would write in JavaScript as a module-level `let instance` with an `if (!instance)` check. That check is a race in Go, because two goroutines can both see nil. `sync.Once` is the correct version and is shorter anyway.

atomic: for single values

For a counter or a flag, a lock is heavier than necessary.

```
var requests atomic.Int64

func handle() {
    requests.Add(1)
```

```
}

func report() {
    fmt.Println(requests.Load())
}
```

The typed atomics (`atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]`) arrived in Go 1.19 and are much nicer than the old function forms. They cannot be copied, which the compiler enforces.

Use atomics only for a **single** value. The moment two values must change together, you need a mutex, because two atomic operations are not one atomic operation:

```
// WRONG: another goroutine can see total updated and count not.
total.Add(n)
count.Add(1)
```

sync.Map, and why you probably do not want it

There is a concurrent map in the standard library. It is specialised, and the documentation says so plainly: it is optimised for two cases, where a key is written once and read many times, or where goroutines work on disjoint key sets.

For everything else, a plain map with a mutex is faster and far more readable, because `sync.Map` has no type safety: everything is `any` and you assert on the way out.

```
// Usually better:
type Store struct {
    mu sync.RWMutex
    m  map[string]User
}
```

Reach for `sync.Map` after measuring, not before.

When to use which

Channel	Passing ownership of a value, distributing work, signalling an event
Mutex	Protecting state that several goroutines read and write in place
RWMutex	The same, when reads dominate and the section is long
atomic	One number or flag, nothing else
Once	Lazy initialisation
WaitGroup	Waiting for a known set of goroutines to finish

The counter from chapter thirteen makes the point. With a channel:

```
// Works, and is the wrong tool.
func counter(inc <-chan int, get chan<- int) {
    n := 0
    for {
        select {
        case v := <-inc:
            n += v
        case get <- n:
        }
    }
}
```

That is a goroutine, two channels, a `select` loop that never exits, and a lifetime you now have to manage. Against:

```
var n atomic.Int64
n.Add(1)
```

The channel version is the one that looks clever and the atomic is the one you want. **Sharing a number is not communication.**

PROJECT 14: AN IN-MEMORY CACHE WITH EXPIRY

Build something with real shared state: a cache with per-entry expiry and a background cleaner. It needs a mutex, a `Once`, an atomic and a goroutine, and the combination is typical of real service code.

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

type entry struct {
    value   string
    expires time.Time
}

type Cache struct {
    mu      sync.RWMutex
    items   map[string]entry
    hits    atomic.Int64      // counters: atomic, not under the lock
    misses  atomic.Int64
    stopper sync.Once
```

```
    done    chan struct{}
}

func NewCache(cleanEvery time.Duration) *Cache {
    c := &Cache{
        items: map[string]entry{},
        done:  make(chan struct{}),
    }
    go c.clean(cleanEvery)
    return c
}

func (c *Cache) Set(k, v string, ttl time.Duration) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.items[k] = entry{value: v, expires: time.Now().Add(ttl)}
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock()
    e, ok := c.items[k]
    c.mu.RUnlock()            // released before the branch below

    if !ok || time.Now().After(e.expires) {
        c.misses.Add(1)
        return "", false
    }
    c.hits.Add(1)
    return e.value, true
}

func (c *Cache) clean(every time.Duration) {
    t := time.NewTicker(every)
    defer t.Stop()
    for {
        select {
        case <-t.C:
            now := time.Now()
            c.mu.Lock()
            for k, e := range c.items {
                if now.After(e.expires) {
                    // deleting while ranging is legal
                    delete(c.items, k)
                }
            }
            c.mu.Unlock()
        case <-c.done:
            return
        }
    }
}
```

```
// Close is safe to call more than once, which a Once gives us free.
func (c *Cache) Close() {
    c.stopper.Do(func() { close(c.done) })
}

func (c *Cache) Stats() (hits, misses int64) {
    return c.hits.Load(), c.misses.Load()
}

func main() {
    c := NewCache(50 * time.Millisecond)
    defer c.Close()

    c.Set("a", "alpha", 200*time.Millisecond)
    c.Set("b", "bravo", 40*time.Millisecond)

    // Hammer it from several goroutines to prove the locking holds.
    var wg sync.WaitGroup
    for i := 0; i < 50; i++ {
        wg.Go(func() {
            c.Get("a")
            c.Set(fmt.Sprintf("k%d", i), "v", time.Second)
        })
    }
    wg.Wait()

    time.Sleep(100 * time.Millisecond)   // let b expire and be cleaned

    if _, ok := c.Get("b"); !ok {
        fmt.Println("b expired, as expected")
    }
    h, m := c.Stats()
    fmt.Printf("hits=%d misses=%d\n", h, m)

    c.Close()
    c.Close()                            // safe, thanks to Once
    fmt.Println("closed twice without panicking")
}
```

Run it with `-race`. It must be clean.

Then break it three ways, one at a time, and run with `-race` after each. Change `hits` from an `atomic.Int64` to a plain `int64` and watch the detector fire. Change `RLock` to no lock at all in `Get`, same result. Finally remove the `sync.Once` from `Close` and call it twice: the second `close(c.done)` panics, which is the “closing twice” rule from the last chapter meeting a real program.

One design note worth absorbing: `Get` releases the read lock **before** checking expiry and updating counters. The counters are atomic so they need no lock, and holding a lock while doing work you do not have to is the habit that turns a cache into a bottleneck.

YOUR TURN

Add a `Len() int` method and decide which lock it takes. Then add a maximum size, evicting the entry closest to expiry when the cache is full. That second one is harder than it looks: finding the minimum requires scanning under the write lock, which makes `Set` O(n). Think about what you would need to make it fast, and notice you are now designing an LRU, which is why real cache libraries exist.

0

Context: cancellation, deadlines, and request scope

You will see `ctx context.Context` as the first parameter of almost every function in a Go codebase, and it looks like noise until you understand what it buys. It is the done channel from chapter fourteen, standardised so that every library in the ecosystem agrees on how to be cancelled.

The problem it solves

A user opens a page, your handler starts a database query, and the user closes the tab. Without cancellation the query runs to completion, the connection stays busy, and the result is thrown away. Multiply by a thousand users on a slow page and the database falls over serving nobody.

Context is a signal that travels **down** the call stack so that work can stop.

Creating one

You rarely create a context from nothing. Two roots exist:

```
ctx := context.Background()   // the root, in main or a top-level handler
ctx := context.TODO()         // a placeholder: "a context belongs here"
```

`TODO` is for when you are threading context through an existing codebase and have not reached the caller yet. It behaves identically; it exists so that `grep TODO` finds the unfinished work.

From a root you **derive**:

```
ctx, cancel := context.WithCancel(parent)
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
ctx, cancel := context.WithDeadline(parent, someTime)
```

Each returns a child and a `cancel` function.

> **WATCH OUT**

Always call cancel, even on the timeout versions. It is not optional:

```
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
```

Not calling it leaks the timer and the goroutine watching it until the deadline passes. `go vet` catches the obvious cases and not all of them. Writing `defer cancel()` on the very next line is the habit to build.

Using one

A context has a `Done()` channel that closes when the context is cancelled, and an `Err()` saying why.

```
func work(ctx context.Context) error {
    for i := 0; i < 1000; i++ {
        select {
        case <-ctx.Done():
            return ctx.Err()           // Canceled or DeadlineExceeded
        default:
        }
        doChunk(i)
    }
    return nil
}
```

That is the shape: check between units of work, return promptly when told.

For anything that blocks, put `ctx.Done()` in the `select` directly:

```
select {
case result := <-ch:
    return result, nil
case <-ctx.Done():
    return nil, ctx.Err()
}
```

IF YOU KNOW JAVASCRIPT

This is `AbortController` and `AbortSignal`. You create a controller, pass `signal` to `fetch`, and call `abort()`. Go's version has two differences: the convention of passing it as the **first parameter of every function** is near-universal, so the whole ecosystem supports it, and contexts form a tree where cancelling a parent cancels every descendant.

Cancellation is cooperative

Nothing is killed. A context cancellation sets a flag and closes a channel; code that does not check it keeps running.

```
func ignoresCancellation(ctx context.Context) {
    time.Sleep(10 * time.Second)    // runs the full ten seconds regardless
}
```

There are no interrupts and no thread killing in Go. If a library does not take a context, it cannot be cancelled, and that is a reason to prefer libraries that do.

The standard library already honours it

This is where it pays off, because you mostly pass a context rather than check one:

```
req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := http.DefaultClient.Do(req)   // aborts if ctx is cancelled

rows, err := db.QueryContext(ctx, "SELECT ...")   // cancels the query
```

An incoming HTTP request already carries one, cancelled automatically when the client disconnects:

```
func handler(w http.ResponseWriter, r *http.Request) {
    // cancelled if the client goes away
    ctx := r.Context()
    rows, err := db.QueryContext(ctx, "SELECT ...")
    ...
}
```

So passing `r.Context()` into your service layer, which passes it to the database, gives you end-to-end cancellation for free. That is the single most valuable thing context does, and it requires no `select` anywhere in your code.

Timeouts compose

A derived context cannot outlive its parent, which gives you sane behaviour without arithmetic.

```
func handler(w http.ResponseWriter, r *http.Request) {
    // Overall budget for this request.
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()

    // One slow dependency gets a tighter budget.
    userCtx, cancel2 := context.WithTimeout(ctx, 500*time.Millisecond)
```

```
    defer cancel2()
    u, err := users.Get(userCtx, id)
    ...
}
```

If the request's three seconds elapse first, the inner context is cancelled too, even though its own 500ms had not expired.

Values, and why to be sparing

A context can carry request-scoped values.

```
type ctxKey string
const userKey ctxKey = "user"

ctx = context.WithValue(ctx, userKey, currentUser)

u, ok := ctx.Value(userKey).(*User)
```

Use an unexported custom key type, never a plain string, or two packages using `"user"` collide silently.

> **WATCH OUT**
> Context values are the most abused part of the package. They are untyped, invisible in a function signature, and found at runtime or not at all.
>
> The rule from the standard library documentation: use them for **request-scoped data that crosses API boundaries**, such as a request ID, an authenticated user, or a trace span. Never for optional parameters. If a function needs a value to do its job, that value is a parameter:
>
> ```
> // No.
> func Send(ctx context.Context) error    // reads the recipient from ctx
>
> // Yes.
> func Send(ctx context.Context, to string) error
> ```

The conventions

Four, and they are followed almost universally.

Context is the first parameter, named `ctx`. `func Get(ctx context.Context, id int)`.

Never store it in a struct. A context describes one call's lifetime; a struct outlives calls. The exception is `http.Request`, which has its own reasons.

Never pass nil. Use `context.TODO()` if you have nothing better.

Do not pass a context to something that outlives the request. Starting a background job with the request's context means the job is cancelled when the response is sent. Use `context.WithoutCancel(ctx)` to keep the values and drop the cancellation.

PROJECT 15: A SEARCH THAT RACES TWO SOURCES WITH A BUDGET

Query two backends at once, take whichever answers first, give up on both after a deadline, and never leak a goroutine. This is context, channels and select doing the job they exist for.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "math/rand"
    "time"
)

type Result struct {
    Source string
    Hits   int
}

// search simulates a backend that takes an unpredictable amount of time
// and
// respects cancellation.
func search(ctx context.Context, source string,
    slow time.Duration) (Result, error) {
    select {
    case <-time.After(slow):
        return Result{Source: source, Hits: rand.Intn(100)}, nil
    case <-ctx.Done():
        // Stop promptly and say why.
        return Result{}, fmt.Errorf("%s: %w", source, ctx.Err())
    }
}

// first returns whichever source answers first, cancelling the other.
func first(ctx context.Context, budget time.Duration) (Result, error) {
    ctx, cancel := context.WithTimeout(ctx, budget)
    defer cancel()              // cancels the loser, and the timer

    type outcome struct {
        res Result
        err error
    }
    // Buffered with room for both, so the loser's send never blocks and
    // its goroutine always exits. With an unbuffered channel the loser
    // would block forever on the send: a goroutine leak.
```

```
    ch := make(chan outcome, 2)

    go func() {
        r, err := search(ctx, "index-a", 300*time.Millisecond)
        ch <- outcome{r, err}
    }()
    go func() {
        r, err := search(ctx, "index-b", 120*time.Millisecond)
        ch <- outcome{r, err}
    }()

    var lastErr error
    for i := 0; i < 2; i++ {
        select {
        case o := <-ch:
            if o.err == nil {
                return o.res, nil      // first success wins
            }
            lastErr = o.err
        case <-ctx.Done():
            return Result{}, ctx.Err()
        }
    }
    return Result{}, lastErr
}

func main() {
    // Generous budget: the fast index wins.
    r, err := first(context.Background(), time.Second)
    fmt.Printf("got %+v err=%v\n", r, err)

    // Tight budget: both are cancelled.
    r, err = first(context.Background(), 50*time.Millisecond)
    fmt.Printf("got %+v err=%v\n", r, err)
    fmt.Println("deadline:",
        errors.Is(err, context.DeadlineExceeded))
}
```

Three things this program teaches.

The buffered channel is a correctness requirement, not an optimisation. When `first` returns after the winner, the loser still sends. With no buffer that send blocks forever and the goroutine never exits. Change the `2` to a `0`, run with `-race` and then inspect goroutine counts, and you have reproduced one of the most common leaks in Go services.

The `defer cancel()` is what makes the loser stop early rather than running its full three hundred milliseconds into a void.

And `errors.Is(err, context.DeadlineExceeded)` works through the `%w` wrap in `search`, which is chapter eleven's design paying off: the caller can tell a timeout from any other failure.

YOUR TURN

Add a third source and make `first` return as soon as **two** of the three agree, rather than on the first answer. Then add a `context.WithoutCancel` branch that logs the losing results to an audit sink after the winner has been returned, and confirm that work still completes after `first` has returned. That is the pattern for "respond now, finish the bookkeeping afterwards", and getting the context wrong there is why audit records go missing under load.

0

Generics, briefly and honestly

Go got generics in 1.18, after a decade of arguing about whether it should. The result is deliberately less powerful than TypeScript's, and the community consensus is that you should use them rarely. This chapter covers enough to read other people's code and to recognise the handful of cases where writing your own is right.

The syntax

Type parameters go in square brackets before the arguments.

```
func First[T any](items []T) (T, bool) {
    var zero T
    if len(items) == 0 {
        return zero, false
    }
    return items[0], true
}

n, ok := First([]int{1, 2, 3})        // T inferred as int
s, ok := First([]string{"a", "b"})    // T inferred as string
```

`T` is a type parameter and `any` is its **constraint**, meaning "any type at all". Inference usually works, so you rarely write `First[int](...)`.

Note `var zero T`. You cannot return `nil` for an arbitrary `T`, because `T` might be `int`. The zero value is the only thing you can produce without knowing the type, and this little declaration appears in almost every generic function.

> **IF YOU KNOW JAVASCRIPT**
> TypeScript's `function first<T>(items: T[]): T | undefined` is the same idea, with a much more capable type system behind it. TypeScript has conditional types, mapped types, template literal types and inference that borders on computation. Go has type parameters and constraints, and that is the lot. If you arrive expecting the TypeScript toolkit you will be disappointed, which is by design: the proposal was accepted on the promise that it would not make Go code hard to read.

Constraints are interfaces

A constraint says what you may **do** with a value of that type.

```
// Will not compile: you cannot add two values of an arbitrary type.
func Sum[T any](items []T) T {
    var total T
    for _, v := range items {
        // invalid operation: operator + not defined on total
        total += v
    }
    return total
}
```

You must constrain `T` to types that support `+`. Constraints are written as interfaces containing **type sets**:

```
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](items []T) T {
    var total T
    for _, v := range items {
        total += v
    }
    return total
}

Sum([]int{1, 2, 3})          // 6
Sum([]float64{1.5, 2.5})     // 4.0
```

The `~` means "any type whose **underlying** type is this", so a named type still qualifies:

```
type Celsius float64
Sum([]Celsius{20.5, 21.0})   // works because of ~float64
```

Without the tilde, `Celsius` would be rejected, which is almost never what you want. Write `~` unless you have a reason not to.

The constraints you will actually use

Two come from the standard library and cover most cases.

```
import "cmp"

func Max[T cmp.Ordered](a, b T) T {    // anything with < and >
    if a > b { return a }
```

```
    return b
}
```

`cmp.Ordered` is every type supporting `<`, which is the numeric types and `string`. `comparable` is built in and means "supports `==`", which is what map keys require:

```
func Keys[K comparable, V any](m map[K]V) []K {
    out := make([]K, 0, len(m))
    for k := range m {
        out = append(out, k)
    }
    return out
}
```

Generic types

Types can take parameters too.

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) {
    s.items = append(s.items, v)
}

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

s := &Stack[string]{}
s.Push("a")
v, ok := s.Pop()
```

WATCH OUT

Methods cannot introduce new type parameters. This compiles nowhere:

```
func (s *Stack[T]) Map[U any](f func(T) U) *Stack[U] {   // illegal
```

If you want a `Map` that changes the element type, it has to be a package function, not a method:

```
func MapStack[T, U any](s *Stack[T], f func(T) U) *Stack[U] { ... }
```

This single restriction is why Go will never have fluent chained collection methods like `items.map(...).filter(...)`. It was a deliberate choice to keep the type checker simple, and it is the main reason generics changed Go code far less than people expected.

What the standard library gives you

Since 1.21 there are generic helpers, and you should use these rather than write your own.

```
import ("slices"; "maps")

slices.Contains(s, v)        // bool
slices.Index(s, v)           // int, -1 if absent
slices.Sort(s)               // for cmp.Ordered
slices.SortFunc(s, cmp)      // for anything
slices.Reverse(s)
slices.Max(s); slices.Min(s)
slices.Clone(s)
slices.Equal(a, b)

maps.Keys(m)                 // an iterator, not a slice
maps.Values(m)
maps.Clone(m)
```

WATCH OUT

`maps.Keys` returns an **iterator** (`iter.Seq[K]`), not a slice. To get a slice, collect it:

```
keys := slices.Collect(maps.Keys(m))
```

This trips up anybody reading older blog posts, where an earlier experimental version did return a slice.

When to write your own, and when not to

The honest guidance, which matches what the Go team says: **write the concrete version first**. Write the generic version when you have written the same function for the third type and the bodies are identical.

Good reasons:

- A container: a stack, a set, a typed cache.

- A function over any slice or map where the body does not care about the type.
- Removing an `any` and the type assertions that came with it.

Bad reasons:

- "It might be reused later." It usually is not.
- To avoid writing a four-line loop. The loop is clearer.
- To model a hierarchy. Go has interfaces for that, and they are better at it.

> **IF YOU KNOW JAVASCRIPT**
> The decisive difference from TypeScript: TypeScript's generics are erased at runtime and cost nothing, so using them liberally is free. Go's are compiled, and a generic function is often **slower** than the concrete one because of how the compiler shares instantiations for pointer-shaped types. If you are reaching for generics for performance, measure. The usual answer is that the concrete function wins.

Interfaces are still usually the answer

The mistake to avoid is reaching for a type parameter where an interface does the job better.

```
// Generic: the caller must have a concrete type, and the body can only do
// what the constraint allows.
func PrintAll[T fmt.Stringer](items []T) {
    for _, v := range items {
        fmt.Println(v.String())
    }
}

// Interface: accepts a mixed slice of different types. Usually better.
func PrintAll(items []fmt.Stringer) {
    for _, v := range items {
        fmt.Println(v.String())
    }
}
```

The rule of thumb: if the function's behaviour depends only on **methods**, use an interface. Use a type parameter when you need the **actual type** back out, which is what interfaces cannot do.

> **PROJECT 16: A TYPED RESULT CACHE**
> A small generic container that is a genuinely good use of type parameters: the body does not care what it stores, and the caller gets its type back with no assertion.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

type cached[V any] struct {
    value   V
    expires time.Time
}

// Cache is generic over both key and value. K must be comparable because
// it is a map key; V can be anything at all.
type Cache[K comparable, V any] struct {
    mu    sync.RWMutex
    items map[K]cached[V]
    ttl   time.Duration
}

func NewCache[K comparable, V any](ttl time.Duration) *Cache[K, V] {
    return &Cache[K, V]{items: map[K]cached[V]{}, ttl: ttl}
}

func (c *Cache[K, V]) Set(k K, v V) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.items[k] = cached[V]{value: v, expires: time.Now().Add(c.ttl)}
}

func (c *Cache[K, V]) Get(k K) (V, bool) {
    c.mu.RLock()
    e, ok := c.items[k]
    c.mu.RUnlock()

    var zero V
    if !ok || time.Now().After(e.expires) {
        return zero, false
    }
    return e.value, true
}

// GetOrLoad is where the design earns itself: the loader returns a V and
// the caller receives a V, with no any and no type assertion anywhere.
func (c *Cache[K, V]) GetOrLoad(k K,
    load func(K) (V, error)) (V, error) {
    if v, ok := c.Get(k); ok {
        return v, nil
    }
    v, err := load(k)
```

```
    if err != nil {
        var zero V
        return zero, err
    }
    c.Set(k, v)
    return v, nil
}

type User struct {
    ID   int
    Name string
}

func main() {
    users := NewCache[int, User](time.Minute)
    counts := NewCache[string, int](time.Minute)

    calls := 0
    loadUser := func(id int) (User, error) {
        calls++
        return User{ID: id, Name: fmt.Sprintf("user-%d", id)}, nil
    }

    u, _ := users.GetOrLoad(1, loadUser)
    fmt.Println(u.Name, "loader calls:", calls)   // user-1  1

    u, _ = users.GetOrLoad(1, loadUser)
    fmt.Println(u.Name, "loader calls:", calls)   // user-1  1 (cached)

    // u is a User, not an any. This compiles, and would not if the cache
    // stored any.
    fmt.Println("id plus one:", u.ID+1)

    counts.Set("hits", 42)
    n, _ := counts.Get("hits")
    fmt.Println("count doubled:", n*2)            // 84, n is an int
}
```

Now prove the value. Rewrite `Cache` with `map[any]any` and no type parameters, then try `u.ID+1`: you need `u.(User).ID+1`, an assertion that can panic, and the compiler no longer checks that you put a `User` in and got a `User` out. That assertion, repeated at every call site, is exactly what generics removed.

Then try to add a `Map` method that converts a `Cache[K, V]` into a `Cache[K, W]` and watch the compiler refuse, because methods cannot introduce type parameters. Make it a package function instead and notice how that constraint pushes Go code away from fluent chaining.

YOUR TURN

Add a `Keys() []K` method, then write a package-level `Filter[K comparable, V any](c *Cache[K, V], keep func(V) bool) []V`. Then stop and ask whether either of them earns its place, or whether the caller ranging over a plain map would have been clearer. That question, asked honestly every time, is the whole discipline of generics in Go.

0

Testing: the runner is already installed

Go ships a test runner, an assertion-free testing philosophy, a benchmark harness, a coverage tool, a fuzzer and a race detector. There is nothing to install and nothing to configure. After a JavaScript project's testing setup, this takes some adjusting to, mostly because you keep looking for the config file.

The rules

A test file ends in `_test.go`, sits beside the code it tests, and contains functions named `TestXxx` taking `*testing.T`.

```
// math.go
package math

func Add(a, b int) int { return a + b }
```

```
// math_test.go
package math

import "testing"

func TestAdd(t *testing.T) {
    got := Add(2, 3)
    if got != 5 {
        t.Errorf("Add(2, 3) = %d, want 5", got)
    }
}
```

```
go test ./...
```

```
ok      github.com/you/myapp/math    0.002s
```

> **IF YOU KNOW JAVASCRIPT**
> No `describe`, no `it`, no `expect`. A test is a function, and it fails by calling `t.Errorf`. There is no assertion library in the standard library, and this is deliberate: the Go team's

position is that `if got != want` is clearer than learning which of `toBe`, `toEqual` and `toStrictEqual` you wanted.

You will still meet `testify` in real codebases, which adds `assert.Equal(t, want, got)`. It is popular and it is fine. Start without it.

Error messages are the whole skill

A failing test must tell you what happened without opening the source. The convention is `got, want`:

```
t.Errorf("Add(2, 3) = %d, want 5", got)
```

```
    math_test.go:8: Add(2, 3) = 6, want 5
```

That line names the input, the result and the expectation. Compare it to `t.Error("failed")`, which tells you nothing you did not already know.

Errorf against Fatalf

`t.Errorf` records a failure and **keeps going**. `t.Fatalf` records it and stops the test function immediately.

```
u, err := Load(1)
if err != nil {
    t.Fatalf("Load(1): unexpected error: %v", err)   // stop: u is unusable
}
if u.Name != "Ada" {
    t.Errorf("Name = %q, want Ada", u.Name)          // keep checking
}
```

Use `Fatalf` when continuing would panic or produce noise. Use `Errorf` when you want to see every failure in one run.

Table-driven tests

This is the dominant Go testing idiom. One test function, a slice of cases.

```
func TestSlugify(t *testing.T) {
    tests := []struct {
        name string
        in   string
        want string
    }{
        {"simple", "Hello World", "hello-world"},
        {"punctuation", "Hello, World!", "hello-world"},
```

```
        {"already a slug", "hello-world", "hello-world"},
        {"leading space", "  padded  ", "padded"},
        {"empty", "", ""},
        {"unicode", "Café Noir", "cafe-noir"},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Slugify(tt.in); got != tt.want {
                t.Errorf("Slugify(%q) = %q, want %q", tt.in, got, tt.want)
            }
        })
    }
}
```

`t.Run` creates a subtest with its own name, so output identifies the case:

```
--- FAIL: TestSlugify (0.00s)
    --- FAIL: TestSlugify/unicode (0.00s)
        slug_test.go:21: Slugify("Café Noir") =
            "caf-noir", want "cafe-noir"
```

You can run one case directly:

```
go test -run 'TestSlugify/unicode' ./...
```

> **IF YOU KNOW JAVASCRIPT**
> This replaces `describe.each` and `test.each`. The advantage over both is that the cases are an ordinary slice of an ordinary struct, so adding a field to the table is a normal code change and the compiler checks every row.

Comparing things that are not numbers

`==` works on structs of comparable fields. For slices, maps and nested structs, use `reflect.DeepEqual` or, much better, `go-cmp`:

```
import "github.com/google/go-cmp/cmp"

if diff := cmp.Diff(want, got); diff != "" {
    t.Errorf("Load() mismatch (-want +got):\n%s", diff)
}
```

```
    user_test.go:31: Load() mismatch (-want +got):
          user.User{
                ID:    1,
```

```
        -       Name:  "Ada",
        +       Name:  "ada",
                Email: "ada@example.com",
          }
```

A readable diff instead of `false`. This is the one testing dependency worth adding on day one.

Helpers and t.Helper

A helper that reports failures should call `t.Helper()` so the line number points at the **caller**:

```
func mustLoad(t *testing.T, id int) *User {
    t.Helper()                        // failures blame the caller's line
    u, err := Load(id)
    if err != nil {
        t.Fatalf("Load(%d): %v", id, err)
    }
    return u
}
```

Without it, every failure points at the line inside `mustLoad`, which is useless when ten tests call it.

Setup and cleanup

`t.Cleanup` registers work to run when the test finishes, including on failure.

```
func newTestDB(t *testing.T) *sql.DB {
    t.Helper()
    db, err := sql.Open("sqlite", ":memory:")
    if err != nil {
        t.Fatal(err)
    }
    t.Cleanup(func() { db.Close() })    // runs even if the test fails
    return db
}
```

For package-wide setup, `TestMain`:

```
func TestMain(m *testing.M) {
    // before all tests
    code := m.Run()
    // after all tests
    os.Exit(code)
}
```

> **IF YOU KNOW JAVASCRIPT**
> `t.Cleanup` is `afterEach`, scoped to one test and registered where the resource is created rather than in a separate block. `TestMain` is `globalSetup` and `globalTeardown`. There is no `beforeEach`: you call a helper at the top of each test, which is more typing and means a reader can see what a test depends on without scrolling.

Parallel tests

```go
func TestSomething(t *testing.T) {
    t.Parallel()
    // ...
}
```

Tests in different packages already run in parallel. `t.Parallel()` marks tests **within** a package as safe to interleave.

> **WATCH OUT**
> A parallel test must not share mutable state with another test. Writing to the same database table, the same temp file or the same package-level variable will fail intermittently, which is worse than failing reliably.
>
> Combine `t.Parallel()` with `-race` so that a shared-state mistake is reported rather than merely flaky.

Testing errors

Test the **kind**, not the message. Message text is not a contract.

```go
// Brittle: breaks when someone improves the wording.
if err.Error() != "user not found" { ... }

// Correct.
if !errors.Is(err, ErrNotFound) {
    t.Errorf("got %v, want ErrNotFound", err)
}
```

And always test that an error **is** returned when you expect one:

```go
{"missing email", "", "", true},   // a wantErr field in the table
...
_, err := Create(tt.name, tt.email)
if (err != nil) != tt.wantErr {
    t.Errorf("Create() error = %v, wantErr %v", err, tt.wantErr)
}
```

Coverage

```
go test -cover ./...
go test -coverprofile=cover.out ./... && go tool cover -html=cover.out
```

The second opens a browser showing every line in green or red. It is the fastest way to find the branch nobody tested.

> **WATCH OUT**
> Coverage measures lines executed, not behaviour verified. A test that calls every function and asserts nothing reports high coverage and proves nothing. Use it to find what you **missed**, never as a target to hit.

Benchmarks

```
func BenchmarkSlugify(b *testing.B) {
    for b.Loop() {
        Slugify("Hello, World! This is a title")
    }
}
```

```
go test -bench=. -benchmem ./...
```

```
BenchmarkSlugify-8    2419371    489.2 ns/op    144 B/op    3 allocs/op
```

Operations per second, nanoseconds each, bytes allocated and allocation count. `b.Loop()` replaced the older `for i := 0; i < b.N; i++` in Go 1.24 and is harder to get wrong, because it stops the compiler optimising your work away.

> **PROJECT 17: TEST THE USER STORE PROPERLY**
> Take the store from chapter eleven and give it a real test suite: table driven, testing error kinds rather than messages, with a helper and cleanup.
>
> ```
> package store
>
> import (
>     "errors"
>     "testing"
> )
>
> func newStore(t *testing.T) *Store {
>     t.Helper()
>     s := NewStore()
>     t.Cleanup(func() {
> ```

```
        // Nothing to close here, but the hook is where a real test would
        // drop a temp database. Wire it now so it is not retrofitted
        // later.
    })
    return s
}

func TestCreate(t *testing.T) {
    tests := []struct {
        name    string
        user    string
        email   string
        wantErr error       // nil, a sentinel, or errSomeValidation
        field   string      // for validation errors
    }{
        {"valid", "Ada", "ada@example.com", nil, ""},
        {"empty name", "", "x@example.com", errValidation, "name"},
        {"bad email", "Ada", "not-an-email", errValidation, "email"},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel()
            s := newStore(t)

            u, err := s.Create(tt.user, tt.email)

            if tt.wantErr == nil {
                if err != nil {
                    t.Fatalf("Create(%q, %q): unexpected error: %v",
                        tt.user, tt.email, err)
                }
                if u.Name != tt.user {
                    t.Errorf("Name = %q, want %q", u.Name, tt.user)
                }
                if u.ID == 0 {
                    t.Error("ID was not assigned")
                }
                return
            }

            if err == nil {
                t.Fatalf("Create(%q, %q): want error, got nil",
                    tt.user, tt.email)
            }
            var ve *ValidationError
            if !errors.As(err, &ve) {
                t.Fatalf("got %v, want a *ValidationError", err)
            }
            if ve.Field != tt.field {
                t.Errorf("Field = %q, want %q", ve.Field, tt.field)
```

```
            }
        })
    }
}

func TestCreateDuplicateEmail(t *testing.T) {
    t.Parallel()
    s := newStore(t)

    if _, err := s.Create("Ada", "ada@example.com"); err != nil {
        t.Fatalf("first create: %v", err)
    }
    _, err := s.Create("Grace", "ada@example.com")

    // The identity of the failure is the contract. The wording is not.
    if !errors.Is(err, ErrConflict) {
        t.Errorf("got %v, want ErrConflict", err)
    }
}

func TestGetMissing(t *testing.T) {
    t.Parallel()
    s := newStore(t)

    if _, err := s.Get(999); !errors.Is(err, ErrNotFound) {
        t.Errorf("got %v, want ErrNotFound", err)
    }
}
```

Run it four ways and notice what each tells you.

`go test ./...` gives you pass or fail. `go test -v ./...` names every subtest, which is how you confirm the table ran every row rather than silently skipping. `go test -race ./...` would catch the shared-state bug if `newStore` returned a package-level store instead of a fresh one, which is the mistake `t.Parallel()` makes dangerous. And `go test -cover ./...` tells you which branch of `Create` you forgot.

The important design point is in `TestCreateDuplicateEmail`: it asserts `errors.Is(err, ErrConflict)` and never looks at the message. Change the wording of that error and the test still passes, which is correct, because the wording was never the promise.

YOUR TURN

Add a benchmark for `Create` and run it with `-benchmem`. Then change the duplicate-email check from a linear scan to a second map keyed by email, and run the benchmark again with a thousand existing users. The allocation count and the nanoseconds per

> operation will both move, and seeing a design change show up as a number is the habit this chapter is really teaching.

0

JSON, struct tags, and the capital letter that bites

JSON is where your Go backend meets your React frontend, so this chapter earns its place twice: once now, and again in part four when Grit generates the types on both sides of that boundary.

Encoding and decoding

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
}

u := User{ID: 1, Name: "Ada", Email: "ada@example.com"}

data, err := json.Marshal(u)
// {"id":1,"name":"Ada","email":"ada@example.com"}

var back User
err = json.Unmarshal(data, &back)     // note the &
```

`Unmarshal` takes a pointer, because it fills in the value you give it. Passing `back` instead of `&back` compiles and silently does nothing useful, which is the chapter's first trap.

For readable output:

```
data, _ := json.MarshalIndent(u, "", "  ")
```

The capital letter rule, again

This is the single most common Go and JSON bug, and you met the rule in chapter two.

```
type User struct {
    id    int       // lowercase: unexported
    name  string
}
```

```
json.Marshal(User{id: 1, name: "Ada"})   // {}
```

An empty object. No error, no warning. The `encoding/json` package lives in another package and literally cannot see unexported fields.

> **WATCH OUT**
> **If your JSON comes out as `{}`, check for lowercase field names first.** It is the answer about nine times out of ten. The same applies on the way in: an unexported field is never populated by `Unmarshal`.

Struct tags in full

```
type Product struct {
    ID        int       `json:"id"`
    Name      string    `json:"name"`
    Price     float64   `json:"price"`
    Notes     string    `json:"notes,omitempty"`   // omitted when ""
    Internal  string    `json:"-"`                 // never encoded
    CreatedAt time.Time `json:"created_at"`
    Count     int       `json:"count,string"`      // encoded as "42"
}
```

`json:"name"`	Use this key
`json:"-"`	Never encode or decode this field
`json:",omitempty"`	Omit when the value is the zero value
`json:",string"`	Encode a number or bool as a JSON string
no tag	Use the Go field name, capital included

> **WATCH OUT**
> The tag syntax is unforgiving and silent. All of these are wrong and none produce an error:
>
> ```
> Name string `json: "name"`     // space after the colon: tag ignored
> Name string `json:'name'`      // single quotes: tag ignored
> Name string "json:\"name\""    // works, but use backticks
> ```
>
> The field then encodes as `Name`. If one key in your JSON is capitalised and the rest are not, you have found a stray space.
>
> `go vet` catches malformed tags. Run it.

omitempty, and what it actually means

`omitempty` omits the **zero value**, which is rarely what people assume.

```
type Filter struct {
    Page     int    `json:"page,omitempty"`      // 0 is omitted
    Active   bool   `json:"active,omitempty"`    // false is omitted!
    Query    string `json:"query,omitempty"`     // "" is omitted
}
```

`Active: false` disappears from the output. If false is a meaningful value and not merely "unset", `omitempty` is wrong and will cause a real bug.

The fix is a pointer, which is chapter eight's lesson in its natural habitat:

```
type Filter struct {
    Active *bool `json:"active,omitempty"`   // nil omitted, &false encoded
}
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript has no such ambiguity because `undefined` and `false` are different values. Go's zero values collapse them, so a `*bool` is how you get the distinction back. You will see `*string` and `*bool` all over the struct definitions of any API that supports partial updates, and now you know exactly why.

Decoding unknown shapes

Without a struct, JSON decodes into `map[string]any`:

```
var data map[string]any
json.Unmarshal(raw, &data)

name := data["name"].(string)       // panics if it is not a string
name, ok := data["name"].(string)   // safe
```

Every number becomes a `float64`, which surprises everyone:

```
json.Unmarshal([]byte(`{"count": 42}`), &data)
n := data["count"].(int)        // PANIC: it is a float64
n := int(data["count"].(float64))   // correct
```

Prefer a struct whenever you know the shape. The type system is the point.

Decoding is case-insensitive, and lenient

Two behaviours that catch people:

```
type User struct {
    Name string `json:"name"`
}

json.Unmarshal([]byte(`{"NAME": "Ada"}`), &u)   // works: case-insensitive
// works: unknown fields ignored
json.Unmarshal([]byte(`{"nope": 1}`), &u)
```

Unknown fields are silently dropped. For an API that should reject nonsense, use a decoder:

```
dec := json.NewDecoder(r.Body)
dec.DisallowUnknownFields()
if err := dec.Decode(&input); err != nil {
    http.Error(w, "bad request: "+err.Error(), http.StatusBadRequest)
    return
}
```

This turns a typo in a client's payload into a 400 instead of a silently ignored field, which is usually what you want for a write endpoint.

Streaming with Encoder and Decoder

For an HTTP body, do not read it all into memory first:

```
func handler(w http.ResponseWriter, r *http.Request) {
    var input CreateUser
    if err := json.NewDecoder(r.Body).Decode(&input); err != nil {
        http.Error(w, "invalid json", http.StatusBadRequest)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(response)
}
```

`Decoder` reads from any `io.Reader` and `Encoder` writes to any `io.Writer`, which is chapter ten's small interfaces paying off again.

Times, and the format you will meet

`time.Time` marshals to RFC 3339:

```
{"created_at": "2026-10-20T14:30:00Z"}
```

That parses directly with `new Date(...)` in JavaScript, so the default is the right one. A zero time encodes as `"0001-01-01T00:00:00Z"`, which looks alarming in output; use a `*time.Time` with `omitempty` when the field is genuinely optional.

Custom marshalling

Implement `MarshalJSON` to control a type's representation:

```
type Money int64      // stored as cents

func (m Money) MarshalJSON() ([]byte, error) {
    return []byte(fmt.Sprintf(`"%d.%02d"`, m/100, m%100)), nil
}

func (m *Money) UnmarshalJSON(data []byte) error {
    var s string
    if err := json.Unmarshal(data, &s); err != nil {
        return err
    }
    // parse "12.34" into 1234 cents
    ...
}
```

Note the receivers: `MarshalJSON` on the value, `UnmarshalJSON` on the pointer, because unmarshalling has to modify. Getting this backwards means your custom decoder is silently never called, which is the second-most-common JSON bug after lowercase fields.

IF YOU KNOW JAVASCRIPT

This is `toJSON()` and a reviver function, combined into an interface. Money is the classic case: a float64 cannot represent 0.10 exactly, so storing cents as an integer and formatting at the boundary is the correct design in any language. Grit has a `money` field type that does exactly this, and part four explains why it is a type rather than advice.

PROJECT 18: A JSON REFORMATTING CLI

Build `jsonfmt`: read JSON on standard input or from a file, validate it, pretty print it, and optionally extract one field. This is a tool you will actually use, and it exercises streaming, unknown shapes and error handling.

```
package main

import (
    "encoding/json"
    "flag"
```

```
    "fmt"
    "io"
    "os"
    "strings"
)

func main() {
    indent := flag.String("indent", "  ", "indent string")
    compact := flag.Bool("c", false,
        "compact output instead of indented")
    path := flag.String("get", "",
        "dotted path to extract, e.g. user.name")
    flag.Parse()

    // Read from a named file, or standard input when piped.
    var r io.Reader = os.Stdin
    if args := flag.Args(); len(args) > 0 {
        f, err := os.Open(args[0])
        if err != nil {
            fail("opening %s: %v", args[0], err)
        }
        defer f.Close()
        r = f
    }

    // any, not map[string]any: the document might be an array.
    var doc any
    dec := json.NewDecoder(r)
    dec.UseNumber()            // keep numbers exact instead of float64
    if err := dec.Decode(&doc); err != nil {
        fail("invalid json: %v", err)
    }

    if *path != "" {
        v, err := get(doc, *path)
        if err != nil {
            fail("%v", err)
        }
        doc = v
    }

    enc := json.NewEncoder(os.Stdout)
    if !*compact {
        enc.SetIndent("", *indent)
    }
    if err := enc.Encode(doc); err != nil {
        fail("writing: %v", err)
    }
}

// get walks a dotted path through decoded JSON.
```

```
func get(doc any, path string) (any, error) {
    cur := doc
    for _, part := range strings.Split(path, ".") {
        m, ok := cur.(map[string]any)
        if !ok {
            return nil, fmt.Errorf("%q: not an object at %q", path, part)
        }
        cur, ok = m[part]
        if !ok {
            return nil, fmt.Errorf("%q: no such key %q", path, part)
        }
    }
    return cur, nil
}

func fail(format string, args ...any) {
    fmt.Fprintf(os.Stderr, "jsonfmt: "+format+"\n", args...)
    os.Exit(1)
}
```

Build it and try it:

```
go build -o jsonfmt .
echo '{"user":{"name":"Ada","id":1},"tags":["a","b"]}' | ./jsonfmt
echo '{"user":{"name":"Ada"}}' | ./jsonfmt -get user.name
echo '{"bad": }' | ./jsonfmt           # exits 1 with a message on stderr
```

Two details worth understanding. `dec.UseNumber()` stops every number becoming a `float64`, so a large integer ID survives the round trip instead of losing precision: without it, `{"id": 12345678901234567}` comes back subtly wrong. And `doc` is declared as `any` rather than `map[string]any`, so a top-level JSON array works too.

The `...any` in `fail` is a variadic parameter, Go's rest argument. Passing it on to `Fprintf` with `args...` spreads it again, which is the Go equivalent of `...args` in both directions.

YOUR TURN

Add a `-keys` flag that prints the top-level keys one per line, sorted. You will need a type switch on `doc`, because the document might be an object or an array, and the two cases want different answers. Then add `DisallowUnknownFields` behind a `-strict` flag and find out what it does to a document with no struct to compare against. The answer tells you something real about what that option is actually for.

0

Files, streams and the io interfaces

Two interfaces carry most of Go's I/O, and you met them in chapter ten:

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
```

One method each. Files, network connections, HTTP bodies, gzip streams, in-memory buffers, standard input and standard output all satisfy them, which means a function written against `io.Reader` works with every one of those without knowing which it has.

Reading a whole file

When the file is small and you want it all:

```
data, err := os.ReadFile("config.json")   // []byte
if err != nil {
    return fmt.Errorf("reading config: %w", err)
}
text := string(data)
```

And writing:

```
err := os.WriteFile("out.txt", []byte("hello"), 0o644)
```

`0o644` is the Unix permission mode: owner read and write, everyone else read. On Windows it is mostly ignored. `0o600` for anything secret.

> **WATCH OUT**
> `os.ReadFile` loads the entire file into memory. For a 2GB log file that is 2GB of RAM, and your process is killed. Use it for configuration and small documents; stream anything that could be large, which is the rest of this chapter.

Streaming line by line

```
f, err := os.Open("access.log")
if err != nil {
```

```
    return err
}
defer f.Close()

scanner := bufio.NewScanner(f)
for scanner.Scan() {
    line := scanner.Text()
    // ...
}
if err := scanner.Err(); err != nil {
    return fmt.Errorf("reading access.log: %w", err)
}
```

Memory stays constant regardless of file size.

WATCH OUT

You must check `scanner.Err()`. The loop ends identically whether the file finished or the disk failed, and without the check a read error looks exactly like a successful end. This is the most commonly skipped error check in Go.

Second trap: `Scanner` refuses lines longer than 64KB by default, returning `bufio.Scanner: token too long`. For long lines, enlarge the buffer:

```
scanner.Buffer(make([]byte, 0, 1024*1024), 1024*1024)   // 1MB lines
```

os.Open against os.Create

`os.Open`	Read only. Fails if the file is missing
`os.Create`	Write only. Creates, or truncates an existing file to empty
`os.OpenFile`	Everything else: append, read-write, custom permissions

`os.Create` truncating is worth remembering, because it means opening a file to "check something" with `Create` destroys it. To append:

```
f, err := os.OpenFile("app.log",
    os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0o644)
```

Buffer your writes

Every `Write` on a file is a system call. Writing a million lines one at a time is a million system calls.

```
f, err := os.Create("out.txt")
if err != nil {
    return err
}
defer f.Close()

w := bufio.NewWriter(f)
defer w.Flush()            // without this, you lose the tail of the file

for _, line := range lines {
    fmt.Fprintln(w, line)
}
```

WATCH OUT

A buffered writer must be flushed. The last few kilobytes sit in memory until you do. `defer w.Flush()` immediately after creating it is the habit, and note the order: `defer f.Close()` was registered first so it runs **last**, which is correct, because flushing into a closed file fails.

Better still, check the flush error, because a failed flush means data did not reach the disk:

```
defer func() {
    if ferr := w.Flush(); ferr != nil && err == nil {
        err = fmt.Errorf("flushing: %w", ferr)
    }
}()
```

io.Copy, and the pattern it enables

```
io.Copy(dst, src)      // stream everything from src to dst
```

That one line downloads a file to disk, with constant memory:

```
resp, err := http.Get(url)
if err != nil {
    return err
}
defer resp.Body.Close()

f, err := os.Create("download.bin")
if err != nil {
    return err
}
defer f.Close()
```

```
written, err := io.Copy(f, resp.Body)
```

An HTTP response body is an `io.Reader`. A file is an `io.Writer`. Neither knows about the other.

Useful companions:

```
io.Discard                      // a Writer that throws everything away
// read at most 1MB: use on untrusted input
io.LimitReader(r, 1<<20)
io.MultiWriter(f, os.Stdout)    // write to both at once, like tee
io.TeeReader(r, logFile)        // read from r while copying to logFile
```

WATCH OUT

`io.LimitReader` on anything a user controls is a real defence. Without it, a request body or an uploaded file can be as large as the client wants, and reading it into memory is a denial of service that costs the attacker nothing. For HTTP specifically there is `http.MaxBytesReader`, which also returns a proper error.

Writing testable code against interfaces

This is the practical payoff. A function that takes concrete types is hard to test:

```
// Hard: needs real files on disk.
func Process(inPath, outPath string) error
```

The same function over interfaces is trivial to test:

```
func Process(r io.Reader, w io.Writer) error {
    scanner := bufio.NewScanner(r)
    for scanner.Scan() {
        fmt.Fprintln(w, strings.ToUpper(scanner.Text()))
    }
    return scanner.Err()
}
```

```
func TestProcess(t *testing.T) {
    in := strings.NewReader("hello\nworld\n")
    var out bytes.Buffer

    if err := Process(in, &out); err != nil {
        t.Fatal(err)
    }
    if got, want := out.String(), "HELLO\nWORLD\n"; got != want {
```

```
        t.Errorf("got %q, want %q", got, want)
    }
}
```

No temp files, no cleanup, no filesystem. `strings.NewReader` and `bytes.Buffer` are the two types you will use most in tests.

> **IF YOU KNOW JAVASCRIPT**
> This is why Node's stream API exists, and Go's version is far smaller: one method per direction instead of an event emitter with `data`, `end`, `error` and backpressure semantics you have to learn. The tradeoff is that Go's streams are synchronous and blocking, which is fine because blocking a goroutine is cheap.

Paths, portably

```
import "path/filepath"

// data/users/ada.json or data\users\ada.json
filepath.Join("data", "users", "ada.json")
filepath.Ext("report.pdf")                  // ".pdf"
filepath.Base("/tmp/report.pdf")            // "report.pdf"
filepath.Dir("/tmp/report.pdf")             // "/tmp"
```

Use `filepath`, not `path`, and never build a path with string concatenation. `path` is for URLs and always uses forward slashes; `filepath` adapts to the operating system.

> **WATCH OUT**
> Never join a user-supplied name onto a directory without cleaning it:
>
> ```
> // Directory traversal: "../../etc/passwd" escapes the directory.
> f, _ := os.Open(filepath.Join("uploads", userInput))
> ```
>
> `filepath.Join` does call `Clean`, which resolves the `..`, but the result can still point outside `uploads`. The check is explicit:
>
> ```
> full := filepath.Join(base, filepath.Clean("/"+userInput))
> if !strings.HasPrefix(full, base+string(os.PathSeparator)) {
>     return errors.New("invalid path")
> }
> ```
>
> Go 1.24 added `os.Root` for this, which opens a directory and refuses to escape it, and is the better answer where available.

Temporary files and directories

```
dir, err := os.MkdirTemp("", "myapp-*")
if err != nil {
    return err
}
defer os.RemoveAll(dir)

f, err := os.CreateTemp(dir, "upload-*.tmp")
```

In tests, `t.TempDir()` does this and cleans up automatically:

```
func TestWrite(t *testing.T) {
    dir := t.TempDir()        // removed when the test ends, pass or fail
    path := filepath.Join(dir, "out.txt")
    ...
}
```

PROJECT 19: A LOG ANALYSER

Bring together everything from Parts I and II: stream files that may be large, parse them, aggregate with maps, sort, and process several files concurrently.

```
package main

import (
    "bufio"
    "flag"
    "fmt"
    "io"
    "os"
    "slices"
    "strconv"
    "strings"
    "sync"
)

// A line looks like:
// 192.168.1.1 GET /api/users 200 134ms
type stats struct {
    requests int
    byStatus map[int]int
    byPath   map[string]int
    totalMS  int
    slowest  string
    sloweste int
}

func newStats() *stats {
    return &stats{byStatus: map[int]int{}, byPath: map[string]int{}}
```

```
}

// analyse takes an io.Reader, not a filename, so it is testable with a
// strings.Reader and works on stdin, a file or a network stream alike.
func analyse(r io.Reader) (*stats, error) {
    s := newStats()
    sc := bufio.NewScanner(r)
    sc.Buffer(make([]byte, 0, 256*1024), 256*1024)

    for sc.Scan() {
        f := strings.Fields(sc.Text())
        if len(f) < 5 {
            // skip malformed lines rather than failing
            continue
        }
        status, err := strconv.Atoi(f[3])
        if err != nil {
            continue
        }
        ms, err := strconv.Atoi(strings.TrimSuffix(f[4], "ms"))
        if err != nil {
            continue
        }

        s.requests++
        s.byStatus[status]++
        s.byPath[f[2]]++
        s.totalMS += ms
        if ms > s.sloweste {
            s.sloweste, s.slowest = ms, f[2]
        }
    }
    // The check that is always skipped and always matters.
    if err := sc.Err(); err != nil {
        return nil, fmt.Errorf("scanning: %w", err)
    }
    return s, nil
}

func (s *stats) merge(o *stats) {
    s.requests += o.requests
    s.totalMS += o.totalMS
    for k, v := range o.byStatus {
        s.byStatus[k] += v
    }
    for k, v := range o.byPath {
        s.byPath[k] += v
    }
    if o.sloweste > s.sloweste {
        s.sloweste, s.slowest = o.sloweste, o.slowest
    }
```

```
}

func main() {
    top := flag.Int("top", 5, "how many paths to show")
    flag.Parse()

    paths := flag.Args()
    if len(paths) == 0 {
        // No files given: read standard input, so the tool pipes.
        s, err := analyse(os.Stdin)
        if err != nil {
            fail(err)
        }
        report(s, *top)
        return
    }

    // One goroutine per file, each with its own stats to avoid any
    // shared
    // state, merged at the end under a mutex.
    total := newStats()
    var mu sync.Mutex
    var wg sync.WaitGroup
    var failed int

    for _, p := range paths {
        wg.Go(func() {
            f, err := os.Open(p)
            if err != nil {
                mu.Lock()
                failed++
                mu.Unlock()
                fmt.Fprintf(os.Stderr, "skipping %s: %v\n", p, err)
                return
            }
            defer f.Close()

            s, err := analyse(f)
            if err != nil {
                fmt.Fprintf(os.Stderr, "reading %s: %v\n", p, err)
                return
            }
            mu.Lock()
            total.merge(s)
            mu.Unlock()
        })
    }
    wg.Wait()

    report(total, *top)
    if failed > 0 {
```

```
        os.Exit(1)
    }
}

func report(s *stats, top int) {
    if s.requests == 0 {
        fmt.Println("no requests found")
        return
    }
    fmt.Printf("requests   %d\n", s.requests)
    fmt.Printf("average    %dms\n", s.totalMS/s.requests)
    fmt.Printf("slowest    %s (%dms)\n\n", s.slowest, s.sloweste)

    codes := make([]int, 0, len(s.byStatus))
    for c := range s.byStatus {
        codes = append(codes, c)
    }
    slices.Sort(codes)
    for _, c := range codes {
        fmt.Printf("  %d  %d\n", c, s.byStatus[c])
    }

    type pair struct {
        path string
        n    int
    }
    paths := make([]pair, 0, len(s.byPath))
    for p, n := range s.byPath {
        paths = append(paths, pair{p, n})
    }
    slices.SortFunc(paths, func(a, b pair) int { return b.n - a.n })

    fmt.Println()
    for _, p := range paths[:min(top, len(paths))] {
        fmt.Printf("  %-30s %d\n", p.path, p.n)
    }
}

func fail(err error) {
    fmt.Fprintln(os.Stderr, "loganalyse:", err)
    os.Exit(1)
}
```

Make a test file and run it:

```
cat > access.log <<'EOF'
1.1.1.1 GET /api/users 200 134ms
1.1.1.2 GET /api/users 200 89ms
1.1.1.3 POST /api/login 401 12ms
1.1.1.4 GET /api/reports 200 2400ms
EOF
```

```
go run . access.log
cat access.log | go run .
```

Three design choices worth noticing. `analyse` takes an `io.Reader`, so the same function serves a file, standard input, and a `strings.Reader` in a test: write that test, it is four lines. Each goroutine builds its own `stats` and merges once at the end, so the lock is held briefly rather than on every line. And malformed lines are skipped rather than fatal, because a log analyser that dies on one bad line is useless against real logs.

YOUR TURN

Write the test for `analyse` using `strings.NewReader`, covering a good line, a malformed line and an empty input. Then add a `-json` flag that emits the stats as JSON instead of text, which needs the capital-letter rule and struct tags from the last chapter. Finally, feed it a file with a 100KB single line and watch where the buffer limit bites if you remove the `sc.Buffer` call.

0

net/http: a server with no framework

Go's standard library includes a production-grade HTTP server. Not a toy, not a reference implementation: the thing you actually deploy. This chapter builds a real one with no dependencies at all, because understanding this makes every framework afterwards, Gin and Grit included, read as a convenience rather than magic.

Five lines

```go
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/hello",
        func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello")
    })
    http.ListenAndServe(":8080", nil)
}
```

```
go run . &
curl localhost:8080/hello
```

No `npm install`, no framework, no config. That server handles concurrent requests across every core, because `net/http` starts a goroutine per request.

> **IF YOU KNOW JAVASCRIPT**
> Express needs `npm install express` and gives you `app.get('/hello', (req, res) => res.send('hello'))`. The shapes are similar. The differences: Go's server is in the standard library and versioned with the language, and it is concurrent by default rather than on one event loop.

The two core types

```
func handler(w http.ResponseWriter, r *http.Request)
```

`*http.Request` is the incoming request: method, URL, headers, body, context. `http.ResponseWriter` is where you write the response. Note the asymmetry: the request is a pointer to a struct you read, the writer is an interface you call.

```
func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Println(r.Method)                 // "GET"
    fmt.Println(r.URL.Path)               // "/users/42"
    fmt.Println(r.URL.Query().Get("q"))   // ?q=search
    fmt.Println(r.Header.Get("Authorization"))

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)     // 201
    w.Write([]byte(`{"ok":true}`))
}
```

WATCH OUT

Order matters and the compiler will not help you. Headers must be set before `WriteHeader`, and `WriteHeader` before `Write`. Writing the body first implicitly sends a 200, and your later `WriteHeader(404)` is ignored with a log line:

```
http: superfluous response.WriteHeader call
```

The rule: set headers, then status, then body. Always that order.

Routing with ServeMux

Since Go 1.22 the standard router handles methods and path parameters, which removed the main reason people reached for a third-party router.

```
mux := http.NewServeMux()

mux.HandleFunc("GET /users", listUsers)
mux.HandleFunc("POST /users", createUser)
mux.HandleFunc("GET /users/{id}", getUser)
mux.HandleFunc("DELETE /users/{id}", deleteUser)

http.ListenAndServe(":8080", mux)
```

Path parameters come out by name:

```
func getUser(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    ...
}
```

A request that matches the path but not the method gets a 405 automatically.

> **IF YOU KNOW JAVASCRIPT**
> This is `app.get('/users/:id', ...)` with `req.params.id`. Two differences: the method is part of the pattern string rather than the function name, and there is no `next()`, because middleware composes by wrapping rather than by calling through.

Always configure a Server

`http.ListenAndServe` uses a default server with **no timeouts at all**. A single slow client can hold a connection open indefinitely, and enough of them exhaust your file descriptors. This is a real denial of service and it is the default.

```
srv := &http.Server{
    Addr:              ":8080",
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,   // slowloris defence
    ReadTimeout:       15 * time.Second,
    WriteTimeout:      15 * time.Second,
    IdleTimeout:       60 * time.Second,
    MaxHeaderBytes:    1 << 20,
}
log.Fatal(srv.ListenAndServe())
```

> **WATCH OUT**
> Set `ReadHeaderTimeout` at minimum. Without it, a client that opens a connection and sends one byte per minute occupies a goroutine and a file descriptor forever. This attack is called slowloris, it is twenty years old, and Go's defaults do not defend against it because the standard library cannot know your deployment.

Middleware is a function that wraps a handler

There is no middleware API. A middleware is a function taking a handler and returning a handler.

```
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
```

```
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}
```

Compose by nesting:

```
handler := Logger(Recover(Auth(mux)))
```

The outermost runs first. Reading that line right to left gives you the request path, which is why many codebases add a small helper:

```
func Chain(h http.Handler,
    m ...func(http.Handler) http.Handler) http.Handler {
    for i := len(m) - 1; i >= 0; i-- {
        h = m[i](h)
    }
    return h
}

handler := Chain(mux, Logger, Recover, Auth)   // now reads in order
```

IF YOU KNOW JAVASCRIPT

Express middleware takes `(req, res, next)` and calls `next()` to continue. Go's takes a handler and **returns** one, so "continue" is calling `next.ServeHTTP(w, r)` and "stop" is simply not calling it:

```
func Auth(next http.Handler) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
        if r.Header.Get("Authorization") == "" {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            // next is never called: stop here
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

No `next(err)`, no forgetting to call `next()` and hanging the request.

Passing values down: context, not globals

Middleware that authenticates a user needs to hand it to the handler. The mechanism is the request context from chapter sixteen.

```
type ctxKey string
const userKey ctxKey = "user"

func Auth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        u, err := userFromToken(r.Header.Get("Authorization"))
        if err != nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }
        ctx := context.WithValue(r.Context(), userKey, u)
        // a new request, with the value
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

func handler(w http.ResponseWriter, r *http.Request) {
    u, ok := r.Context().Value(userKey).(*User)
    if !ok {
        http.Error(w, "no user", http.StatusInternalServerError)
        return
    }
    ...
}
```

`r.WithContext` returns a **copy** of the request. Forgetting to pass the new one to `next.ServeHTTP` is a common bug: the value is set on a request nobody sees.

Graceful shutdown

Killing a server mid-request drops responses. Production code waits.

```
func main() {
    srv := &http.Server{Addr: ":8080", Handler: handler}

    go func() {
        if err := srv.ListenAndServe(); err != nil &&
            !errors.Is(err, http.ErrServerClosed) {
            log.Fatalf("listen: %v", err)
        }
    }()
    log.Println("listening on :8080")

    // Wait for an interrupt.
    stop := make(chan os.Signal, 1)
    signal.Notify(stop, os.Interrupt, syscall.SIGTERM)
    <-stop

    log.Println("shutting down")
```

```
    ctx, cancel := context.WithTimeout(
        context.Background(), 10*time.Second)
    defer cancel()

    if err := srv.Shutdown(ctx); err != nil {
        log.Printf("forced shutdown: %v", err)
    }
    log.Println("stopped")
}
```

`Shutdown` stops accepting new connections and waits for in-flight requests, up to the context's deadline. `ListenAndServe` returns `http.ErrServerClosed` when this happens, which is why that error is checked for and ignored.

This matters for every container deployment: Kubernetes sends `SIGTERM` and waits before `SIGKILL`, and a server without this drops every request in flight on each deploy.

Testing handlers without a server

`net/http/httptest` runs a handler in memory.

```
func TestGetUser(t *testing.T) {
    req := httptest.NewRequest("GET", "/users/42", nil)
    req.SetPathValue("id", "42")
    rec := httptest.NewRecorder()

    getUser(rec, req)

    if rec.Code != http.StatusOK {
        t.Fatalf("status = %d, want 200", rec.Code)
    }
    var got User
    if err := json.NewDecoder(rec.Body).Decode(&got); err != nil {
        t.Fatalf("decoding body: %v", err)
    }
    if got.ID != 42 {
        t.Errorf("ID = %d, want 42", got.ID)
    }
}
```

No port, no network, no cleanup, and it runs in microseconds. For testing a whole mux including middleware, `httptest.NewServer` starts a real one on a random port.

> **PROJECT 20: A COMPLETE HTTP SERVICE, NO DEPENDENCIES**
> Everything above in one runnable file: routing, middleware, JSON, timeouts, graceful shutdown.

```
package main

import (
    "context"
    "encoding/json"
    "errors"
    "log"
    "net/http"
    "os"
    "os/signal"
    "sync"
    "syscall"
    "time"
)

type Note struct {
    ID   int    `json:"id"`
    Text string `json:"text"`
}

type store struct {
    mu     sync.RWMutex
    notes  map[int]Note
    nextID int
}

func newStore() *store {
    return &store{notes: map[int]Note{}, nextID: 1}
}

func (s *store) list() []Note {
    s.mu.RLock()
    defer s.mu.RUnlock()
    out := make([]Note, 0, len(s.notes))
    for _, n := range s.notes {
        out = append(out, n)
    }
    return out
}

func (s *store) add(text string) Note {
    s.mu.Lock()
    defer s.mu.Unlock()
    n := Note{ID: s.nextID, Text: text}
    s.notes[n.ID] = n
    s.nextID++
    return n
}

// One helper for every JSON response, so the header is never forgotten.
func writeJSON(w http.ResponseWriter, status int, v any) {
```

```
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    if err := json.NewEncoder(w).Encode(v); err != nil {
        log.Printf("encoding response: %v", err)
    }
}

func logger(next http.Handler) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

func recoverer(next http.Handler) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                log.Printf("panic: %v", rec)
                writeJSON(w, http.StatusInternalServerError,
                    map[string]string{"error": "internal error"})
            }
        }()
        next.ServeHTTP(w, r)
    })
}

func main() {
    s := newStore()
    mux := http.NewServeMux()

    mux.HandleFunc("GET /notes",
        func(w http.ResponseWriter, r *http.Request) {
        writeJSON(w, http.StatusOK, s.list())
    })

    mux.HandleFunc("POST /notes",
        func(w http.ResponseWriter, r *http.Request) {
        var in struct {
            Text string `json:"text"`
        }
        dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20))
        dec.DisallowUnknownFields()
        if err := dec.Decode(&in); err != nil {
            writeJSON(w, http.StatusBadRequest,
                map[string]string{"error": "invalid json"})
            return
        }
```

```
        if in.Text == "" {
            writeJSON(w, http.StatusUnprocessableEntity,
                map[string]string{"text": "must not be empty"})
            return
        }
        writeJSON(w, http.StatusCreated, s.add(in.Text))
    })

    mux.HandleFunc("GET /panic",
        func(w http.ResponseWriter, r *http.Request) {
        panic("deliberate")      // proves the recoverer works
    })

    srv := &http.Server{
        Addr:              ":8080",
        Handler:           logger(recoverer(mux)),
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:       15 * time.Second,
        WriteTimeout:      15 * time.Second,
        IdleTimeout:       60 * time.Second,
    }

    go func() {
        if err := srv.ListenAndServe(); err != nil &&
            !errors.Is(err, http.ErrServerClosed) {
            log.Fatalf("listen: %v", err)
        }
    }()
    log.Println("listening on :8080")

    stop := make(chan os.Signal, 1)
    signal.Notify(stop, os.Interrupt, syscall.SIGTERM)
    <-stop

    ctx, cancel := context.WithTimeout(
        context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctx); err != nil {
        log.Printf("forced: %v", err)
    }
    log.Println("stopped cleanly")
}
```

Exercise it:

```
curl localhost:8080/notes
curl -X POST localhost:8080/notes -d '{"text":"buy milk"}'
# 400, unknown field
curl -X POST localhost:8080/notes -d '{"txt":"typo"}'
curl -X POST localhost:8080/notes -d '{"text":""}'        # 422
# 405, automatic
```

```
curl -X DELETE localhost:8080/notes
# 500, and the server survives
curl localhost:8080/panic
```

Then press Ctrl+C and watch it log `stopped cleanly` rather than dying.

Count the dependencies in `go.mod`: there are none. This is a concurrent, panic-resistant, gracefully shutting down JSON API with request logging and body size limits, in one file, with nothing installed. That is the baseline Go gives you, and it is why Go web frameworks are thin.

YOUR TURN

Add `GET /notes/{id}` using `r.PathValue("id")` and `strconv.Atoi`, returning 404 when absent. Then write the handler test with `httptest` and no running server. Finally, remove `ReadHeaderTimeout` and leave a connection open with `telnet localhost 8080` without sending anything, to see for yourself what the default costs you.

0

Talking to a database with database/sql

Go's `database/sql` is not an ORM. It is a thin, driver-agnostic layer over SQL: you write the queries, it manages connections and scans rows into your variables. Coming from Prisma or Drizzle this feels primitive for about a day and then feels like relief, because nothing is hidden.

Opening a database

```
import (
    "database/sql"
    // the driver, imported for side effects only
    _ "modernc.org/sqlite"
)

db, err := sql.Open("sqlite", "app.db")
if err != nil {
    return fmt.Errorf("opening database: %w", err)
}
defer db.Close()

if err := db.Ping(); err != nil {
    return fmt.Errorf("connecting: %w", err)
}
```

The blank import registers the driver with `database/sql`. You never call the driver package directly, which is why it is imported as `_`.

> **WATCH OUT**
> `sql.Open` **does not connect.** It validates arguments and prepares a pool, lazily. A wrong password or a missing server is not reported until the first query. Always `Ping` at startup, or your service starts happily and fails on the first user request.

The pool is the important part

`*sql.DB` is not a connection. It is a pool, safe for concurrent use, and you should create exactly one and pass it around.

```
db.SetMaxOpenConns(25)                  // hard ceiling on connections
db.SetMaxIdleConns(25)                  // keep them warm
db.SetConnMaxLifetime(5 * time.Minute)  // recycle, for load balancers
db.SetConnMaxIdleTime(5 * time.Minute)
```

WATCH OUT

The default `MaxOpenConns` is **unlimited**. Under load, Go will happily open more connections than your database allows, and Postgres will start refusing them. Set it. A common starting point is 25, and the right number is something like `(cores * 2) + effective spindles`, measured rather than guessed.

Set `MaxIdleConns` to the same value as `MaxOpenConns`. If idle is lower, connections are closed and reopened constantly under steady load, which is pure latency.

Querying one row

```
func GetUser(ctx context.Context, db *sql.DB, id int) (*User, error) {
    var u User
    err := db.QueryRowContext(ctx,
        `SELECT id, name, email FROM users WHERE id = ?`, id,
    ).Scan(&u.ID, &u.Name, &u.Email)

    if errors.Is(err, sql.ErrNoRows) {
        return nil, fmt.Errorf("user %d: %w", id, ErrNotFound)
    }
    if err != nil {
        return nil, fmt.Errorf("querying user %d: %w", id, err)
    }
    return &u, nil
}
```

`Scan` copies columns into the pointers you give it, in order. The number and order must match the `SELECT`.

`sql.ErrNoRows` is the sentinel for "no row", and it is a normal outcome rather than a failure. Translating it into your own `ErrNotFound`, as above, is chapter eleven's design in practice: callers should not have to import `database/sql` to understand your errors.

Querying many rows

```
func ListUsers(ctx context.Context, db *sql.DB) ([]User, error) {
    rows, err := db.QueryContext(ctx,
        `SELECT id, name, email FROM users ORDER BY id`)
    if err != nil {
        return nil, fmt.Errorf("listing users: %w", err)
```

```
    }
    defer rows.Close()

    var users []User
    for rows.Next() {
        var u User
        if err := rows.Scan(&u.ID, &u.Name, &u.Email); err != nil {
            return nil, fmt.Errorf("scanning user: %w", err)
        }
        users = append(users, u)
    }
    // The check everybody forgets: the loop ends the same way on error.
    if err := rows.Err(); err != nil {
        return nil, fmt.Errorf("iterating users: %w", err)
    }
    return users, nil
}
```

Three things are mandatory and two of them are easy to forget.

`defer rows.Close()`. An unclosed `Rows` holds its connection. Leak enough and the pool is exhausted and every query blocks forever.

Check `rows.Err()` after the loop. Exactly like `scanner.Err()` in chapter twenty, and for the same reason.

Check the `Scan` error inside the loop.

Writing, and what you get back

```
res, err := db.ExecContext(ctx,
    `INSERT INTO users (name, email) VALUES (?, ?)`, name, email)
if err != nil {
    return fmt.Errorf("creating user: %w", err)
}

id, err := res.LastInsertId()     // SQLite and MySQL
n, err := res.RowsAffected()
```

WATCH OUT

`LastInsertId` is not supported by Postgres. There you use `RETURNING` and treat the insert as a query:

```
var id int
err := db.QueryRowContext(ctx,
    `INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id`,
```

```
    name, email,
).Scan(&id)
```

Note also the placeholders: SQLite and MySQL use `?`, Postgres uses `$1`, `$2`. This is the main thing that is not portable between drivers, and it is why switching databases is never quite free.

Placeholders are not optional

```
// NEVER do this.
query := "SELECT * FROM users WHERE email = '" + email + "'"

// Always this.
db.QueryRowContext(ctx, "SELECT ... WHERE email = ?", email)
```

A placeholder sends the value separately from the statement, so it can never be parsed as SQL. String concatenation is SQL injection, and `'; DROP TABLE users;--` is only the famous example rather than the dangerous one: the real attacks read data rather than destroy it.

IF YOU KNOW JAVASCRIPT

Prisma and Drizzle protect you by generating the SQL. Here you write the SQL and the protection is the placeholder. The rule is simple: **values are placeholders, always**. Table and column names cannot be placeholders, so if those are dynamic, validate them against an allowlist you control. Grit does exactly this, which is why its generated services carry a `Sortable` map of permitted columns rather than interpolating whatever the client sent.

NULL, and why your struct fields lie

A `NULL` column cannot scan into a `string`. It fails at runtime:

```
sql: Scan error on column index 2: converting NULL to string is unsupported
```

Three options, in order of preference:

```
// 1. Best: make the column NOT NULL DEFAULT '' in the schema.

// 2. A pointer, when NULL is genuinely meaningful.
var bio *string
rows.Scan(&u.ID, &u.Name, &bio)

// 3. sql.NullString, when you want the distinction explicit.
```

```
var bio sql.NullString
rows.Scan(&u.ID, &u.Name, &bio)
if bio.Valid {
    u.Bio = bio.String
}
```

Since Go 1.22 there is also `sql.Null[T]`, the generic version:

```
var bio sql.Null[string]
```

Most NULL problems are schema problems. If a column has no meaningful "absent" state, make it `NOT NULL` and the whole question disappears.

Transactions

```
func Transfer(ctx context.Context, db *sql.DB,
    from, to int, amount int64) error {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return fmt.Errorf("beginning transaction: %w", err)
    }
    // Safe to call after Commit: it becomes a no-op. This is the guard
    // that
    // makes every early return below correct.
    defer tx.Rollback()

    _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance - ? WHERE id = ?`,
        amount, from)
    if err != nil {
        return fmt.Errorf("debiting %d: %w", from, err)
    }

    var balance int64
    err = tx.QueryRowContext(ctx,
        `SELECT balance FROM accounts WHERE id = ?`, from).Scan(&balance)
    if err != nil {
        return fmt.Errorf("checking balance: %w", err)
    }
    if balance < 0 {
        return ErrInsufficientFunds      // the deferred Rollback undoes it
    }

    _, err = tx.ExecContext(ctx,
        `UPDATE accounts SET balance = balance + ? WHERE id = ?`,
           amount, to)
    if err != nil {
        return fmt.Errorf("crediting %d: %w", to, err)
```

```
    }

    return tx.Commit()
}
```

`defer tx.Rollback()` right after `BeginTx` is the pattern. After a successful `Commit` the rollback does nothing, so one line covers every failure path including a panic.

WATCH OUT

Everything inside a transaction must use `tx`, not `db`. A query on `db` inside a transaction takes a **different connection** from the pool, so it cannot see uncommitted changes and does not participate in the rollback. This compiles, passes casual testing, and corrupts data under concurrency.

The usual defence is to pass something that could be either:

```
type Querier interface {
    QueryRowContext(context.Context, string, ...any) *sql.Row
    ExecContext(context.Context, string, ...any) (sql.Result, error)
}
```

Both `*sql.DB` and `*sql.Tx` satisfy it, so a function can be called inside or outside a transaction. This is chapter ten's consumer-declared interface, doing real work.

The N+1 query problem

The most common performance bug in any application, in any language:

```
posts, _ := ListPosts(ctx, db)              // 1 query
for i := range posts {
    posts[i].Author, _ = GetUser(ctx, db, posts[i].AuthorID)   // N queries
}
```

Fifty posts means fifty-one queries. Each is fast, so nothing looks slow, and the page takes two seconds.

The fix is one more query, not fifty:

```
ids := make([]int, 0, len(posts))
for _, p := range posts {
    ids = append(ids, p.AuthorID)
}

// Build the right number of placeholders.
ph := strings.Repeat("?,", len(ids))
ph = ph[:len(ph)-1]
args := make([]any, len(ids))
```

```
for i, id := range ids {
    args[i] = id
}

rows, err := db.QueryContext(ctx,
    `SELECT id, name FROM users WHERE id IN (`+ph+`)`, args...)
// ... build map[int]User, then attach to posts
```

Note that the **placeholders** are generated and the **values** are still passed as arguments. That is safe. Interpolating the IDs themselves would not be.

> **IF YOU KNOW JAVASCRIPT**
> Prisma hides this behind `include: { author: true }`, and under the hood it does the same two-query dance. Here you do it yourself, which is more typing and means the query count is visible in your code rather than in a log you have to go looking for. In part four you will see Grit's profiler report "201 queries, 0 slow" for exactly this pattern.

Should you use an ORM?

You will meet GORM in part four, because Grit uses it. The honest comparison:

`database/sql`	You write SQL. Nothing hidden, no surprises, more code
`sqlc`	You write SQL, it generates typed Go. Excellent, and a build step
GORM	Structs and methods. Fast to write, and you must know what it emits

The position worth holding: an ORM is a productivity tool, not a replacement for knowing SQL. Every team that has been burned by an ORM was burned because nobody could say what query it was running. Learn this chapter first, then use whatever your project uses, and keep the query log open.

> **PROJECT 21: A DATABASE-BACKED NOTE STORE**
> Wire a real SQLite database behind the HTTP service from chapter twenty one. Pure standard library plus one driver.

```
go get modernc.org/sqlite      # pure Go, no cgo, no compiler needed
```

```
package main

import (
    "context"
    "database/sql"
    "errors"
```

```
    "fmt"
    "time"

    _ "modernc.org/sqlite"
)

var ErrNotFound = errors.New("not found")

type Note struct {
    ID        int       `json:"id"`
    Text      string    `json:"text"`
    CreatedAt time.Time `json:"created_at"`
}

type Store struct{ db *sql.DB }

func Open(path string) (*Store, error) {
    db, err := sql.Open("sqlite", path)
    if err != nil {
        return nil, fmt.Errorf("opening %s: %w", path, err)
    }
    db.SetMaxOpenConns(1)     // SQLite: one writer. For Postgres, 25.

    if err := db.Ping(); err != nil {
        return nil, fmt.Errorf("connecting: %w", err)
    }

    // Schema on startup. Real projects use migration files; chapter 31
    // covers how Grit does it.
    _, err = db.Exec(`
        CREATE TABLE IF NOT EXISTS notes (
            id         INTEGER PRIMARY KEY AUTOINCREMENT,
            text       TEXT    NOT NULL,
            created_at TEXT    NOT NULL
        )`)
    if err != nil {
        return nil, fmt.Errorf("creating schema: %w", err)
    }
    return &Store{db: db}, nil
}

func (s *Store) Close() error { return s.db.Close() }

func (s *Store) Create(ctx context.Context, text string) (*Note, error) {
    now := time.Now().UTC()
    res, err := s.db.ExecContext(ctx,
        `INSERT INTO notes (text, created_at) VALUES (?, ?)`,
        text, now.Format(time.RFC3339))
    if err != nil {
        return nil, fmt.Errorf("creating note: %w", err)
    }
```

```
    id, err := res.LastInsertId()
    if err != nil {
        return nil, fmt.Errorf("reading new id: %w", err)
    }
    return &Note{ID: int(id), Text: text, CreatedAt: now}, nil
}

func (s *Store) Get(ctx context.Context, id int) (*Note, error) {
    var n Note
    var created string
    err := s.db.QueryRowContext(ctx,
        `SELECT id, text, created_at FROM notes WHERE id = ?`, id,
    ).Scan(&n.ID, &n.Text, &created)

    if errors.Is(err, sql.ErrNoRows) {
        return nil, fmt.Errorf("note %d: %w", id, ErrNotFound)
    }
    if err != nil {
        return nil, fmt.Errorf("getting note %d: %w", id, err)
    }
    n.CreatedAt, _ = time.Parse(time.RFC3339, created)
    return &n, nil
}

func (s *Store) List(ctx context.Context, limit int) ([]Note, error) {
    rows, err := s.db.QueryContext(ctx,
        `SELECT id, text, created_at FROM notes
         ORDER BY id DESC LIMIT ?`, limit)
    if err != nil {
        return nil, fmt.Errorf("listing notes: %w", err)
    }
    defer rows.Close()

    var out []Note
    for rows.Next() {
        var n Note
        var created string
        if err := rows.Scan(&n.ID, &n.Text, &created); err != nil {
            return nil, fmt.Errorf("scanning note: %w", err)
        }
        n.CreatedAt, _ = time.Parse(time.RFC3339, created)
        out = append(out, n)
    }
    if err := rows.Err(); err != nil {
        return nil, fmt.Errorf("iterating notes: %w", err)
    }
    return out, nil
}

func (s *Store) Delete(ctx context.Context, id int) error {
    res, err := s.db.ExecContext(ctx,
```

```
        `DELETE FROM notes WHERE id = ?`, id)
    if err != nil {
        return fmt.Errorf("deleting note %d: %w", id, err)
    }
    n, err := res.RowsAffected()
    if err != nil {
        return fmt.Errorf("checking delete: %w", err)
    }
    if n == 0 {
        return fmt.Errorf("note %d: %w", id, ErrNotFound)
    }
    return nil
}
```

Now swap this in behind the handlers from chapter twenty one and you have a persistent JSON API with two dependencies: the SQLite driver and nothing else.

Two details worth dwelling on. `RowsAffected() == 0` is how `Delete` knows the row was missing, because a `DELETE` that matches nothing is not an error in SQL. And `SetMaxOpenConns(1)` is correct for SQLite, which allows one writer: leaving it unlimited gives you `database is locked` errors under concurrency, which is the single most common SQLite complaint and is a configuration problem rather than a SQLite problem.

YOUR TURN

Write a test using `t.TempDir()` for the database file, covering create, get, list and a `Get` that returns `ErrNotFound`. Then add an `Update` and give it a transaction that refuses to blank out the text, returning a validation error, and prove with a test that the row is unchanged after the failure. That last test is the one that proves you understand `defer tx.Rollback()`.

0

Project: a URL shortener

Everything from Parts I to III in one finished program: an HTTP server, a database, concurrency, context, error design, tests. Around three hundred lines, no framework, and it is genuinely deployable.

This chapter is one long project. Type it as you go.

What it does

```
POST /shorten  {"url": "https://example.com/very/long"}
               ->  {"code":"aB3dQ"}
GET  /aB3dQ                                ->  302 redirect
GET  /api/stats/aB3dQ                      ->  {"hits": 17}
```

Plus: it refuses nonsense URLs, counts hits without slowing the redirect, and shuts down cleanly.

The schema and the store

```go
// store.go
package main

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "time"

    _ "modernc.org/sqlite"
)

var (
    ErrNotFound = errors.New("short code not found")
    ErrInvalid  = errors.New("invalid url")
)

type Link struct {
    Code      string    `json:"code"`
```

```
    URL       string    `json:"url"`
    Hits      int64     `json:"hits"`
    CreatedAt time.Time `json:"created_at"`
}

type Store struct{ db *sql.DB }

func OpenStore(path string) (*Store, error) {
    db, err := sql.Open("sqlite", path)
    if err != nil {
        return nil, fmt.Errorf("opening %s: %w", path, err)
    }
    db.SetMaxOpenConns(1)
    if err := db.Ping(); err != nil {
        return nil, fmt.Errorf("connecting: %w", err)
    }
    if _, err := db.Exec(`
        CREATE TABLE IF NOT EXISTS links (
            code       TEXT PRIMARY KEY,
            url        TEXT    NOT NULL,
            hits       INTEGER NOT NULL DEFAULT 0,
            created_at TEXT    NOT NULL
        )`); err != nil {
        return nil, fmt.Errorf("creating schema: %w", err)
    }
    return &Store{db: db}, nil
}

func (s *Store) Close() error { return s.db.Close() }

func (s *Store) Save(ctx context.Context,
    code, url string) (*Link, error) {
    now := time.Now().UTC()
    _, err := s.db.ExecContext(ctx,
        `INSERT INTO links (code, url, created_at) VALUES (?, ?, ?)`,
        code, url, now.Format(time.RFC3339))
    if err != nil {
        return nil, fmt.Errorf("saving %s: %w", code, err)
    }
    return &Link{Code: code, URL: url, CreatedAt: now}, nil
}

func (s *Store) Lookup(ctx context.Context, code string) (*Link, error) {
    var l Link
    var created string
    err := s.db.QueryRowContext(ctx,
        `SELECT code, url, hits, created_at FROM links
         WHERE code = ?`, code,
    ).Scan(&l.Code, &l.URL, &l.Hits, &created)
```

```
    if errors.Is(err, sql.ErrNoRows) {
        return nil, fmt.Errorf("%q: %w", code, ErrNotFound)
    }
    if err != nil {
        return nil, fmt.Errorf("looking up %q: %w", code, err)
    }
    l.CreatedAt, _ = time.Parse(time.RFC3339, created)
    return &l, nil
}

// RecordHit is deliberately separate from Lookup so the redirect does not
// wait for a write.
func (s *Store) RecordHit(ctx context.Context, code string) error {
    _, err := s.db.ExecContext(ctx,
        `UPDATE links SET hits = hits + 1 WHERE code = ?`, code)
    if err != nil {
        return fmt.Errorf("recording hit for %q: %w", code, err)
    }
    return nil
}
```

Generating codes

```
// code.go
package main

import (
    "crypto/rand"
    "math/big"
)

// Base58: no 0, O, I or l, so a code read aloud or copied by hand
// survives.
const alphabet = "123456789ABCDEFGHJKLMNPQRSTUVWXYZ" +
    "abcdefghijkmnopqrstuvwxyz"

func newCode(n int) (string, error) {
    b := make([]byte, n)
    max := big.NewInt(int64(len(alphabet)))
    for i := range b {
        k, err := rand.Int(rand.Reader, max)
        if err != nil {
            return "", err
        }
        b[i] = alphabet[k.Int64()]
    }
```

```
    return string(b), nil
}
```

WATCH OUT

`crypto/rand`, not `math/rand`. A short code generated from a predictable sequence can be guessed, which for a link shortener means enumerating everybody's private links. Use `crypto/rand` for anything a user should not be able to predict: codes, tokens, session IDs, password reset links.

Validating the URL

```
// validate.go
package main

import (
    "fmt"
    "net/url"
    "strings"
)

func validateURL(raw string) (string, error) {
    raw = strings.TrimSpace(raw)
    if raw == "" {
        return "", fmt.Errorf("url is empty: %w", ErrInvalid)
    }
    if len(raw) > 2048 {
        return "", fmt.Errorf("url too long: %w", ErrInvalid)
    }

    u, err := url.Parse(raw)
    if err != nil {
        return "", fmt.Errorf("parsing url: %w", ErrInvalid)
    }
    // Scheme must be http or https: javascript: and data: are XSS vectors
    // the moment this code appears in an href.
    if u.Scheme != "http" && u.Scheme != "https" {
        return "", fmt.Errorf("scheme %q not allowed: %w",
            u.Scheme, ErrInvalid)
    }
    if u.Host == "" {
        return "", fmt.Errorf("url has no host: %w", ErrInvalid)
    }
    return u.String(), nil
}
```

> **WATCH OUT**
> The scheme check is not optional. Without it, somebody shortens `javascript:alert(document.cookie)` and your redirect endpoint serves it. Any time user input ends up in an `href` or a `Location` header, allowlist the scheme.

The server

```go
// main.go
package main

import (
    "context"
    "encoding/json"
    "errors"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

type Server struct {
    store *Store
    log   *log.Logger
}

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(v)
}

func writeErr(w http.ResponseWriter, status int, msg string) {
    writeJSON(w, status, map[string]string{"error": msg})
}

func (s *Server) handleShorten(w http.ResponseWriter, r *http.Request) {
    var in struct {
        URL string `json:"url"`
    }
    dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 8<<10))
    dec.DisallowUnknownFields()
    if err := dec.Decode(&in); err != nil {
        writeErr(w, http.StatusBadRequest, "invalid json body")
        return
    }
```

```
    clean, err := validateURL(in.URL)
    if err != nil {
        writeErr(w, http.StatusUnprocessableEntity, err.Error())
        return
    }

    // Retry on the astronomically unlikely collision rather than failing.
    for attempt := 0; attempt < 5; attempt++ {
        code, err := newCode(6)
        if err != nil {
            s.log.Printf("generating code: %v", err)
            writeErr(w, http.StatusInternalServerError, "internal error")
            return
        }
        link, err := s.store.Save(r.Context(), code, clean)
        if err == nil {
            writeJSON(w, http.StatusCreated, link)
            return
        }
        s.log.Printf("save attempt %d: %v", attempt+1, err)
    }
    writeErr(w, http.StatusInternalServerError,
        "could not allocate a code")
}

func (s *Server) handleRedirect(w http.ResponseWriter, r *http.Request) {
    code := r.PathValue("code")

    link, err := s.store.Lookup(r.Context(), code)
    if errors.Is(err, ErrNotFound) {
        writeErr(w, http.StatusNotFound, "no such link")
        return
    }
    if err != nil {
        s.log.Printf("lookup %q: %v", code, err)
        writeErr(w, http.StatusInternalServerError, "internal error")
        return
    }

    // Count the hit without making the user wait for the write. The
    // request
    // context is about to be cancelled, so the goroutine gets its own.
    go func() {
        ctx, cancel := context.WithTimeout(
            context.Background(), 5*time.Second)
        defer cancel()
        if err := s.store.RecordHit(ctx, code); err != nil {
            s.log.Printf("recording hit: %v", err)
```

```
        }
    }()

    http.Redirect(w, r, link.URL, http.StatusFound)
}

func (s *Server) handleStats(w http.ResponseWriter, r *http.Request) {
    link, err := s.store.Lookup(r.Context(), r.PathValue("code"))
    if errors.Is(err, ErrNotFound) {
        writeErr(w, http.StatusNotFound, "no such link")
        return
    }
    if err != nil {
        writeErr(w, http.StatusInternalServerError, "internal error")
        return
    }
    writeJSON(w, http.StatusOK, link)
}

func (s *Server) routes() http.Handler {
    mux := http.NewServeMux()
    mux.HandleFunc("POST /shorten", s.handleShorten)
    mux.HandleFunc("GET /api/stats/{code}", s.handleStats)
    mux.HandleFunc("GET /{code}", s.handleRedirect)
    mux.HandleFunc("GET /healthz",
        func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    return s.recoverer(s.logger(mux))
}

func (s *Server) logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        s.log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

func (s *Server) recoverer(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                s.log.Printf("panic on %s %s: %v",
                    r.Method, r.URL.Path, rec)
                writeErr(w, http.StatusInternalServerError,
                    "internal error")
            }
        }()
```

```
        next.ServeHTTP(w, r)
    })
}

func main() {
    addr := os.Getenv("ADDR")
    if addr == "" {
        addr = ":8080"
    }
    dbPath := os.Getenv("DB_PATH")
    if dbPath == "" {
        dbPath = "links.db"
    }

    logger := log.New(os.Stdout, "", log.LstdFlags)

    store, err := OpenStore(dbPath)
    if err != nil {
        logger.Fatalf("opening store: %v", err)
    }
    defer store.Close()

    srv := &http.Server{
        Addr:              addr,
        Handler:           (&Server{store: store, log: logger}).routes(),
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:       15 * time.Second,
        WriteTimeout:      15 * time.Second,
        IdleTimeout:       60 * time.Second,
    }

    go func() {
        if err := srv.ListenAndServe(); err != nil &&
            !errors.Is(err, http.ErrServerClosed) {
            logger.Fatalf("listen: %v", err)
        }
    }()
    logger.Printf("listening on %s, database %s", addr, dbPath)

    stop := make(chan os.Signal, 1)
    signal.Notify(stop, os.Interrupt, syscall.SIGTERM)
    <-stop

    logger.Println("shutting down")
    ctx, cancel := context.WithTimeout(
        context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctx); err != nil {
        logger.Printf("forced shutdown: %v", err)
```

```
    }
    logger.Println("stopped")
}
```

Running it

```
go mod init github.com/you/shortener
go get modernc.org/sqlite
go run .
```

```
curl -X POST localhost:8080/shorten -d '{"url":"https://go.dev/doc/"}'
# {"code":"7fKpQ2","url":"https://go.dev/doc/","hits":0,...}

curl -i localhost:8080/7fKpQ2          # 302, Location: https://go.dev/doc/
curl localhost:8080/api/stats/7fKpQ2   # {"hits":1,...}

curl -X POST localhost:8080/shorten -d '{"url":"javascript:alert(1)"}'
# 422, scheme "javascript" not allowed
```

The tests

```
// server_test.go
package main

import (
    "context"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "path/filepath"
    "strings"
    "testing"
)

func newTestServer(t *testing.T) *Server {
    t.Helper()
    store, err := OpenStore(filepath.Join(t.TempDir(), "test.db"))
    if err != nil {
        t.Fatalf("opening store: %v", err)
    }
    t.Cleanup(func() { store.Close() })
    return &Server{store: store, log: testLogger(t)}
}

func TestShortenAndRedirect(t *testing.T) {
    s := newTestServer(t)
```

```
    h := s.routes()

    req := httptest.NewRequest("POST", "/shorten",
        strings.NewReader(`{"url":"https://example.com/x"}`))
    rec := httptest.NewRecorder()
    h.ServeHTTP(rec, req)

    if rec.Code != http.StatusCreated {
        t.Fatalf("status = %d, want 201: %s", rec.Code, rec.Body)
    }
    var link Link
    if err := json.NewDecoder(rec.Body).Decode(&link); err != nil {
        t.Fatalf("decoding: %v", err)
    }
    if len(link.Code) != 6 {
        t.Errorf("code = %q, want 6 characters", link.Code)
    }

    req = httptest.NewRequest("GET", "/"+link.Code, nil)
    rec = httptest.NewRecorder()
    h.ServeHTTP(rec, req)

    if rec.Code != http.StatusFound {
        t.Fatalf("redirect status = %d, want 302", rec.Code)
    }
    got := rec.Header().Get("Location")
    if got != "https://example.com/x" {
        t.Errorf("Location = %q", got)
    }
}

func TestRejectsBadSchemes(t *testing.T) {
    s := newTestServer(t)
    h := s.routes()

    for _, bad := range []string{
        `{"url":"javascript:alert(1)"}`,
        `{"url":"data:text/html,<script>"}`,
        `{"url":""}`,
        `{"url":"not a url at all"}`,
    } {
        req := httptest.NewRequest("POST", "/shorten",
          strings.NewReader(bad))
        rec := httptest.NewRecorder()
        h.ServeHTTP(rec, req)
        if rec.Code != http.StatusUnprocessableEntity {
            t.Errorf("%s: status = %d, want 422", bad, rec.Code)
        }
    }
```

```
}

func TestUnknownCodeIs404(t *testing.T) {
    s := newTestServer(t)
    rec := httptest.NewRecorder()
    s.routes().ServeHTTP(rec, httptest.NewRequest("GET", "/nope12", nil))
    if rec.Code != http.StatusNotFound {
        t.Errorf("status = %d, want 404", rec.Code)
    }
}

func TestLookupMissing(t *testing.T) {
    s := newTestServer(t)
    _, err := s.store.Lookup(context.Background(), "missing")
    if err == nil {
        t.Fatal("want an error for a missing code")
    }
}
```

```
go test -race -cover ./...
```

WHAT THIS PROGRAM DEMONSTRATES

Walk back through it and name the chapter each decision came from.

Error design (11). `ErrNotFound` and `ErrInvalid` are sentinels, wrapped with `%w`, and the handlers branch on `errors.Is`. No handler reads an error message.

Pointers and nil (8). `Lookup` returns `*Link`, nil on failure, so a caller cannot accidentally use a half-built value.

Goroutines and context (13, 16). The hit counter runs in its own goroutine with a **fresh** context, because `r.Context()` is cancelled the moment the redirect is written. Using the request context there would silently lose most of your statistics under load, and it is the kind of bug that never shows up in testing.

Interfaces (10). `writeJSON` takes `http.ResponseWriter`, and the tests pass an `httptest.ResponseRecorder` instead. Nothing was designed for testability; it is testable because the standard library is built on small interfaces.

The database chapter (22). `defer rows.Close()`, `rows.Err()`, placeholders, `sql.ErrNoRows` translated at the boundary.

Security. `crypto/rand` for unguessable codes, a scheme allowlist against XSS, `MaxBytesReader` against oversized bodies, `DisallowUnknownFields` against typos, and timeouts on the server.

Count the dependencies: one, the SQLite driver.

YOUR TURN

Three extensions, in order of difficulty. Add an expiry column and a `GET /{code}` that returns 410 Gone for expired links. Add rate limiting to `POST /shorten` using a map of IP to a token bucket, protected by a mutex, which is chapter fifteen's material. Then add a custom alias (`{"url":..., "alias":"docs"}`), which forces you to handle a real `UNIQUE` constraint violation rather than the random collision retry, and that is a different error path entirely.

0

Configuration, environment and secrets

A program that works on your laptop and nowhere else usually has its configuration hard-coded. This short chapter is how to avoid that, and how not to leak a database password into your git history on the way.

Environment variables, the baseline

```
addr := os.Getenv("ADDR")
if addr == "" {
    addr = ":8080"       // a default, so the program runs with no setup
}
```

`os.Getenv` returns `""` for a variable that is unset, so you cannot tell unset from set-to-empty. When that matters:

```
addr, ok := os.LookupEnv("ADDR")
if !ok {
    // genuinely not set
}
```

> **IF YOU KNOW JAVASCRIPT**
> This is `process.env.ADDR`, with the same string-only limitation. There is no `dotenv` in the standard library, so a `.env` file needs a package, usually `joho/godotenv`. Many Go teams skip it entirely and export variables from a shell script or a compose file, because production almost never uses a `.env` file anyway.

Parse once, into a struct

Scattered `os.Getenv` calls are the thing to avoid. Read everything at startup, validate it, and pass the result down.

```
type Config struct {
    Addr        string
    DatabaseURL string
    LogLevel    string
    Debug       bool
```

```
    Timeout     time.Duration
    MaxWorkers  int
}

func Load() (*Config, error) {
    c := &Config{
        Addr:       getEnv("ADDR", ":8080"),
        LogLevel:   getEnv("LOG_LEVEL", "info"),
        Debug:      getEnvBool("DEBUG", false),
        Timeout:    getEnvDuration("TIMEOUT", 30*time.Second),
        MaxWorkers: getEnvInt("MAX_WORKERS", 4),
    }

    // Required: no sensible default exists.
    c.DatabaseURL = os.Getenv("DATABASE_URL")
    if c.DatabaseURL == "" {
        return nil, errors.New("DATABASE_URL is required")
    }
    if c.MaxWorkers < 1 {
        return nil, fmt.Errorf("MAX_WORKERS must be at least 1, got %d",
            c.MaxWorkers)
    }
    return c, nil
}

func getEnv(key, def string) string {
    if v, ok := os.LookupEnv(key); ok {
        return v
    }
    return def
}

func getEnvInt(key string, def int) int {
    if v, ok := os.LookupEnv(key); ok {
        if n, err := strconv.Atoi(v); err == nil {
            return n
        }
    }
    return def
}

func getEnvBool(key string, def bool) bool {
    if v, ok := os.LookupEnv(key); ok {
        if b, err := strconv.ParseBool(v); err == nil {
            return b
        }
    }
    return def
}
```

```
func getEnvDuration(key string, def time.Duration) time.Duration {
    if v, ok := os.LookupEnv(key); ok {
        if d, err := time.ParseDuration(v); err == nil {
            return d
        }
    }
    return def
}
```

`time.ParseDuration` accepts `"30s"`, `"5m"`, `"1h30m"`, which is far better than an integer whose unit lives in a comment.

WATCH OUT

Those helpers silently fall back to the default when parsing fails, so `MAX_WORKERS=eight` quietly becomes 4. For anything that matters, fail loudly instead:

```
func mustInt(key string, def int) (int, error) {
    v, ok := os.LookupEnv(key)
    if !ok {
        return def, nil
    }
    n, err := strconv.Atoi(v)
    if err != nil {
        return 0, fmt.Errorf("%s=%q: not a number", key, v)
    }
    return n, nil
}
```

A typo in a deployment variable should stop the program at startup, not change its behaviour silently three weeks later.

Fail at startup, not on the first request

```
func main() {
    cfg, err := Load()
    if err != nil {
        log.Fatalf("configuration: %v", err)
    }
    // From here, cfg is valid. Nothing downstream re-checks it.
}
```

This is the single most valuable habit in this chapter. A missing variable should crash the container immediately, so the deploy fails visibly, rather than letting a half-configured process serve errors.

Secrets

Three rules, and the first one is the one people break.

Never commit a secret. Not in a `.env` file, not in a test fixture, not "temporarily". Git keeps it forever, and a repository that goes public later publishes your history too.

```
# .gitignore
.env
.env.local
*.pem
```

Commit a `.env.example` with the **keys** and no values, so a new developer knows what to set.

Never log one. This is easy to do by accident:

```
log.Printf("config: %+v", cfg)    // prints DatabaseURL, password and all
```

The defence is a `String` method that redacts:

```
func (c Config) String() string {
    return fmt.Sprintf("Config{Addr:%s LogLevel:%s DatabaseURL:%s}",
        c.Addr, c.LogLevel, redact(c.DatabaseURL))
}

func redact(s string) string {
    if s == "" {
        return ""
    }
    return "[redacted]"
}
```

Because `fmt` looks for `String()`, this covers `%v`, `%s` and `Println`. It does not cover `%+v` on a struct that embeds yours, so the surer answer is a dedicated type:

```
type Secret string

func (Secret) String() string          { return "[redacted]" }
func (Secret) GoString() string        { return "[redacted]" }
func (s Secret) Reveal() string        { return string(s) }
```

Now the only way to see the value is to call `Reveal()`, which is greppable in review.

Never put one in a URL you log. `postgres://user:pass@host/db` is a secret in its entirety, which is why `redact` above hides the whole string rather than trying to mask the password portion.

IF YOU KNOW JAVASCRIPT
The mechanics are the same as Node: environment variables, a gitignored `.env`, a secret manager in production. The Go-specific part is that a compiled binary feels like a sealed box and is not: strings compiled into it are readable with the `strings` command. Never embed a key in source, even in a binary you only ship internally.

Flags, for tools rather than services

For a CLI, `flag` is in the standard library:

```
func main() {
    var (
        verbose = flag.Bool("v", false, "verbose output")
        out     = flag.String("o", "", "output file (default stdout)")
        limit   = flag.Int("n", 10, "how many results")
    )
    flag.Parse()

    for _, path := range flag.Args() {   // the non-flag arguments
        ...
    }
}
```

`flag` gives you `-h` for free, printing every flag with its default. It does not support subcommands or long-and-short pairs; for those, `spf13/cobra` is the near-universal choice and is what Grit's CLI is built on.

The usual division: **flags for tools, environment for services**. A service configured by flags is awkward in a container, and a CLI configured by environment variables is awkward to use.

PROJECT 22: CONFIGURE THE SHORTENER PROPERLY
Take the URL shortener from the last chapter and give it a real configuration layer, with validation that fails at startup.

```
// config.go
package main

import (
    "errors"
    "fmt"
    "os"
    "strconv"
    "time"
)
```

```
type Config struct {
    Addr         string
    DBPath       string
    CodeLength   int
    ReadTimeout  time.Duration
    WriteTimeout time.Duration
    BaseURL      string
}

func LoadConfig() (*Config, error) {
    c := &Config{
        Addr:    envOr("ADDR", ":8080"),
        DBPath:  envOr("DB_PATH", "links.db"),
        BaseURL: envOr("BASE_URL", "http://localhost:8080"),
    }

    var err error
    if c.CodeLength, err = envInt("CODE_LENGTH", 6); err != nil {
        return nil, err
    }
    c.ReadTimeout, err = envDur("READ_TIMEOUT", 15*time.Second)
    if err != nil {
        return nil, err
    }
    c.WriteTimeout, err = envDur("WRITE_TIMEOUT", 15*time.Second)
    if err != nil {
        return nil, err
    }

    // Validation. Every one of these is a mistake worth catching before
    // the first request rather than after.
    if c.CodeLength < 4 || c.CodeLength > 32 {
        return nil, fmt.Errorf("CODE_LENGTH must be 4 to 32, got %d",
            c.CodeLength)
    }
    if c.DBPath == "" {
        return nil, errors.New("DB_PATH must not be empty")
    }
    if c.ReadTimeout <= 0 || c.WriteTimeout <= 0 {
        return nil, errors.New("timeouts must be positive")
    }
    return c, nil
}

// String keeps the config loggable without leaking. Nothing here
// is secret today, and the method exists so that it stays safe when
// somebody adds an API key next month.
func (c Config) String() string {
    return fmt.Sprintf(
        "Config{Addr:%s DBPath:%s CodeLength:%d BaseURL:%s}",
```

```
        c.Addr, c.DBPath, c.CodeLength, c.BaseURL)
}

func envOr(key, def string) string {
    if v, ok := os.LookupEnv(key); ok && v != "" {
        return v
    }
    return def
}

func envInt(key string, def int) (int, error) {
    v, ok := os.LookupEnv(key)
    if !ok || v == "" {
        return def, nil
    }
    n, err := strconv.Atoi(v)
    if err != nil {
        return 0, fmt.Errorf("%s=%q is not a number", key, v)
    }
    return n, nil
}

func envDur(key string, def time.Duration) (time.Duration, error) {
    v, ok := os.LookupEnv(key)
    if !ok || v == "" {
        return def, nil
    }
    d, err := time.ParseDuration(v)
    if err != nil {
        return 0, fmt.Errorf("%s=%q is not a duration, try 15s", key, v)
    }
    return d, nil
}
```

And in `main`:

```
cfg, err := LoadConfig()
if err != nil {
    fmt.Fprintln(os.Stderr, "configuration:", err)
    os.Exit(1)
}
logger.Printf("starting with %v", cfg)
```

Try it:

```
go run .                                # defaults
ADDR=:9000 CODE_LENGTH=8 go run .       # overrides
CODE_LENGTH=2 go run .                  # exits 1 with a clear message
CODE_LENGTH=eight go run .              # exits 1, names the bad value
```

The last two are the point. A deployment with a typo'd variable now fails immediately and says which variable and what it received, instead of starting up and generating two-character codes that collide constantly.

Write the test too, since config is easy to test and almost nobody does:

```
func TestLoadConfigRejectsShortCodes(t *testing.T) {
    t.Setenv("CODE_LENGTH", "2")      // restored automatically
    if _, err := LoadConfig(); err == nil {
        t.Fatal("want an error for CODE_LENGTH=2")
    }
}
```

`t.Setenv` sets a variable for one test and restores it afterwards, and it refuses to run in a parallel test, because environment variables are process global and that would be a race.

YOUR TURN

Add an `API_KEY` using a `Secret` type with a redacting `String()`, and prove with a test that `fmt.Sprintf("%v", cfg)` does not contain the value. Then add `-config` as a flag pointing at a JSON file, with environment variables taking precedence over the file and the file over the defaults. Getting that precedence order right, and writing a test for each layer, is most of what configuration libraries do for you.

0

Building, cross compiling and shipping one binary

This is the chapter that explains why people who deploy Node sound envious when Go comes up. Deployment is `scp` and `./app`, and the rest of this chapter is the detail around that sentence.

The build

```
go build -o app .
./app
```

One file. It contains your code, every dependency, and the Go runtime. The target machine needs nothing: no Go, no package manager, no shared libraries.

> **IF YOU KNOW JAVASCRIPT**
> Deploying a Next.js app means Node of the right major version, a `node_modules` tree, a build step, and a process manager. Deploying this means copying one file. That is the whole difference, and it is the reason Go dominates CLI tools and containers.

Smaller binaries

The default build includes debug symbols and a symbol table.

```
go build -ldflags="-s -w" -o app .
```

`-s` drops the symbol table, `-w` the DWARF debug information. Expect roughly 25 to 30 percent off. The cost is that a stack trace from a panic loses some detail and the binary cannot be inspected with a debugger, so many teams ship unstripped and accept the size.

For genuinely small, `upx` compresses further, at the cost of slower startup and antivirus software treating your binary with suspicion. Rarely worth it.

Embedding the version

A binary that cannot tell you which commit it is costs real time during an incident.

```
package main

var (
    version = "dev"
    commit  = "none"
    date    = "unknown"
)

func main() {
    if len(os.Args) > 1 && os.Args[1] == "version" {
        fmt.Printf("%s (%s) built %s\n", version, commit, date)
        return
    }
    ...
}
```

```
go build -ldflags="\
  -X main.version=1.4.2 \
  -X main.commit=$(git rev-parse --short HEAD) \
  -X main.date=$(date -u +%Y-%m-%dT%H:%M:%SZ)" -o app .
```

`-X` sets a string variable at link time. It only works on `string` variables in the `main` package or a named package path, and only ones that are not constants.

> **IF YOU KNOW JAVASCRIPT**
> Grit does exactly this, which is why `grit version` reports a real version rather than "dev". Its release process bumps three places that must agree: the Go variable, the scaffolder's default, and the documentation site's constant. Chapter 47 covers why a version that disagrees with itself is worse than no version.

Cross compiling

Two environment variables, no toolchain to install.

```
GOOS=linux   GOARCH=amd64 go build -o app-linux-amd64 .
GOOS=linux   GOARCH=arm64 go build -o app-linux-arm64 .
GOOS=darwin  GOARCH=arm64 go build -o app-darwin-arm64 .
GOOS=windows GOARCH=amd64 go build -o app-windows-amd64.exe .
```

From any machine, to any target. This is not an exaggeration: a Windows laptop builds the Linux ARM binary that runs on your Raspberry Pi, with no container and no configuration.

```
go tool dist list      # every supported pair
```

> **WATCH OUT**
> Cross compilation is free **until you use cgo**. Any package binding to a C library (the classic `mattn/go-sqlite3`, some image codecs, some crypto) needs a C cross-compiler for the target, and the simplicity evaporates.
>
> The defence is to prefer pure Go packages. This is why the database chapter used `modernc.org/sqlite` rather than `mattn/go-sqlite3`: the former is Go all the way down, so `GOOS=linux go build` works from anywhere. Check for cgo with:
>
> ```
> CGO_ENABLED=0 go build ./...
> ```
>
> If that fails, something in your tree needs C.

Containers, in two stages

A multi-stage Dockerfile builds in one image and ships in a tiny one.

```
# Build
FROM golang:1.25-alpine AS build
WORKDIR /src

# Copy manifests first: this layer caches unless dependencies change.
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /bin/app .

# Ship
FROM gcr.io/distroless/static-debian12
COPY --from=build /bin/app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

The final image contains your binary, CA certificates and nothing else: no shell, no package manager, no libc. Typical size is your binary plus about two megabytes.

`CGO_ENABLED=0` is required for the `static` base, because there is no libc in it to dynamically link against.

> **WATCH OUT**
> Copy `go.mod` and `go.sum` and run `go mod download` **before** copying your source. Docker caches layers, and if the source is copied first then every code change re-downloads

every dependency. This one ordering decision is the difference between a ten second rebuild and a two minute one.

Embedding files into the binary

`go:embed` puts files inside the executable, so your single-file deployment can still serve a frontend.

```
import "embed"

//go:embed static/*
var staticFiles embed.FS

//go:embed schema.sql
var schema string

func main() {
    http.Handle("/static/", http.FileServerFS(staticFiles))
    db.Exec(schema)
}
```

The comment is the directive and must sit immediately above the variable with no blank line. `embed.FS` for a tree, `string` or `[]byte` for one file.

This is how a Go program ships a built React app inside itself: compile the frontend, embed `dist/`, and deploy one file that serves both the API and the UI. Grit's single-binary architecture mode does precisely this.

Production flags worth knowing

```
go build -trimpath -ldflags="-s -w" -o app .
```

`-trimpath` removes absolute file paths from the binary, so a panic trace says `main.go:42` rather than `/home/you/projects/secret-name/main.go:42`. It also makes builds reproducible, which matters if you sign releases.

```
GOAMD64=v3 go build -o app .
```

Targets a newer x86 instruction set for a few percent. Only do this if you control the hardware, because the binary will not run on older CPUs.

Automating releases

A tag-triggered workflow that cross compiles and attaches binaries:

```yaml
name: Release
on:
  push:
    tags: ['v*']

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-go@v5
        with: { go-version: '1.25' }
      - run: go test -race ./...
      - uses: goreleaser/goreleaser-action@v6
        with: { args: release --clean }
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

`goreleaser` reads a config file, builds every platform, writes checksums, generates release notes from commits and uploads everything. It is the standard tool, and it is what turns "I tagged v1.2.0" into binaries for six platforms without further thought.

PROJECT 23: SHIP THE SHORTENER

Take the URL shortener and make it genuinely deployable: versioned, containerised and cross compiled.

First, the version:

```go
// version.go
package main

import (
    "fmt"
    "runtime"
)

var (
    version = "dev"
    commit  = "none"
    date    = "unknown"
)

func versionString() string {
    return fmt.Sprintf("shortener %s (%s) built %s with %s",
```

```
        version, commit, date, runtime.Version())
}
```

Wire it into `main` before anything else happens:

```
if len(os.Args) > 1 && os.Args[1] == "version" {
    fmt.Println(versionString())
    return
}
```

A `Makefile`, because the build command is now too long to retype:

```
VERSION ?= $(shell git describe --tags --always --dirty)
COMMIT  := $(shell git rev-parse --short HEAD)
DATE    := $(shell date -u +%Y-%m-%dT%H:%M:%SZ)
LDFLAGS := -s -w \
    -X main.version=$(VERSION) \
    -X main.commit=$(COMMIT) \
    -X main.date=$(DATE)

.PHONY: build test release clean

build:
  go build -trimpath -ldflags="$(LDFLAGS)" -o bin/shortener .

test:
  go test -race -cover ./...

release:
  GOOS=linux   GOARCH=amd64 go build -trimpath \
      -ldflags="$(LDFLAGS)" -o bin/shortener-linux-amd64 .
  GOOS=linux   GOARCH=arm64 go build -trimpath \
      -ldflags="$(LDFLAGS)" -o bin/shortener-linux-arm64 .
  GOOS=darwin  GOARCH=arm64 go build -trimpath \
      -ldflags="$(LDFLAGS)" -o bin/shortener-darwin-arm64 .
  GOOS=windows GOARCH=amd64 go build -trimpath \
      -ldflags="$(LDFLAGS)" -o bin/shortener-windows-amd64.exe .

clean:
  rm -rf bin
```

The `Dockerfile`:

```
FROM golang:1.25-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
ARG VERSION=docker
RUN CGO_ENABLED=0 go build -trimpath \
```

```
    -ldflags="-s -w -X main.version=$VERSION" -o /bin/app .

FROM gcr.io/distroless/static-debian12
COPY --from=build /bin/app /app
USER nonroot:nonroot
EXPOSE 8080
ENV DB_PATH=/data/links.db
ENTRYPOINT ["/app"]
```

Build and check the results:

```
make build && ./bin/shortener version
make release && ls -lh bin/

docker build --build-arg VERSION=1.0.0 -t shortener .
docker images shortener          # a few megabytes, not a few hundred
docker run -p 8080:8080 -v $(pwd)/data:/data shortener
```

Three things to notice. `make release` produces four binaries for four platforms from one machine, in seconds, with no cross-compilation toolchain, because nothing in this project uses cgo. The container has no shell at all, so `docker exec -it ... sh` fails: that is the point of distroless, and it is worth experiencing once so the tradeoff is a decision rather than a surprise. And the whole image is small enough that pulling it is not a deployment consideration.

Compare the image size to a Node service doing the same job. That number is the argument of this chapter.

YOUR TURN

Add `go:embed` for a small HTML page at `/` that posts to `/shorten`, so the single binary serves a usable interface as well as an API. Then add a `/healthz` that reports the version, and confirm with `docker run` that the version baked in by the build argument is what the endpoint reports. That round trip, from build flag to running container to HTTP response, is the whole deployment story in one check.

0

The twenty six problems, and what a framework is for

You can now write Go. You have built a CLI, an HTTP service, a database layer and a deployable binary with no framework at all. So the fair question before part four is: **why would you add one?**

The answer is not "to write less code". It is that every project runs into the same list of problems, the list is longer than people admit, and most of it has nothing to do with the product you were hired to build.

Two lists, and they belong to different people

The problems split in two, and the split matters because the two halves are paid for by different people at different times.

The developer's twelve cost you the first two weeks. You feel them immediately and you can name them from memory.

The client's fourteen are questions asked before anybody mentions your actual feature. They decide whether there is a project at all, and they are always promised for later.

The developer's twelve

D1	The copy-paste start. Auth, storage and email lifted from the last repository, bugs included
D2	Wiring the monorepo: workspaces, ports, env files, Docker, before anything runs
D3	The same type written twice, or four times: Go, TypeScript, the validation schema, the mobile client
D4	The same UI for every table. Twenty tables, twenty slightly different list pages
D5	Testing performance is its own project, so it waits for a quiet week that does not come
D6	No window into the database without installing one more tool

D7	The API nobody wrote down. Hand-written OpenAPI is accurate for one afternoon
D8	Import and export are always a ticket, and always a week rather than an afternoon
D9	Nothing to develop against: empty tables, and mail you cannot read
D10	Changing the schema is the scary part: the migration that half ran, with no way back
D11	The frontend is somebody else's job, and there is nobody else
D12	Generating it with a model costs money every time, and none of it compounds

Most of these you have met in this book. You wrote the error boundary in chapter eleven, the middleware in twenty one, the config validation in twenty four. None of it was hard. All of it was time, and none of it was the product.

> **IF YOU KNOW JAVASCRIPT**
> D11 deserves a sentence, because it is the reason many readers picked this book up. You are fluent in React and now also in Go, which makes you the rare person who can do both. The cost is that you are now the person who does both, for every project, forever. A generated admin panel is not about disliking frontend work; it is about not rebuilding the same table for the twentieth time.

The client's fourteen

These arrive as questions, usually in the first meeting, before anybody has described the actual product.

B1	Can different people see different things?
B2	What happens if we get hacked?
B3	Does it work without internet?
B4	How will we know when it breaks?
B5	Is any of this backed up?
B6	What if it gets busy?
B7	Who changed this?
B8	Our users photograph things on their phones
B9	Can two of our clients use one system?
B10	Does the screen update by itself?
B11	Can we switch this on for one customer only?

B12	What if two people edit the same record?
B13	Does it talk to our other systems?
B14	Someone has asked us to delete their data

The honest answer to every one of them is "yes, in about three weeks", and the client hears that as a no.

Notice what they have in common: each is real work with **no demo attached**. You cannot show a stakeholder an audit log. So each gets promised for phase two, and offline, audit and observability are the three that never arrive.

Why these survive a decade of excellent libraries

Go's ecosystem is superb at solving one problem each. A router, a queue, a migration tool, an image decoder, all excellent. So why is the list still here?

Libraries solve one problem each. Nobody hands you the shape. A router solves routing. Nothing solves "what does an application made of these look like", so the shape is hand-carved on every project, slightly differently.

A template rots the day you clone it. A boilerplate repository freezes the bugs of the day you started. The fix you make on project five never reaches projects one to four, and you cannot tell which repository has which. This is D1 with extra steps.

The client's fourteen are always phase two. Expensive, invisible, always cut. Which is a precise definition of what belongs in a framework.

> **WATCH OUT**
> D12 is the newest and it is worth stating carefully, because the obvious version of the argument is wrong.
>
> An agent will write this scaffold for you. A Grit project measures about 74,500 lines, which is roughly three quarters of a million output tokens: at current frontier pricing, tens of dollars. That is cheap for two weeks of work and anybody who tells you the cost alone is the objection is overselling.
>
> The objection is the other three properties. You pay it **again** on the next project. The model writes something **different** each time, so the auth it gives you in March and in June are different code with different bugs. And because they are different, a fix you make in one project has nowhere to go.
>
> That is D1, the copy-paste start, with a bill attached.

The question a framework has to answer

So the question is not **which library**. It is:

> *Can the shape of an application be generated, owned outright by the team that generated it, and still improve after they own it?*

Three clauses, and they fight each other.

Generated is easy. Plenty of tools generate code.

Generated and owned is harder, and is where most scaffolders stop: they hand you a repository and walk away.

Generated, owned, and still improving is the actual problem, because the moment code is yours, nobody can fix it but you. That third clause is what `grit upgrade` exists for, and chapter 29 is about how the line between "yours" and "the framework's" is drawn.

What this means for the rest of the book

Part four explains Grit's answer to that question: what `grit new` produces, how the architecture is laid out, why the **resource** is the unit of generation, and how the generated code stays plain Go that you can read with everything you learned in parts one to three.

Part five builds real applications with it.

Two promises about how this is written. Every claim about what Grit does comes from the codebase rather than from marketing, and where it does not close a problem, the book says so: B6, "what if it gets busy", is the honest example, because a framework can hand you a cache, read replicas and a job queue, and it cannot tell you which of them your bottleneck needs.

IF YOU KNOW JAVASCRIPT

You could also solve all twenty six yourself. You now know enough to. The question is whether you want to solve them on this project, and then again on the next one, and whether the version you write on project five can reach projects one to four.

If your application is genuinely unusual, a handful of endpoints with an unusual data model and no admin surface, then the honest advice is to keep writing what you wrote in part three. A framework earns its place on CRUD-shaped products with an admin panel, several entities and a client who asks the fourteen questions. Most business software is exactly that, which is why these frameworks exist.

YOUR TURN

Before the next chapter, go through both tables and mark each problem as one of three things: **solved** on your current project, **promised** for later, or **not yet asked about**. Be honest about the middle column.

> Then count the “promised” ones. That number is what a framework is offering to buy from you, and whether the trade is worth it depends almost entirely on how large it is.

0

grit new: what lands, and the shape of a project

One command, and the answers to most of chapter twenty six are already in your directory. This chapter is a tour of what arrives, so that nothing in part five is a surprise.

Installing

```
curl -fsSL https://gritframework.dev/install.sh | sh    # macOS, Linux
# Windows PowerShell
irm https://gritframework.dev/install.ps1 | iex
go install github.com/MUKE-coder/grit/v3/cmd/grit@latest
```

```
grit version
```

Creating a project

```
grit new clinic
```

It asks a few questions, or you can answer them up front:

```
grit new clinic --api              # API only, no frontend
grit new clinic --architecture triple
```

The **architecture** decides how many frontends you get. It is a word this framework uses in a specific way, and it is worth pinning down now because it is not the same thing as an architecture style:

`triple`	Web app, admin panel and API, as three applications
`double`	Web app and API, with admin as role-protected routes in the web app
`single`	One binary serving an embedded SPA
`api`	The Go API alone
`mobile`	API plus an Expo application

> **WATCH OUT**
> In Grit, **architecture** means that choice and nothing else. The arrangement of the code inside, the modular monolith with its layers, is called the architecture **style**, and the two are documented separately for exactly this reason. When someone asks "which architecture does Grit use", find out which question they are asking.

What arrives

```
clinic/
├── grit.json                 the project's own description
├── docker-compose.yml        Postgres, Redis, MinIO, Mailhog
├── docker-compose.prod.yml
├── apps/
│   ├── api/                  the Go backend
│   ├── web/                  the public Next.js app
│   └── admin/                the admin panel
├── packages/
│   └── shared/               Zod schemas and TypeScript types
└── tests/
```

The Go side is where this book's attention goes:

```
apps/api/
├── go.mod
├── Dockerfile
├── cmd/
│   ├── server/               the API
│   ├── migrate/              schema changes
│   ├── seed/                 realistic data
│   ├── backup/               and
│   └── restore/              the half that matters
└── internal/                 about fifty packages
```

Five commands rather than one. `cmd/restore` existing beside `cmd/backup` is B5 from the last chapter answered honestly: a backup you have never restored is a rumour, so the restore is a first-class command rather than a runbook.

The internal packages

Around fifty, and the names tell you what has already been decided for you. Grouped by what they answer:

Your layers	`models`, `services`, `handlers`, `routes`, `policies`
Identity	`authz`, `password`, `totp`, `access`, `crypto`

Data	`database`, `migrate`, `paginate`, `concurrency`, `money`, `ids`
Work	`jobs`, `cron`, `outbox`, `saga`, `workflow`, `events`
Outside	`storage`, `files`, `media`, `mail`, `sms`, `webhooks`, `safefetch`
Operations	`audit`, `appendonly`, `backup`, `health`, `tracing`, `flags`
Compliance	`erasure`, `gdpr`-adjacent pieces, `settings`
Delivery	`respond`, `sanitize`, `imports`, `export`, `pdf`, `realtime`, `sync`

You will not read all of these. The point of listing them is that each one is a thing you did not write, and most of them correspond to a lettered problem from chapter twenty six.

Running it

```
cd clinic
grit migrate        # create the schema
grit seed           # realistic data to develop against
grit start          # everything, together
```

`grit start` brings up the API, the frontends and the infrastructure at once, which is D2 gone. What is now listening:

`:8080`	The Go API
`:3000`	The web app
`:3001`	The admin panel
`/studio`	A database browser: tables, queries, the ERD
`/docs`	The API reference, generated from the registered routes
`/pulse`	Latency, query counts, profiles
`/sentinel`	Errors, with the request attached

The second group is the interesting one. `/studio` is D6, `/docs` is D7 and `/pulse` is D5, and all three are paths that were already there rather than tools you installed.

> **IF YOU KNOW JAVASCRIPT**
> `/docs` deserves a note because it is the kind of thing that is usually a lie. It is assembled from the routes actually registered at runtime, not from comments or a hand-written specification. It cannot drift, because there is no second description to drift from. That is the only way an API document stays true, and it is why Grit generates one rather than asking you to maintain one.

grit.json, and the seeding that matters

`grit.json` records what the project is: its architecture, its module toggles, its name. The CLI reads it so that later commands know the shape of the thing they are editing.

The seed is worth insisting on, because it is the habit people skip:

```
grit seed Patient --count 50000
```

Fifty thousand rows is where you discover whether the list page works. At ten rows the sort is instant, pagination has one page, and an N+1 costs ten queries instead of ten thousand. That is D9, and the seeder is built as the pipeline from chapter fourteen: rows built in chunks, written in batches, with the number in flight bounded by a channel, so a million rows runs in about a minute with the heap flat.

Modules you can switch off

Eleven subsystems are toggles rather than permanent fixtures:

```
MODULE_AI=false
MODULE_AUDIT=false
MODULE_BACKUP=false
MODULE_CRON=false
MODULE_FILES=false
MODULE_FLAGS=false
MODULE_JOBS=false
MODULE_MAIL=false
MODULE_REALTIME=false
MODULE_TWOFACTOR=false
MODULE_WEBHOOKS=false
```

Set one to `false` in `.env` and that subsystem does not start. Delete its package and the rest still builds. This is the "batteries included, individually removable" claim, and it is checkable: the packages are separate and the seams are real.

> **WATCH OUT**
> The default project is the **hardened** one. Auth, sessions, the audit chain, backups and the upload pipeline are present before you write a line, rather than being plugins you remember to enable.
>
> That is deliberate and it is worth stating as a principle: an opt-in security feature is a security feature most projects do not have. The cost is a bigger project on day one, which is a real cost and the trade this framework makes on purpose.

It is still just Go

Open `apps/api/internal/services/user.go`. It is a struct with a `*gorm.DB` field and methods on it. Open a handler: it reads a request, calls one service method and writes a response. There is no framework type in your code and nothing to learn before you can read it.

```
// The shape of every generated service.
type ContactService struct {
    DB *gorm.DB
}
```

That is the whole dependency. If Grit disappeared tonight, this project compiles tomorrow, because there is nothing of Grit inside it. Everything you learned in parts one to three applies directly, which was the reason for putting those parts first.

PROJECT 24: MAKE A PROJECT AND LOOK AROUND

No code to write. Make one and read it, because the rest of part four assumes you have seen these files.

```
grit new bookshop --api
cd bookshop
grit migrate
grit seed
grit start
```

Then answer these by reading, not by searching the documentation.

One. Open `apps/api/internal/handlers/user.go` and find where it calls the service. How many lines of that handler are about HTTP, and how many are about users? That ratio is the thin-handler rule from chapter eleven, enforced by the generator.

Two. Open `apps/api/internal/services/user.go` and find `paginate.Config`. It lists which columns a client may search, sort and filter by. Why is that a whitelist rather than passing the client's column name through? The answer is in chapter twenty two, under placeholders.

Three. Visit `/studio` and look at the tables. Then visit `/docs` and find the route for listing users. Compare the parameters it documents with the `paginate.Config` you just read: they are the same list, because one is derived from the other.

Four. Count the entries in `apps/api/go.mod`. Then count what a comparable Express project needs in `package.json`. Neither number is a virtue on its own, and the comparison is worth seeing.

Five. Set `MODULE_MAIL=false` in `.env`, restart, and confirm the API still starts. Then look at what disappeared from the startup log.

YOUR TURN

Run `grit new` twice with the same name into different directories and compare the two trees byte for byte. They should be identical: the generator is deterministic, which is the property chapter twenty six said a model cannot give you. Then run `grit version` and note it, because the next chapters describe this version and the framework moves quickly.

0

The architecture: a modular monolith, layer by layer

You have the project. This chapter is the map: how the code is arranged, which layer is allowed to call which, and why those rules exist. Get this and everything in part five is predictable.

The short answer

A modular monolith with a layered API and an in-process event bus. One binary, one database, about fifty internal packages.

That is the headline and it is slightly misleading, because no real system is one style. A Grit project runs six at once, each where it earns its keep:

Modular monolith	The whole API: one deployment, one transaction boundary
Layered (n-tier)	routes, handlers, services, models
Event-driven	`internal/events` : a write emits once, subscribers react
Transactional outbox	`internal/outbox` : durable delivery of those events
Orchestration	`internal/saga` : multi-step work that can half fail
Microkernel	The framework itself: `grit plugin add`

> **IF YOU KNOW JAVASCRIPT**
> Coming from Next.js, the mental shift is that there is one process rather than a function per route. State can live in memory, a connection pool is warm, a background worker runs, and nothing is cold-started. The cost is that you deploy and scale one thing, which for most products is a relief rather than a limitation.

The request path

Follow one request all the way down. This is the shape every generated resource has.

```
HTTP request
  -> routes       which URL maps to which handler, and what guards it
```

```
  -> middleware   auth, CORS, rate limit, recover, request id
  -> handler      read the request, call one service method,
                  write the answer
  -> service      the rules, and every database call
  -> model        the struct, its GORM tags, its hooks
  -> database
```

Each layer talks only to the one below it. A handler never touches the database; a model never calls a service.

The rule the layering exists for

Handlers are thin **on purpose**, and the reason is not tidiness.

```go
// A generated handler, in essence.
func (h *ContactHandler) List(c *gin.Context) {
    opts := paginate.FromQuery(c, contactListConfig)
    result, err := h.service().List(h.ctx(c), opts)
    if err != nil {
        respond.Error(c, err)
        return
    }
    respond.Paginated(c, result)
}
```

Read the request, call one method, write the answer. Every rule lives one layer down, in the service.

The payoff: a rule enforced in the service is enforced **everywhere**. The background job uses it. The CSV import uses it. The CLI command uses it. The test uses it. A rule written in a handler holds until the first time something other than a browser does the same thing, and then it silently does not.

```go
// Every service method takes a context, not a *gin.Context.
func (s *ContactService) List(ctx context.Context,
    opts paginate.Options) (*paginate.Result[models.Contact], error)
```

That signature is the whole architecture in one line. `context.Context` rather than `*gin.Context` means the service has no idea it is being called over HTTP, so a job can call it with `context.Background()`.

> **WATCH OUT**
> The context carries more than cancellation here. It holds the organisation the multi-tenant plugin resolved and the actor that per-record ownership is scoped by. That is why passing the request's context down matters rather than creating a fresh one: a

`context.Background()` in the middle of a request loses the caller's identity, and the query comes back unscoped.

When a background job genuinely needs to act as nobody, there is an explicit way to say so rather than an accident.

Where the rules live, concretely

Open any generated service and you find three things above the methods:

```
// What a client may search, sort and filter by. Whitelisted, because each
// name ends up in SQL.
var contactListConfig = paginate.Config{
    Searchable: []string{"name", "phone"},
    Sortable:   map[string]bool{"id": true, "created_at": true,
        "name": true, "phone": true},
    Filterable: map[string]bool{"id": true, "name": true,
        "phone": true, "group_id": true},
}

// Every column Patch and Bulk may write. id, the timestamps and
// the version are the framework's, and are dropped.
var writableContact = map[string]bool{
    "name": true, "phone": true, "group_id": true,
}
```

These are the answer to chapter twenty two's warning that a column name cannot be a placeholder. The client sends `?sort=name`, and `name` is checked against a map the generator wrote, rather than being interpolated into SQL.

IF YOU KNOW JAVASCRIPT

This is the difference between an ORM protecting you and a convention protecting you. Prisma builds the query so the column is validated by its schema. Here the query is yours, so the whitelist is explicit and sitting at the top of the file where a reviewer sees it.

The event bus

A write emits once, and the systems that care subscribe.

```
events.Emit(c, events.Event{
    Name: "invoices.paid", Resource: "invoices", ID: inv.ID, After: inv,
})
```

The audit log, outbound webhooks and realtime clients are **subscribers**, not calls the handler has to remember. Adding a fourth consumer does not mean editing every handler.

Three delivery modes, and choosing correctly matters:

`Sync`	Runs inline, before the response. For the audit record, which must be written before the caller is told the write succeeded. A slow Sync subscriber is a slow endpoint
`Async`	Runs on a worker pool after the handler returns. For anything with a network call: a webhook that takes four seconds must not make the API take four seconds
`Durable`	Through the transactional outbox: at least once, retried, surviving a restart. For work another module depends on

`Durable` is the one worth understanding, because it is the difference between an event system and a reliable one. The event is written **in the same database transaction as the data**, so a crash between the commit and the HTTP call loses nothing. A relay then delivers it, claiming messages with a lease so another replica takes over if one dies.

> **IF YOU KNOW JAVASCRIPT**
> The naive version is a `POST` at the end of the handler. It works in the demo and loses events in production: the commit succeeded, the HTTP call did not, and nothing records that anything was owed. The outbox is the standard fix and it is already wired, which is B13 from chapter twenty six.

What it deliberately is not

Four, each with a reason rather than a dismissal:

Not microservices. One binary by design. A split is worth paying for when teams need to release independently, and until then it buys network failures and a deployment pipeline in exchange for nothing.

Not CQRS or event sourcing. Both earn their complexity in a hot spot, not across a codebase. The read path is already separate enough to point at another store when one query needs it.

Not hexagonal. The generated service talks to GORM directly rather than through a port. Less indirection to read, at the cost of swapping the ORM being real work. That is a trade, stated as one.

Not serverless. Jobs, a scheduler, realtime connections and a warm pool all assume a process that stays up.

Organised by layer, not by feature

A fair question, since package-by-feature is what many Go teams land on: why is it `models/`, `services/`, `handlers/` rather than `invoice/`, `customer/`?

What feature folders buy is real: a feature is a folder, you can delete one by deleting a directory, and the root stops growing. At forty resources Grit's `services/` is forty files and the directory stops telling you which three form a subsystem.

What the layered layout buys back is the **entity graph**. A business application has thirty entities pointing at each other: invoice to customer to organisation and back. **Go forbids import cycles**, so with feature folders the moment `customer` refers to `invoice` you need an event or an interface dance to express what is really a foreign key.

Grit's thirty-odd models are one flat `package models`. Any model can reference any other with no ceremony, which is what lets `client:belongs_to` be a single word in a generate command rather than a design conversation.

> **WATCH OUT**
> The import-cycle rule still bites, in the other direction. `models` cannot import `services`, so a hook on a model calls the lower-level package directly rather than the service that wraps it. If you add a `BeforeCreate` that needs a sequence number, call the sequence package, not the service.
>
> This is the kind of constraint that is obvious once stated and costs an hour when discovered by trial.

Can you refactor to something else?

Yes, and the useful part of the answer is this: **what decides whether a refactor is a weekend or a quarter is almost never the code. It is the data.**

Moving a package is easy. Moving a table out from under three foreign keys, two joins and an admin screen that lists both sides of a relationship is the real work, and no amount of clean code makes it cheaper.

CQRS on one endpoint	hours	The read path is already a separate whitelist. Point one query at a read replica
Extract one module	weeks	The service is the seam. The cost is the foreign keys you must cut
Fully hexagonal	ongoing	Mechanical, and the generator keeps writing the un-ported version
Microservices	months	Only with an organisational reason. "It will scale" is not one

Event sourcing	a rewrite	State stops being a row. Not a refactor
Collapse services in	weeks	Easier than the reverse, and often the right direction

PROJECT 25: TRACE A REQUEST END TO END

No new code. Follow one request through every layer of the project you made last chapter, with the profiler open, and the architecture stops being a diagram.

```
cd bookshop
grit start
```

One. Open `apps/api/internal/routes/` and find where the user routes are registered. Note which middleware guards them and in what order. The order is the composition from chapter twenty one: outermost runs first.

Two. Open the handler that route points at. Count its lines, then count how many are about HTTP rather than about users. Follow the one service call.

Three. In the service, find `paginate.Config` and the writable map. Make a request with `?sort=nonsense` and see what happens. Then make one with a column that **is** in the whitelist and watch it work. You have just tested the defence from chapter twenty two.

Four. Open `/pulse` and make the same request again. Note the query count for one request. Then create a resource with a `belongs_to` relationship, list it, and watch the query count: if it jumps by the number of rows, you are looking at an N+1, and the "Issued from" column names the file and line that caused it.

Five. Make a write, then open `/studio` and look at the audit table. The handler never called the audit log; it emitted an event, and audit was a subscriber. That indirection is the event bus doing its job.

YOUR TURN

Find a service method that takes `ctx context.Context` and call it from a `cmd/` command with `context.Background()`, bypassing HTTP entirely. It should work, because the service never knew it was being called over HTTP. That is the thin-handler rule paying off, and proving it once is worth more than reading the argument for it.

0

The resource, and why it is the unit of generation

This is the idea the framework rests on. If you disagree with it, the rest of part four is wrong, so it gets a chapter of its own.

The claim

A resource is the only thing that spans the whole stack, so it is the thing worth generating. Not a file. Not a layer. The invoice, from the database column to the form field.

Consider what "add invoices to this application" actually requires:

Database	a table, columns, indexes, a foreign key
Go	a model, a service with the rules, a handler, routes
Contract	a validation schema, a TypeScript type
Frontend	a data hook, a list page, a form, a detail view

Thirteen or so files. A scaffolder that generates **a model** has automated the easy part, because the model was never the expensive bit. The expense is that one concept, "invoice", has to be spelled correctly in thirteen places, and kept correct as it changes.

> **IF YOU KNOW JAVASCRIPT**
> This is D3 from chapter twenty six, and you have felt it. The Go struct, the Zod schema, the TypeScript type and the mobile client are four descriptions of one thing, kept in step by memory. Nothing fails to compile when they drift. You find out in the browser, from a user, as a blank column or a wrong total.

One command

```
grit generate resource Invoice \
  --fields "number:string, total:money, due:date, client:belongs_to"
```

Four field types. One of them is a relationship. That is the entire input.

What it writes, by name:

```
apps/api/internal/models/invoice.go
apps/api/internal/services/invoice.go
apps/api/internal/services/invoice_import.go
apps/api/internal/handlers/invoice.go
apps/api/internal/handlers/invoice_import.go
apps/api/internal/handlers/invoice_test.go
apps/api/internal/policies/invoice.go
apps/api/internal/factory/invoice.go
packages/shared/schemas/invoice.ts
packages/shared/types/invoice.ts
apps/admin/app/_dashboard/resources/invoice/index.tsx
```

Plus the routes registered, the model added to the migration list, and the resource added to the admin's navigation.

> **WATCH OUT**
> Note `index.tsx` in the resource folder. The generator writes it once and then **never regenerates it**, because that is where your custom cells and forms go. The distinction between "regenerated" and "written once" runs through the whole system, and chapter thirty one is about where that line sits.

Why one description is the point

The thirteen files are not the achievement. The achievement is that they come from **one description**, so they cannot disagree.

Add a field later:

```
grit generate field Invoice paid_at:datetime
```

That reaches the model, the migration, the Zod schema, the TypeScript type, the admin table and the admin form in one step. There is no version of this where the backend has `paid_at` and the frontend does not, because there is no second place to update.

That is the real answer to D3. Not "less typing": **no drift**.

A field type is a decision made once

Thirty one field types ship. The important ones are not the obvious ones.

```
string  text  richtext  int  uint  float  bool  toggle  select  radio
check  datetime  date  money  slug  belongs_to  one_to_one  many_to_many
string_array  file  files  email  url  domain  tel  country  color
percent  rating  time  json
```

Take `money`. Writing `total:money` decides a column type, a Go type, a Zod rule, an input mask and how the number is formatted in a table cell. Say `float` instead and you have shipped a rounding bug that will surface in a total months later, found by an accountant.

```
// What money generates: an integer, in minor units.
Total money.Amount    // stored as cents, never a float64
```

This is the lesson from chapter nineteen, where you implemented `MarshalJSON` for exactly this. The difference is that here it is a **type** rather than advice, so the correct decision is the default rather than something you must remember.

Take `belongs_to`. One word, and you get:

- a `ClientID` foreign key column, with a constraint
- a `Client` field on the struct, preloaded on read
- a nested type in the generated TypeScript
- a searchable dropdown in the admin form that shows the client's **name** rather than a UUID

> **IF YOU KNOW JAVASCRIPT**
> In a hand-written stack each of those is a separate decision someone makes slightly differently. The dropdown showing a UUID instead of a name is the canonical example: it is nobody's bug, everybody has shipped it, and it happens because the frontend had no way to know which column is the human label. Here the generator knows, because `belongs_to` told it.

It is still your code afterwards

Open the generated service. It is a struct with a `*gorm.DB` and methods, with comments explaining **why** rather than what:

```
// ContactService owns every database read and write for contacts.
//
// The handler reads the request, calls one of these and writes the
// answer. A job, a command or a test calls the same methods with no
// request at all, so a rule enforced here is enforced everywhere.
type ContactService struct {
    DB *gorm.DB
}
```

One dependency, and it is GORM. You can edit any of it. The generated code is the starting point, not a sealed artifact.

WATCH OUT

Re-running `grit generate resource` on an existing resource does **not** silently overwrite your edits. Every generated file is hashed into a manifest when written, and a file whose hash has moved is reported rather than replaced.

This behaviour exists because the opposite happened: an early version rewrote the model, service and handler on a re-run, printing a green tick for each, and everything hand-written in them was gone. No prompt, no backup, and the documentation had promised that the service was where your business logic goes. A newcomer lost a `BeforeSave` rule and a service method in their first week.

The lesson generalises past this framework: a generator that overwrites without asking is a generator you cannot trust with a file you have edited.

What it does not generate

Worth saying plainly, because the demo makes it look like magic.

It generates the **shape**. It does not generate the rule that an invoice cannot be billed twice, the report the client asks for every month, or the pricing logic that is the reason they hired you. That is still yours, it is still the hard part, and it is still where the value is.

What has gone is the two weeks before you got to any of it.

PROJECT 26: GENERATE A RESOURCE AND READ EVERY FILE

In the project from chapter twenty seven:

```
grit generate resource Book \
  --fields "title:string, isbn:string, price:money, published:date"
grit migrate
grit seed Book --count 500
grit start
```

Then open every file it named, in this order, and answer one question about each.

`models/book.go`. Find the `money` field. What Go type is it, and what does the GORM tag say the column is? Why is that not a `float64`?

`services/book.go`. Find `paginate.Config`. Which columns can a client sort by, and which cannot? Now try `?sort=price` and `?sort=secret` against the running API.

`handlers/book.go`. Count the lines that are about HTTP. Find the one call into the service per endpoint.

`packages/shared/schemas/book.ts`. Find the Zod rule for `price` and compare it to the Go validation. They came from the same word in your command.

`packages/shared/types/book.ts`. Compare the TypeScript type to the Go struct field by field.

The admin panel at `:3001`. Find the Books page. Sort by title, search, open the form. Notice the price input: it is a money input, not a number input, because of one word in your command.

Now add a field and watch the whole chain move:

```
grit generate field Book pages:int
grit migrate
```

Reload the admin. The column is in the table and the field is in the form, and you edited nothing. Check `types/book.ts`: it is there too.

Finally, prove the manifest. Add a method to `services/book.go` by hand, then re-run the original `grit generate resource Book` command. It should refuse to overwrite rather than quietly eating your method, and tell you why.

YOUR TURN

Generate a resource with a deliberately wrong field type, `price:float` instead of `price:money`, and look at what changes in the admin form and the Zod schema. Then compute `0.1 + 0.2` in the Go REPL of your choice, or just `fmt.Println(0.1 + 0.2)`, and decide for yourself whether a `money` type is pedantry or the single most valuable field type in the list.

0

Field types: what each one decides

Thirty one field types ship. This chapter is not a reference, it is an argument about why a field type is the right place to put a decision.

The argument

When you write `total:money`, you are not choosing a storage format. You are making five decisions at once:

Database	an integer column in minor units, never a float
Go	a `money.Amount`, with arithmetic that cannot drift
Validation	a Zod rule that accepts `12.34` and rejects `12.345`
Form	a money input with a currency prefix and two decimal places
Table	right aligned, formatted for the locale

Say `float` instead and all five change, each in a direction that is slightly wrong, and the one that costs you is the first. `0.1 + 0.2` is not `0.3` in any language with IEEE floats, and an invoice total that is out by a penny is found by an accountant rather than by a test.

> **IF YOU KNOW JAVASCRIPT**
> You already wrote this fix by hand in chapter nineteen, implementing `MarshalJSON` on a `Money int64`. That was the right instinct. The difference here is that the correct decision is the **default**, applied in five places consistently, rather than something each developer remembers or does not.

The full list

```
string  text  richtext  int  uint  float  bool  toggle  select  radio
check  datetime  date  money  slug  belongs_to  one_to_one  many_to_many
string_array  file  files  email  url  domain  tel  country  color
percent  rating  time  json
```

Most are unsurprising. The ones worth knowing properly are below.

Text in three sizes

`string`, `text` and `richtext` differ in the column type and the input:

`string`	A varchar, a single line input. Names, codes, references
`text`	A text column, a textarea. Descriptions, notes
`richtext`	A text column holding HTML, and an editor in the form

`richtext` is the one with a hidden cost: it stores HTML that a user supplied, so it has to be sanitised on the way in or you have shipped stored cross-site scripting. The generated service does that, which is the reason to use the field type rather than a `text` plus an editor you wire yourself.

Choices, in three shapes

`select`, `radio` and `check` all constrain a value to a set, and differ only in the control:

```
grit generate resource Ticket \
  --fields "status:select:open=Open|closed=Closed|held=On hold"
```

The values before the `=` go in the database; the labels after it go on the screen. This produces a constraint, a Go constant set, a Zod enum and a dropdown, from one string.

> **WATCH OUT**
> Use `select` for one choice from many, `radio` when there are three or fewer and the user should see them all at once, and `check` for several choices at once, which stores a `string_array`.
>
> The trap is choosing `string` and validating in your head. A free `status` column collects `open`, `Open`, `OPEN` and `opened` within a year, and no report ever agrees again.

Relationships

Three, and they are the field types that save the most work.

`belongs_to` is one word that produces a foreign key column with a constraint, a preloaded struct field, a nested TypeScript type, and a searchable dropdown in the form that shows the related record's **label column** rather than a UUID.

```
--fields "client:belongs_to"
```

```
ClientID uint    `json:"client_id"`
Client   *Client `json:"client"`     // preloaded on read
```

`one_to_one` is the same with a uniqueness constraint. `many_to_many` creates the join table and a multi-select picker.

> **IF YOU KNOW JAVASCRIPT**
> The dropdown showing a name rather than an ID is the detail worth noticing, because it is the canonical hand-written bug. It happens because the frontend has no way to know which column is the human label, so it falls back to the primary key. The generator knows, because `belongs_to` told it to look.

Identity and files

`slug` derives a URL-safe string from another field and keeps it unique, which is three decisions people usually get wrong separately: how to slugify, when to regenerate, and what to do about collisions.

`file` and `files` store an upload reference rather than bytes in a column, and wire the upload pipeline from chapter forty two: resize, re-encode, strip EXIF, generate a thumbnail. Writing `photo:file` is how B8 gets answered.

Validated strings

`email`, `url`, `domain`, `tel` and `country` are all strings with a format. They look cosmetic and are not: each produces a matching rule in Go **and** in Zod, so the browser and the API agree about what is valid.

> **WATCH OUT**
> The point is the agreement, not the validation. Validating only in the frontend is a user-experience feature, not a security control, because anybody can post directly to your API. Validating only in the backend means the user discovers the problem after a round trip.
>
> One field type produces both, from one definition, which is D3 again.

Numbers with meaning

`int`, `uint` and `float` are the raw ones. `percent`, `rating` and `money` are numbers that carry their own rules: a range, a step, a format. Prefer the specific one, because the constraint travels with it.

`json` stores arbitrary structure. Use it for genuinely unstructured data such as a webhook payload, and resist using it to avoid designing a schema, because nothing can query it well and nothing validates it.

Time, in three precisions

`date` is a calendar day with no time and no zone: a birthday, an invoice due date. `datetime` is an instant, stored UTC. `time` is a clock time with no date, for opening hours.

> **WATCH OUT**
> Using `datetime` for a birthday is a real bug, not a style preference. Store it as an instant and somebody in a different timezone sees the day before. The rule: **if the answer should be the same everywhere in the world, it is a `date`.**

Choosing well, in one table

Money	`money`, never `float`
A fixed set	`select`, never `string`
A calendar day	`date`, never `datetime`
A relationship	`belongs_to`, never an `int` you join by hand
An upload	`file`, never a `string` holding a path
An email	`email`, so both sides validate it
Unstructured	`json`, and only when it really is

Every row of that table is a decision that is cheap now and expensive in six months, which is the whole case for field types being a type system rather than a convenience.

> **PROJECT 27: ONE RESOURCE WITH EVERY INTERESTING TYPE**
> Generate a resource that exercises the types that carry decisions, then look at what each produced.

```
grit generate resource Property \
  --fields "title:string, slug:slug, description:richtext, \
price:money, bedrooms:int, rating:rating, available_from:date, \
viewing_time:time, status:select:draft=Draft|live=Live|sold=Sold, \
contact_email:email, website:url, photos:files, owner:belongs_to"
grit migrate
grit seed Property --count 200
grit start
```

Then check each claim rather than taking it on trust.

The money column. Open `models/property.go` and find `price`. What Go type, and what does the GORM tag say? Then open the admin form and type `12.345` into it. Then post `{"price": 12.345}` directly to the API with curl. Both should refuse, and that is the agreement this chapter is about.

The date against the time. Find `available_from` and `viewing_time` in the generated TypeScript. They are different types, because one is a day and one is a clock reading.

The select. Find the status constants in Go and the enum in the Zod schema. Then try posting `{"status": "pending"}` and read the error.

The relationship. Open the Property form and find the owner dropdown. It shows names. Now look at `services/property.go` and find the preload that makes that possible without an N+1.

The slug. Create a property called "Garden Flat", then another with the same name, and look at the two slugs.

The files. Upload two photos and then look at what landed in storage: the original is not what is served.

YOUR TURN

Generate the same resource twice, once with `price:money` and once with `price:float`, into throwaway projects. Diff the two generated models, Zod schemas and admin forms. Count how many files differ from that one word.

Then, in the float version, set a price of `0.1`, add `0.2` to it in a Go test, and assert it equals `0.3`. Watch it fail. That failing test is the entire argument of this chapter in four lines.

0

Models, migrations and seeding at scale

D10 from chapter twenty six was “changing the schema is the scary part: the migration that half ran, with no way back”. This chapter is how that is answered, and it is more interesting than it sounds, because the usual answer is a lie.

The model

A generated model is a struct with tags:

```
type Invoice struct {
    ID        uint          `gorm:"primarykey" json:"id"`
    Number    string        `gorm:"size:50;uniqueIndex" json:"number"`
    Total     money.Amount `gorm:"not null" json:"total"`
    Due       jsontime.Date `json:"due"`
    ClientID  uint          `gorm:"index" json:"client_id"`
    Client    *Client       `json:"client,omitempty"`
    Version   int           `gorm:"default:1" json:"version"`
    CreatedAt time.Time     `json:"created_at"`
    UpdatedAt time.Time     `json:"updated_at"`
    DeletedAt gorm.DeletedAt `gorm:"index" json:"-"`
}
```

Three of those fields are the framework’s rather than yours.

`Version` is optimistic locking, and chapter thirty three explains the 409 it produces. `DeletedAt` is soft delete: a `DELETE` sets the timestamp and every query excludes it automatically, which is how the Trash page works. `CreatedAt` and `UpdatedAt` are maintained by GORM.

> **WATCH OUT**
> Soft delete has a consequence people meet late: a `uniqueIndex` on a column still applies to deleted rows. Delete a user with the email `ada@example.com` and you cannot create another with that email, because the old row is still there.
>
> The usual fix is a composite unique index including `deleted_at`, so the same value can exist once among live rows and many times among deleted ones. It is worth knowing before a user reports it.

Migrations, and the honest problem

```
grit migrate
```

It reads your models and makes the database match. No migration files to write.

That convenience has a well-known cost and this is where most frameworks are quiet: **GORM's AutoMigrate has no down scripts**. It can add a column. It cannot tell you what it added last Tuesday, and it cannot undo it.

So the answer here is not a pretend rollback. It is a record:

```
grit migrate           # apply, and record what this run changed
grit migrate --status  # what previous runs did
grit migrate --down    # undo the last run, using that record
```

Each run writes down what it actually added. `--down` reads that record and drops exactly those things. It is not a general rollback, it is an undo of **recorded** runs, and the distinction matters: a change made by someone typing SQL into a console was never recorded, so it cannot be undone this way.

> **IF YOU KNOW JAVASCRIPT**
> Prisma and Rails generate a migration file per change, with an explicit `up` and `down`, which is more ceremony and gives you a reviewable artifact. This trades the file for automatic detection plus a recorded history.
>
> Neither is free. The file-based approach costs a step per change and gives you a precise diff in code review. This approach costs you reviewability and gives you speed. Know which you are using, and never assume a framework that offers `--down` can undo something it did not do.

The refusal that protects you

```
grit migrate --down
```

Dropping a column destroys data. The command asks, and a non-interactive run needs `--yes`, deliberately, so a CI pipeline cannot quietly delete a column because somebody renamed a struct field.

A rename is the case to understand. Changing `Name` to `FullName` in a struct is not a rename to AutoMigrate: it is an unknown column appearing and a known one no longer being mentioned. The safe sequence is to add the new column, backfill it, then remove the old one in a later change, which is the same expand-and-contract dance every schema tool requires.

Seeding, and why it is not optional

D9 was "nothing to develop against".

```
grit seed                        # every seeder
grit seed Invoice --count 50000  # one, at scale
```

The generator writes a factory per resource that produces realistic values for each field type: a real-looking name for `string`, a valid address for `email`, an amount in range for `money`.

At ten rows everything works. The sort is instant, pagination has one page, and an N+1 costs ten queries instead of ten thousand. Every list-page bug you have is invisible until the data arrives, which is why the habit is to seed **before** you look at a screen rather than after a user complains.

A million rows, with the heap flat

The seeder is the pipeline from chapter fourteen, in production:

```
build rows in chunks  ->  bounded channel  ->  batch writers  ->  database
```

Measured on a laptop: **999,800 rows in 59 seconds, with the heap flat at 10 MB the whole time**.

The number that matters is the third one. Sixteen thousand rows a second is a fine figure and it is not the point. The point is that memory is a property of the **pipeline** rather than of how many rows you asked for: the channel holds a fixed number of batches in flight, so ten thousand rows and ten million look the same to the heap.

> **WATCH OUT**
> The naive versions both fail, in opposite directions. One row per `INSERT` keeps memory perfect and takes an afternoon, because you pay query parsing, a transaction and a round trip a million times. Building all million rows into a slice first is fast until the process is holding a million structs plus the driver's copy, and then it swaps or is killed.
>
> **Batching the writes is the win, not the goroutines.** Going from one insert per row to one insert per batch moves the number by an order of magnitude. Parallelism on top is worth perhaps five times more. Parallelism without batching is worth almost nothing, because a million round trips is a million round trips however many goroutines are waiting on them.

The seeder is also **resumable**: it counts what is in the table and inserts only the difference. Run it twice and the second run does nothing. Kill it halfway through a million rows and run it again, and it continues.

Backups, and the half that counts

```
grit backup           # write an archive
grit restore <file>   # replay one
```

`cmd/restore` existing beside `cmd/backup` is the point. B5 asked "is any of this backed up", and the honest reading of that question is "has anyone ever restored one".

A backup you have never restored is a rumour. The archive carries a row count per table, so a restore can be checked rather than assumed, and the restore is a command rather than a runbook page nobody has followed.

PROJECT 28: BREAK A MIGRATION ON PURPOSE

The point of this project is to experience the failure modes while they are cheap.

```
grit new migrations-lab --api
cd migrations-lab
grit generate resource Product \
  --fields "name:string, price:money, sku:string"
grit migrate
grit seed Product --count 1000
```

One: watch a run get recorded.

```
grit migrate --status
```

Note what the last run says it changed.

Two: add a column and undo it.

```
grit generate field Product weight:float
grit migrate
grit migrate --status      # a new run, with the new column
grit migrate --down        # and gone again
```

Check in `/studio` that the column really left.

Three: the rename trap. Edit `models/product.go` by hand and change `SKU` to `Code`. Run `grit migrate --status` after migrating and look carefully: a column was added, and the old one is still there with the data in it. Nothing was renamed, because nothing could know that was your intention.

Now do it properly: add the new field, write a tiny `cmd/` program that copies the values across, and only then remove the old one. That is expand-and-contract, and doing it once by hand is worth more than reading about it.

Four: the scale that matters.

```
time grit seed Product --count 200000
```

Watch the progress line report rows per second and the heap. Then open the admin Products page, sort by price and search. If anything is slow, open `/pulse` and look at the query count: this is the moment that a missing index announces itself, and it is the entire argument for seeding before you demo.

Five: prove the restore.

```
grit backup
grit seed Product --count 10          # change the data
grit restore <the archive>
```

Then count the rows. Restoring once, in a project where it does not matter, is how you find out your backup process works before the night it has to.

YOUR TURN

Add a `uniqueIndex` to the product SKU, create a product, soft delete it, and try to create another with the same SKU. Read the error, then fix it with a composite index including `deleted_at` and confirm both cases now work: one live row with that SKU, and any number of deleted ones.

That is a bug most applications ship, discovered by a user who deleted something and could not re-add it.

0

Services: where the rules live, and why not the handler

The thin-handler rule has come up three times now. This chapter is the one that justifies it properly and shows what a generated service actually contains.

The rule, and the reason

Every database read and write for a resource lives in that resource's service. The handler reads the request, calls one method, and writes the answer.

The reason is not tidiness. It is this:

```
func (s *InvoiceService) Create(ctx context.Context,
    in CreateInvoice) (*models.Invoice, error)
```

That signature takes a `context.Context`, not a `*gin.Context`. The service has no idea it is being called over HTTP. So the same method serves:

- the HTTP route
- a background job
- a CLI command in `cmd/`
- the CSV importer
- a test, with no server running

A rule enforced here is enforced in all five. A rule written in the handler holds until the first time something other than a browser does the same thing, and then it silently does not. That is the whole argument, and it is why "put the validation in the handler, it is right there" is the most expensive shortcut in a web codebase.

> **IF YOU KNOW JAVASCRIPT**
> In an Express app the equivalent discipline is a service module that never imports `req` or `res`. Most teams intend this and drift, because the handler is where the data already is. Go makes the drift visible: the moment a service needs `*gin.Context` it must import Gin, and that import is a review comment waiting to happen.

What a generated service contains

Four things, in this order.

The struct, with one dependency.

```
type ContactService struct {
    DB *gorm.DB
}
```

The list configuration, which is a security boundary:

```
// What a client may search, sort and filter by. Whitelisted, because each
// name ends up in SQL.
var contactListConfig = paginate.Config{
    Searchable: []string{"name", "phone"},
    Sortable: map[string]bool{
        "id": true, "created_at": true, "name": true, "phone": true,
    },
    Filterable: map[string]bool{
        "id": true, "name": true, "phone": true, "group_id": true,
    },
}
```

The writable set, which is the other one:

```
// Every column Patch and Bulk may write. id, the timestamps and the
// version are the framework's, and are dropped.
var writableContact = map[string]bool{
    "name": true, "phone": true, "group_id": true,
}
```

The methods: `List`, `GetByID`, `Create`, `Update`, `Patch`, `Delete`, `Bulk`, `BulkCreate`, `BulkEdit`, `Export`.

The two whitelists are the chapter

Chapter twenty two said a value can be a placeholder and a column name cannot. These maps are the consequence.

A client sends `?sort=name&filter[group_id]=3`. Those strings end up in SQL as **identifiers**, which no placeholder can protect. So each is checked against a map the generator wrote:

```
// Roughly what paginate does with it.
if !cfg.Sortable[requested] {
    requested = "created_at"      // fall back, never interpolate
}
```

`writableContact` is the same defence for writes, and it closes a different hole. A `PATCH` that merged the client's JSON straight into the model would let anybody set `id`, `created_at`, `version`, or on a user record, `role`. That is mass assignment, and it is how a normal user becomes an administrator.

> **WATCH OUT**
> When you add a field by hand rather than with `grit generate field`, **you must add it to these maps yourself**. The symptom of forgetting is not an error: sorting by the new column silently falls back, and patching it silently does nothing.
>
> This is the main reason to use `grit generate field` rather than editing the struct: the command updates the model, the migration, the two maps, the Zod schema, the type and the admin form together.

Ownership travels in the context

```
func (s *ContactService) db(ctx context.Context) *gorm.DB {
    return s.DB.WithContext(ctx)
}
```

That looks trivial and is not. The context carries the organisation the multitenant plugin resolved and the actor that per-record ownership is scoped by, and the scoping is applied through the context rather than by each method remembering to add a `WHERE`. So a service method that uses `s.db(ctx)` is automatically scoped, and one that reaches for `s.DB` directly is not.

> **WATCH OUT**
> This is the strongest argument for threading the request's context all the way down rather than creating a fresh one in the middle. A `context.Background()` halfway through a request does not merely lose cancellation, it loses the caller's identity, and the query comes back unscoped: every organisation's rows, not just this one's.
>
> When a background job genuinely must act as nobody, there is an explicit way to say so. The explicitness is the point: an unscoped query should be a sentence someone wrote on purpose.

Errors, by design

The service returns the sentinels and typed errors from chapter eleven:

```
func (s *ContactService) GetByID(ctx context.Context,
    id uint) (*models.Contact, error) {
    var c models.Contact
```

```
    err := s.db(ctx).First(&c, id).Error
    if errors.Is(err, gorm.ErrRecordNotFound) {
        return nil, fmt.Errorf("contact %d: %w", id, respond.ErrNotFound)
    }
    if err != nil {
        return nil, fmt.Errorf("getting contact %d: %w", id, err)
    }
    return &c, nil
}
```

Note what it does **not** do: it does not know about status codes, and it does not log. It returns, and the boundary decides. That is chapter eleven's "handle an error once" as a structural rule rather than advice.

Adding your own method

This is the part people worry about. The generated service is a starting point, and adding to it is ordinary Go:

```
// InvoiceService methods you add live alongside the generated ones.
func (s *InvoiceService) MarkPaid(ctx context.Context,
    id uint, paidAt time.Time) (*models.Invoice, error) {

    inv, err := s.GetByID(ctx, id)      // reuse the generated method
    if err != nil {
        return nil, err
    }
    if inv.PaidAt != nil {
        return nil, fmt.Errorf("invoice %d: %w", id, ErrAlreadyPaid)
    }

    inv.PaidAt = &paidAt
    if err := s.db(ctx).Save(inv).Error; err != nil {
        return nil, fmt.Errorf("marking invoice %d paid: %w", id, err)
    }

    events.Emit(ctx, events.Event{
        Name: "invoices.paid", Resource: "invoices",
        ID: inv.ID, After: inv,
    })
    return inv, nil
}
```

Three things worth copying from that. It reuses `GetByID` rather than writing another query. It returns a sentinel for the business rule, so the handler can map it to 409. And it emits an event rather than calling the audit log, the webhook sender and the realtime pusher by hand.

WATCH OUT

Put it in the **same file** as the generated methods, not a separate one.

The manifest from chapter twenty nine hashes generated files, so an edited file is reported on a re-run rather than silently overwritten. That protection only applies to the file it knows about. A method in `invoice_custom.go` is invisible to that machinery, and the next person to regenerate will not be warned about anything.

Where a rule does not belong in a service

Two exceptions worth naming.

A rule about the shape of a request belongs in the validation layer: "email must look like an email" is a Zod rule and a binding tag, not a service method.

A rule that spans resources may belong in a saga rather than in either service, because neither owns the sequence. Marking an invoice paid, adjusting stock and sending a receipt is three services and a failure mode in the middle, which is what `internal/saga` exists for.

PROJECT 29: ADD A BUSINESS RULE PROPERLY

Take the bookshop from chapter twenty seven and add a real rule: a book cannot be discounted below its cost price.

```
grit generate field Book cost:money
grit generate field Book discount_percent:percent
grit migrate
```

Then add the method to `services/book.go`, below the generated ones:

```
var ErrBelowCost = errors.New("discount would price below cost")

func (s *BookService) ApplyDiscount(ctx context.Context,
    id uint, percent int) (*models.Book, error) {

    if percent < 0 || percent > 90 {
        return nil, fmt.Errorf("discount %d: %w", percent,
            respond.ErrValidation)
    }

    b, err := s.GetByID(ctx, id)
    if err != nil {
        return nil, err
    }

    final := b.Price.MinusPercent(percent)
    if final.Cents() < b.Cost.Cents() {
```

```
        return nil, fmt.Errorf("book %d: %w", id, ErrBelowCost)
    }

    b.DiscountPercent = percent
    if err := s.db(ctx).Save(b).Error; err != nil {
        return nil, fmt.Errorf("discounting book %d: %w", id, err)
    }
    return b, nil
}
```

Now prove the rule holds in more than one place, which is the point of the chapter. Write a test that calls it directly, with no HTTP:

```
func TestApplyDiscountRefusesBelowCost(t *testing.T) {
    s := newTestBookService(t)
    b, err := s.Create(context.Background(), CreateBook{
        Title: "Go", Price: money.FromCents(2000),
        Cost: money.FromCents(1500),
    })
    if err != nil {
        t.Fatal(err)
    }

    _, err = s.ApplyDiscount(context.Background(), b.ID, 50)
    if err == nil {
        t.Fatal("want an error for a discount below cost")
    } else if !errors.Is(err, ErrBelowCost) {
        t.Errorf("got %v, want ErrBelowCost", err)
    }

    _, err = s.ApplyDiscount(context.Background(), b.ID, 10)
    if err != nil {
        t.Errorf("a 10 percent discount should be allowed: %v", err)
    }
}
```

That test runs with `context.Background()` and no server, which only works because the rule lives in the service. Move the check into a handler and try to write the same test: you now need a request, a recorder and a router, and the CSV importer bypasses your rule entirely.

Finally, write a four-line `cmd/discount/main.go` that calls the same method from the command line, and watch the rule apply there too, for free.

YOUR TURN

Add the discount field to the admin by hand instead of with `grit generate field`: edit only the model. Then try to sort the Books table by it and patch it through the API. Both

fail silently, because the two maps in the service do not mention it. Fix them, and you will not forget what those whitelists are for again.

0

Build your own key-value store: a write-ahead log and crash recovery

A map in memory is a key-value store until the process restarts. This chapter builds one that survives a crash, which turns out to be a question about exactly one system call.

Everything here uses only the standard library, and everything you need is in Parts I to III.

The problem, stated precisely

```
m := map[string]string{}
m["user:1"] = "Ada"
// power cut
```

The data is gone. The obvious fix, writing the map to a file on every change, is both slow and **still wrong**: a crash halfway through writing leaves a corrupt file, and you have traded losing everything for losing everything in a less obvious way.

The standard answer is a **write-ahead log**. Before changing memory, append the change to a file. On startup, replay the file. Appending is fast because it is sequential, and a partially written record at the end is detectable and discardable.

> **IF YOU KNOW JAVASCRIPT**
> This is what Postgres, SQLite, Kafka and every serious database do. The WAL is the durable thing; the in-memory structure is a cache of it that can be rebuilt. Once you have written one, a great deal of database documentation stops being mysterious.

The record format

A log is a sequence of records. Each needs a length, a checksum and a payload:

```
+--------+----------+--------+---------+-------+---------+
| crc32  | keyLen   | valLen |   op    |  key  |  value  |
| 4 byte | 4 byte   | 4 byte | 1 byte  |  ...  |   ...   |
+--------+----------+--------+---------+-------+---------+
```

The checksum is what makes a half-written record detectable. Without it a truncated write looks like a valid record with garbage in it.

```
package kv

import (
    "bufio"
    "encoding/binary"
    "errors"
    "fmt"
    "hash/crc32"
    "io"
    "os"
    "sync"
)

type op byte

const (
    opSet op = 1
    opDel op = 2
)

const headerSize = 4 + 4 + 4 + 1   // crc, keyLen, valLen, op
```

Encoding a record

```
func encode(o op, key, val []byte) []byte {
    buf := make([]byte, headerSize+len(key)+len(val))

    binary.BigEndian.PutUint32(buf[4:8], uint32(len(key)))
    binary.BigEndian.PutUint32(buf[8:12], uint32(len(val)))
    buf[12] = byte(o)
    copy(buf[headerSize:], key)
    copy(buf[headerSize+len(key):], val)

    // The checksum covers everything after itself.
    sum := crc32.ChecksumIEEE(buf[4:])
    binary.BigEndian.PutUint32(buf[0:4], sum)
    return buf
}
```

`encoding/binary` writes fixed-width integers in a defined byte order. Big-endian is conventional for file formats; the only rule that matters is that the reader and writer agree.

WATCH OUT

> Do not use `fmt.Sprintf` or JSON for a log format. Both are variable-width, so you cannot know how many bytes to read without scanning for a delimiter, and a delimiter can appear inside a value. A length prefix removes the question entirely: read the header, then read exactly that many bytes.

The store

```
type Store struct {
    mu   sync.RWMutex
    data map[string][]byte

    log  *os.File
    w    *bufio.Writer
    path string
}

func Open(path string) (*Store, error) {
    f, err := os.OpenFile(path, os.O_RDWR|os.O_CREATE|os.O_APPEND, 0o600)
    if err != nil {
        return nil, fmt.Errorf("opening log %s: %w", path, err)
    }

    s := &Store{
        data: map[string][]byte{},
        log:  f,
        w:    bufio.NewWriter(f),
        path: path,
    }

    // Rebuild memory from the log before accepting any writes.
    if err := s.replay(); err != nil {
        f.Close()
        return nil, fmt.Errorf("replaying log: %w", err)
    }
    return s, nil
}

func (s *Store) Close() error {
    if err := s.w.Flush(); err != nil {
        s.log.Close()
        return fmt.Errorf("flushing: %w", err)
    }
    return s.log.Close()
}
```

`O_APPEND` matters: every write goes to the end regardless of where the file offset happens to be, which is what makes concurrent appends safe at the operating system level.

Replay, and tolerating a torn tail

```
func (s *Store) replay() error {
    if _, err := s.log.Seek(0, io.SeekStart); err != nil {
        return err
    }
    r := bufio.NewReader(s.log)
    header := make([]byte, headerSize)
    offset := int64(0)

    for {
        if _, err := io.ReadFull(r, header); err != nil {
            if errors.Is(err, io.EOF) {
                break                       // clean end
            }
            if errors.Is(err, io.ErrUnexpectedEOF) {
                break                       // torn header: stop here
            }
            return err
        }

        sum := binary.BigEndian.Uint32(header[0:4])
        keyLen := binary.BigEndian.Uint32(header[4:8])
        valLen := binary.BigEndian.Uint32(header[8:12])
        o := op(header[12])

        body := make([]byte, keyLen+valLen)
        if _, err := io.ReadFull(r, body); err != nil {
            break                           // torn body: stop here
        }

        // Verify before trusting any of it.
        check := crc32.ChecksumIEEE(append(header[4:], body...))
        if check != sum {
            break                           // corrupt: the log ends here
        }

        key := string(body[:keyLen])
        switch o {
        case opSet:
            s.data[key] = body[keyLen:]
        case opDel:
            delete(s.data, key)
        }
        offset += int64(headerSize) + int64(len(body))
    }

    // Discard anything after the last good record, so the next append does
    // not sit behind garbage.
    if err := s.log.Truncate(offset); err != nil {
```

```
        return fmt.Errorf("truncating to %d: %w", offset, err)
    }
    _, err := s.log.Seek(0, io.SeekEnd)
    return err
}
```

The three `break`s are the whole crash-recovery story. A torn header, a torn body and a failed checksum all mean the same thing: **the log ends at the last good record**, and everything after it was never acknowledged to a caller, so discarding it is correct.

> **WATCH OUT**
> `append(header[4:], body...)` is the slice trap from chapter six wearing a disguise. `header[4:]` has spare capacity, so `append` may write into `header` rather than allocating. It happens to be harmless here because `header` is re-read next iteration, but it is exactly the pattern that corrupts data elsewhere. The careful version:
>
> ```
> h := crc32.NewIEEE()
> h.Write(header[4:])
> h.Write(body)
> check := h.Sum32()
> ```
>
> Which is also faster, because it allocates nothing.

Writing

```
func (s *Store) Set(key string, val []byte) error {
    s.mu.Lock()
    defer s.mu.Unlock()

    if _, err := s.w.Write(encode(opSet, []byte(key), val)); err != nil {
        return fmt.Errorf("appending set: %w", err)
    }
    if err := s.sync(); err != nil {
        return err
    }

    // Memory is updated only after the log is durable. The other order
    // means a reader can see a value that a crash will erase.
    s.data[key] = val
    return nil
}

func (s *Store) Delete(key string) error {
    s.mu.Lock()
    defer s.mu.Unlock()
```

```
    if _, err := s.w.Write(encode(opDel, []byte(key), nil)); err != nil {
        return fmt.Errorf("appending delete: %w", err)
    }
    if err := s.sync(); err != nil {
        return err
    }
    delete(s.data, key)
    return nil
}

func (s *Store) Get(key string) ([]byte, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    v, ok := s.data[key]
    return v, ok
}
```

The ordering in `Set` is the correctness argument, and it is worth saying out loud: **log first, then memory**. If memory were updated first, a reader could observe a value that the crash removes, and the store would have lied.

The one system call that matters

```
func (s *Store) sync() error {
    if err := s.w.Flush(); err != nil {       // bufio -> kernel
        return fmt.Errorf("flushing: %w", err)
    }
    if err := s.log.Sync(); err != nil {      // kernel -> disk
        return fmt.Errorf("syncing: %w", err)
    }
    return nil
}
```

Two steps, and people routinely do only the first.

`Flush` moves bytes out of Go's buffer into the operating system. At that point the data survives your **process** crashing, because the kernel has it.

`Sync` is `fsync`, and it asks the kernel to put the bytes on the physical device. Only after this does the data survive the **machine** losing power.

> **WATCH OUT**
> `fsync` is slow: on spinning disks milliseconds, on an SSD still far slower than anything else in this program. Calling it on every write makes a store that does maybe a few thousand writes a second.

That is the durability dial, and every database exposes it. Postgres calls it `synchronous_commit`, Redis calls it `appendfsync`, and the options are always the same three:

every write	Slowest, loses nothing
every N ms	Fast, loses up to N milliseconds on a power cut
never	Fastest, the operating system decides, can lose a lot

There is no correct answer, only a choice about what you can afford to lose. A store that does not tell you which one it made is the one to distrust.

Compaction

The log grows forever. Setting one key a million times means a million records and a one-entry map.

```
// Compact writes a fresh log containing only the current state, then swaps
// it in atomically.
func (s *Store) Compact() error {
    s.mu.Lock()
    defer s.mu.Unlock()

    tmp := s.path + ".compact"
    f, err := os.OpenFile(tmp, os.O_RDWR|os.O_CREATE|os.O_TRUNC, 0o600)
    if err != nil {
        return fmt.Errorf("creating %s: %w", tmp, err)
    }

    w := bufio.NewWriter(f)
    for k, v := range s.data {
        if _, err := w.Write(encode(opSet, []byte(k), v)); err != nil {
            f.Close()
            os.Remove(tmp)
            return fmt.Errorf("writing compacted record: %w", err)
        }
    }
    if err := w.Flush(); err != nil {
        f.Close()
        os.Remove(tmp)
        return err
    }
    if err := f.Sync(); err != nil {        // durable before the swap
        f.Close()
        os.Remove(tmp)
        return err
    }
```

```
    f.Close()

    // Rename is atomic on the same filesystem: the log is either the old
    // one or the new one, never a half-written mixture.
    if err := os.Rename(tmp, s.path); err != nil {
        os.Remove(tmp)
        return fmt.Errorf("swapping log: %w", err)
    }

    s.log.Close()
    nf, err := os.OpenFile(s.path, os.O_RDWR|os.O_APPEND, 0o600)
    if err != nil {
        return fmt.Errorf("reopening log: %w", err)
    }
    s.log, s.w = nf, bufio.NewWriter(nf)
    return nil
}
```

`os.Rename` on the same filesystem is atomic, which is the trick that makes this safe. Write the new file completely, sync it, then swap. A crash at any point leaves either the old log or the new one, and both are valid.

IF YOU KNOW JAVASCRIPT

Write to a temp file, fsync, rename is **the** durable-replace pattern, and it applies far beyond databases. Any time you overwrite a file that matters, a configuration file or an export, this is the correct way to do it. Writing in place can leave a half-written file that nothing can read.

PROJECT 34: MAKE IT CRASH AND WATCH IT RECOVER

The tests are the chapter. Each proves a property the code was written for.

```
package kv

import (
    "os"
    "path/filepath"
    "testing"
)

func TestSurvivesReopen(t *testing.T) {
    path := filepath.Join(t.TempDir(), "test.log")

    s, err := Open(path)
    if err != nil {
        t.Fatal(err)
    }
```

```
    if err := s.Set("user:1", []byte("Ada")); err != nil {
        t.Fatal(err)
    }
    if err := s.Set("user:2", []byte("Grace")); err != nil {
        t.Fatal(err)
    }
    if err := s.Delete("user:1"); err != nil {
        t.Fatal(err)
    }
    s.Close()

    // Reopen: everything comes from the log.
    s2, err := Open(path)
    if err != nil {
        t.Fatal(err)
    }
    defer s2.Close()

    if _, ok := s2.Get("user:1"); ok {
        t.Error("deleted key came back")
    }
    v, ok := s2.Get("user:2")
    if !ok || string(v) != "Grace" {
        t.Errorf("user:2 = %q %v, want Grace true", v, ok)
    }
}

// The important one: a log cut off mid record must not break the store.
func TestTornTailIsDiscarded(t *testing.T) {
    path := filepath.Join(t.TempDir(), "torn.log")

    s, _ := Open(path)
    s.Set("a", []byte("one"))
    s.Set("b", []byte("two"))
    s.Close()

    // Simulate a crash during the third write by appending a partial
    // record: a header with no body behind it.
    f, _ := os.OpenFile(path, os.O_WRONLY|os.O_APPEND, 0o600)
    f.Write(encode(opSet, []byte("c"), []byte("three"))[:8])
    f.Close()

    s2, err := Open(path)
    if err != nil {
        t.Fatalf("a torn tail must not fail the open: %v", err)
    }
    defer s2.Close()

    if _, ok := s2.Get("c"); ok {
        t.Error("a half written record was replayed")
    }
```

```
    if v, ok := s2.Get("b"); !ok || string(v) != "two" {
        t.Error("records before the tear were lost")
    }

    // And the store must still be writable afterwards.
    if err := s2.Set("d", []byte("four")); err != nil {
        t.Fatalf("writing after recovery: %v", err)
    }
}

func TestCompactionKeepsState(t *testing.T) {
    path := filepath.Join(t.TempDir(), "compact.log")
    s, _ := Open(path)

    for i := 0; i < 1000; i++ {
        s.Set("counter", []byte("value"))
    }
    before, _ := os.Stat(path)

    if err := s.Compact(); err != nil {
        t.Fatal(err)
    }
    after, _ := os.Stat(path)

    if after.Size() >= before.Size() {
        t.Errorf("compaction did not shrink: %d -> %d",
            before.Size(), after.Size())
    }
    if v, ok := s.Get("counter"); !ok || string(v) != "value" {
        t.Error("compaction lost the data")
    }
    s.Close()

    s2, _ := Open(path)
    defer s2.Close()
    if _, ok := s2.Get("counter"); !ok {
        t.Error("compacted log did not replay")
    }
}
```

```
go test -race -v ./...
```

`TestTornTailIsDiscarded` is the one to run first. It writes eight bytes of a record that should have been twenty, which is exactly what a power cut mid write leaves behind, and asserts three things: the open succeeds, the partial record is not replayed, and the store still accepts writes. A store that fails any of those will one day refuse to start, and refusing to start is how a durable store loses data.

YOUR TURN

Three extensions, each teaching something different.

Add a `syncEvery` option so a caller can choose the durability dial, and benchmark `Set` with sync on every write against sync every 100ms. The ratio you measure is the reason databases have that setting.

Add automatic compaction when the log exceeds a multiple of the live data size, and think about what happens if the process dies during compaction. The rename makes it safe; prove it with a test that leaves a stale `.compact` file behind.

Then replace the map with the B-tree from the next chapter and watch the store gain ordered range queries, which is the step from a key-value store to something you could call a database.